Document number: P0355R7

Howard E. Hinnant
Tomasz Kamiński
2018-03-16

Extending <chrono> to Calendars and Time Zones

Contents

Revision History

Changes since R5

- Miscellaneous editorial changes.
- Un-deprecate `system_clock::to_time_t` and `system_clock::from_time_t`.
- Rename `std::literals::chrono_literals::sun` to `std::chrono::Sunday` et al.

Changes since R4

- Make the parsing of minutes optional under the flags `%z`, `%Ez`, and `%Oz`.
- Replace the family of overloaded functions `to_sys_time`, `to_utc_time`, `to_tai_time`, `to_gps_time`, `to_file_time`, with `clock_cast`.
- Replace undefined behavior with unspecified behavior in [time.calendar].
- Allow `zoned_time` conversion among different `TimeZonePtr` types.
- Specify the constructors for `nonexistent_local_time` and `ambiguous_local_time`.

Changes since R3

- Presume that the time zone database is supplied only by the std::lib implementation, but can be updated at run time. Removed `remote_download` and `remote_install`. This functionality is now subsumed by the implementation.
- `to_stream` sets `failbit` if it is required to create a name for an invalid month or weekday.
- Add note to promise compatibility with `fmt` (P0645)
- Have `format` throw an exception if anything happens so that it could not return an accurate string.
- Template `zoned_time` on `TimeZonePtr`.
- Give `weekday_indexed` a defaulted default constructor.
- Make `from_stream` and `to_stream` customization points.
- Make the database singleton a singly linked list of `tzdb` with an atomic head pointer, instead of a single `tzdb`.
- Rewrite in terms of `string_view`.
- Improve spec for operator-(const year_month& x, const year_month& y).
- Refine constraints on conversions from calendar types to sys_days.
- Added `zoned_time` default constructor.
- Added `zoned_time` deduction guides.
- Correct minor type-o's.
- Correct html bugs.

Changes since R2

- Add `to_stream` and `from_stream` to `utc_time<Duration>`, `tai_time<Duration>`, and `gps_time<Duration>`.
- Add `from_stream`, and rewrite `format` and `parse` in terms of `to_stream` and `from_stream`.
- Add `to_stream` and `from_stream` for `year`, `month`, `day`, `weekday`, `year_month`, and `month_day`.
- `to_stream` will set `failbit` instead of `throw`.
- Add `file_clock` and hook it into `filesytem`.
- Relax `zoned_time` to be coarser than seconds.
- Remove `make_time` and `make_zoned` in favor of the implicit deduction guides.
- Create `zoned_time(const char* name, ...)` overloads to enable implicit deduction guides.
- Give `time_of_day` default constructor.
- Add `Alloc` to `basic_string` everywhere possible.
- Deprecate `system_clock::to_time_t` and `system_clock::from_time_t`.

- Make duration streaming respect padding and alignment requests.
- Add `is_clock` type trait.
- Allow the Clock template parameter of `time_point` to be a Clock or a `local_t`.
- Make functions in [thread] that take a Clock template parameter ill-formed if `is_clock{}` is false.

Changes since R1

- Make `time_point` incremental and decrementable
- Add unary operators + and - to `year`
- Remove `enum {am, pm}`, `time_of_day` constructors which use it, and `make_time` factory functions which use it.
- Add `to_stream`.
- Add `format` and `parse` for `duration`.

Changes since R0

- Tighten up `utc_clock::utc_to_sys` and `utc_clock::sys_to_utc`.
- Eliminate `utc_clock::utc_to_sys` and `utc_clock::sys_to_utc` in favor of free functions such as `to_sys_time` and `to_utc_time`.
- Give `utc_time` a streaming operator.
- Change `format` to take `time_point`s by `const&` instead of by value.
- Add trivial default constructors to most calendar types.
- Create %Ez & %Oz to put ':' in offset for format and parse.
- Add %F to parse.
- Changed parse functions to parse manipulators.
- Excuse local_t from being a clock
- Guarantee Unix Time.
- Add duration stream insertion.
- Introduce tai_clock.
- Introduce gps_clock.
- Removed `noexcept` from `make_time`.

Introduction

The purpose of a calendar is to give a name to each day.[1] There are many different ways this can be accomplished. This paper proposes only the Gregorian calendar. However the design of this proposal is such that clients can code other calendars and have them interoperate with `<chrono>`, the civil calendar, and with time zones, all with a minimal coupling. For example:

```
#include "coptic.h"  // not proposed, just an example
#include <chrono>
#include <iostream>

int
main()
{
    using namespace std::chrono;
    auto date = 2016y/May/29;
    cout << date << " is " << coptic::year_month_day{date} << " in the Coptic calendar\n";
    // 2016-05-29 is 1732-09-21 in the Coptic calendar
}
```

The above example creates a date in the Gregorian calendar (proposed) with the literal `2016y/May/29`. The meaning of this literal is without question. It is conventional and clearly readable. This proposal has no knowledge whatsoever of the Coptic calendar. However it is relatively easy to create a Coptic calendar (which knows nothing about the Gregorian calendar), which will convert to and from the Gregorian calendar. This is done by establishing a clear and simple communication channel between calendar systems and the `<chrono>` library (specifically a `system_clock::time_point` with a precision of days).

The paper proposes:

1. Minimal extensions to `<chrono>` to support calendar and time zone libraries.
2. A proleptic Gregorian calendar, hereafter referred to as the *civil* calendar.
3. A time zone library based on the IANA Time Zone Database.
4. `strftime`-like formatting and parsing facilities with fully operational support for fractional seconds, time zone abbreviations, and UTC offsets.
5. Several `<chrono>` clocks for computing with leap seconds which is also supported by the IANA Time Zone Database.

Everything proposed herein has been fully implemented here:

> https://github.com/HowardHinnant/date

The implementation includes full documentation, and an active community of users with positive field experience. The implementation has been ported to Windows, Linux, Android, macOS and iOS.

The API stresses:

- Seamless integration with the existing `<chrono>` library.
- Type safety.
- Detection of errors at compile time.
- Performance.
- Ease of use.
- Readable code.
- No artificial restrictions on precision. Note: current financial software is currently in the middle of a transition from seconds precision to milliseconds precision. This library handles that transition as seamlessly as `<chrono>` does.

Listing "Performance" in the API design deserves a little explanation as one usually thinks of that as an implementation issue. Think of it this way:

- If `vector<T>::push_front(const T&)` existed, that would encourage inefficient code, even though it would be trivial to implement.
- If `list<T>::operator[](size_type index)` existed, that would encourage inefficient code, even though it would be trivial to implement (by incrementing from `begin()` or decrementing from `end()`).

This API makes it convenient to write efficient code, and inconvenient to write inefficient code. It turns out that conversion between a field type such as `{year, month, day}` and a serial type such as `{count-of-days}` is one of the more expensive operations when dealing with calendrical computations. Both data structures are very useful (just as both `vector` and `list` are very useful). So this library puts *you* in control of when and how often that conversion takes place, and makes it easy to avoid such conversions when not necessary.

Description

One can create a `year` like this:

```
auto y = year{2016};
```

Just like `<chrono>`, type safety is taken very seriously. The type `year` is distinct from type `int`, just as `3` can never mean "3 seconds", unless it is *explicitly* typed to do so: `seconds{3}`.

And just like `seconds`, there is a year literal suffix which can help make your code more readable:

```
auto y = 2016y;
```

`year` is a *partial-calendar-type*. It can be combined with other partial-calendar-types to create a *full-calendar-type* such as `year_month_day`. Full-calendar-types can be converted to and from the family of `system_clock::time_point`s. Full-calendar-types such as `year_month_day` are time points with a precision of a day, but they are also *field* types. They are composed of 3 fields under the hood: `year`, `month` and `day`. Thus when you construct a `year_month_day` from a `year`, `month` and `day`, absolutely no computation takes place. The only thing that happens is a `year`, `month` and `day` are stored inside the `year_month_day`.

```
year_month_day ymd1{2016y, month{5}, day{29}};
```

This is a *very* simple operation and can even be made `constexpr` when all of the inputs are compile-time constants. And *conventional syntax* is available which means the exact same thing, with the same run-time or compile-time performance. It can make date literals much more readable without sacrificing type safety:

```
constexpr year_month_day ymd1{2016y, month{5}, day{29}};
constexpr auto ymd2 = 2016y/May/29d;
static_assert(ymd1         == ymd2);
static_assert(ymd1.year()  == 2016y);
static_assert(ymd1.month() == May);
static_assert(ymd1.day()   == 29d);
```

`year_month_day` is a *very* simple, *very* understandable *calendrical* data structure:

```
class year_month_day
{
    chrono::year  y_;  // exposition only
    chrono::month m_;  // exposition only
    chrono::day   d_;  // exposition only

public:
    constexpr year_month_day(const chrono::year& y, const chrono::month& m, const chrono::day& d) noexcept;
    // ...
```

By now you should be yawning and muttering "so what?"

Now we introduce a little `<chrono>` infrastructure that serves as the communication channel with simplistic calendrical data structures such as `year_month_day`.

```
using days = duration<int32_t, ratio_multiply<ratio<24>, hours::period>>;
template <class Duration> using sys_time = time_point<system_clock, Duration>;
using sys_days = sys_time<days>;
```

`sys_days` **is** a `std::chrono::time_point`. This `time_point` is based on `system_clock` and has a very coarse precision: 24 hours. Just as `system_clock::time_point` is nothing more than a count of microseconds (or nanoseconds, or whatever), `sys_days` is simply a count of days since the `system_clock` epoch. And `sys_days` is *fully* interoperable with `system_clock::time_point` in all of the ways normal to the `<chrono>`

library:

- `sys_days` implicitly converts to `system_clock::time_point` with no truncation error.
- `system_clock::time_point` *does not* implicitly convert to `sys_days` because it would involve truncation error.
- Explicit conversion can be achieved from `system_clock::time_point` by using the existing `<chrono>` facilities `time_point_cast` or `floor`.

```
constexpr system_clock::time_point tp = sys_days{2016y/May/29d}; // Convert date to time_point
static_assert(tp.time_since_epoch() == 1'464'480'000'000'000us);
constexpr auto ymd = year_month_day{floor<days>(tp)};            // Convert time_point to date
static_assert(ymd == 2016y/May/29d);
```

The calendrical type `year_month_day` provides conversions to and from `sys_days`. This conversion is easy to do for std::lib implementors using algorithms such as these. If the committee standardizes existing practice and specifies that `system_clock` measures Unix Time, then it will be equally easy for anyone to write their own calendar system which converts to and from `sys_days` (e.g. the coptic example in the introduction).

This proposal actually contains a second calendar. It is so closely related to the civil calendar that we normally don't think of it as another calendar. We often refer to dates like "the 5th Sunday of May in 2016" as opposed to "the 29th of May in 2016." This proposal makes it so easy to build fully functional calendars that interoperate with `system_clock::time_point`, that it is nearly trivial to include such functionality:

```
constexpr system_clock::time_point tp = sys_days{Sunday[5]/May/2016}; // Convert date to time_point
static_assert(tp.time_since_epoch() == 1'464'480'000'000'000us);
constexpr auto ymd = year_month_weekday{floor<days>(tp)};          // Convert time_point to date
static_assert(ymd == Sunday[5]/May/2016);
```

The literal `Sunday[5]/May/2016` means "the 5th Sunday of May in 2016." The *conventional* syntax is remarkably readable. Constructor syntax is also available to do the same thing. The type constructed is `year_month_weekday` which does nothing but store a `year`, `month`, `weekday`, and the number 5. This "auxiliary calendar" converts to and from `sys_days` just like `year_month_day` as demonstrated above. As such, `year_month_weekday` will interoperate with `year_month_day` (by bouncing off of `sys_days`) just as it will with any other calendar that interoperates with `sys_days`:

```
static_assert(2016y/May/29d == year_month_day{Sunday[5]/May/2016});
```

Since `year_month_day` is so easy to convert to (or from) a `time_point` it makes sense to convert to a `time_point` when you need to talk about a date and time-of-day:

```
constexpr auto tp = sys_days{2016y/May/29d} + 7h + 30min; // 2016-05-29 07:30 UTC
```

The time zone is implicitly UTC because `system_clock` tracks Unix Time which is (a very close approximation to) UTC. If you need another time zone, no worries, we'll get there. And remember, `tp` above is a `system_clock::time_point`, except with minutes precision. You can compare it with `system_clock::now()` to find out if the date is in the past or the future. Also note that the syntax above (like `<chrono>`) is *precision neutral*. That's because the syntax above **is** `<chrono>`, except for the part converting a calendar type into the `<chrono>` system. If you suddenly need to convert your minutes-precision time point into seconds or milliseconds (or whatever) precision, the change is seamlessly handled by the existing `<chrono>` system:

```
constexpr auto tp = sys_days{2016y/May/29d} + 7h + 30min + 6s + 153ms; // 2016-05-29 07:30:06.153 UTC
```

Simple streaming is provided:

```
cout << tp << '\n';  // 2016-05-29 07:30:06.153
```

But I need the time in Tokyo!

```
auto tp = sys_days{2016y/May/29d} + 7h + 30min + 6s + 153ms; // 2016-05-29 07:30:06.153 UTC
zoned_time zt = {"Asia/Tokyo", tp};
cout << zt << '\n';                                          // 2016-05-29 16:30:06.153 JST
```

`zoned_time` is templated on the duration type of `tp`, which is automatically deduced from the initialization expression (milliseconds in this example). This effectively pairs a time zone with a time point. In this example we pair the time zone "Asia/Tokyo" with a `sys_time` (which is implicitly UTC). When printed out, you see the local time, and by default the current time zone abbreviation. Also by default, you see the full precision of the `zoned_time`.

Sometimes, instead of specifying the time in UTC as above, it is convenient to specify the time in terms of the local time of the time zone. It is very easy to change the above example to mean 7:30 JST instead of 7:30 UTC:

```
auto tp = local_days{2016y/May/29d} + 7h + 30min + 6s + 153ms; // 2016-05-29 07:30:06.153
auto zt = zoned_time{"Asia/Tokyo", tp};
cout << zt << '\n';                                            // 2016-05-29 07:30:06.153 JST
```

The *only* change to the code is the use of `local_days` in place of `sys_days`. `local_days` is also a `std::chrono::time_point` but its "clock type" `local_t` has no `now()` function. This `time_point` is called `local_time`. A `local_time` can refer to *any* time zone. In the above example when we pair "Asia/Tokyo" with the `local_time`, the result becomes a `zoned_time` with the local time specified by the `local_time`.

To interoperate with time zones, calendrical types must convert to and from `local_days` as well as `sys_days`. The math is identical for both conversions, so it is very easy for the calendar author to provide. But as seen in this example, the meaning can be quite different.

The client of the calendar library can easily use the calendar types with the time zone library, specifying times either in the local time, or in

UTC, simply by switching between `local_days` and `sys_days`. Here is an example that sets up a meeting at 9am on the third Tuesday of June, 2016 in New York:

```
auto zt = zoned_time{"America/New_York", local_days{Tuesday[3]/June/2016} + 9h};
cout << zt << '\n';  // 2016-06-21 09:00:00 EDT
```

Need to set up a video conference with your partners in Helsinki?

```
cout << zoned_time{"Europe/Helsinki", zt} << '\n';
```

This converts one `zoned_time` into another `zoned_time` where the only difference is changing from "America/New_York" to "Europe/Helsinki". The conversion preserves the UTC equivalent in both `zoned_time`s, and therefore outputs:

```
2016-06-21 16:00:00 EEST
```

And if this is not the formatting you prefer, that is easily fixed too:

```
cout << format("%F %H:%M %z", zoned_time{"Europe/Helsinki", zt}) << '\n';
// 2016-06-21 16:00 +0300
```

Or perhaps properly localized:

```
cout << format(locale{"fi_FI"}, "%c", zoned_time{"Europe/Helsinki", zt}) << '\n';
// Ti 21 Kes 16:00:00 2016
```

Wait, slow down, this is too much information! Let's start at the beginning. How do I get the current time?

```
cout << system_clock::now() << " UTC\n";
// 2016-05-30 17:57:30.694574 UTC
```

My current local time?

```
cout << zoned_time{current_zone(), system_clock::now()} << '\n';
// 2016-05-30 13:57:30.694574 EDT
```

Current time in Budapest?

```
cout << zoned_time{"Europe/Budapest", system_clock::now()} << '\n';
// 2016-05-30 19:57:30.694574 CEST
```

For more documentation about the calendar portion of this proposal, including more details, more examples, and performance analyses, please see:

> http://howardhinnant.github.io/date/date.html

For a video introduction to the calendar portion, please see:

> https://www.youtube.com/watch?v=tzyGjOm8AKo

For a video introduction to the time zone portion, please see:

> https://www.youtube.com/watch?v=Vwd3pduVGKY

For more documentation about the time zone portion of this proposal, including more details, and more examples, please see:

> http://howardhinnant.github.io/date/tz.html

For more examples, some of which are written by users of this library, please see:

> https://github.com/HowardHinnant/date/wiki/Examples-and-Recipes

For another example calendar which models the ISO week-based calendar, please see:

> http://howardhinnant.github.io/date/iso_week.html

Proposed Wording

Table of Contents for Proposed Wording

[time.duration.arithmetic] 23.17.5.3
[time.duration.special] 23.17.5.4
[time.duration.nonmember] 23.17.5.5
[time.duration.comparisons] 23.17.5.6
[time.duration.cast] 23.17.5.7
[time.duration.literals] 23.17.5.8
[time.duration.alg] 23.17.5.9
[time.duration.io]
[time.point] 23.17.6
[time.point.cons] 23.17.6.1
[time.point.observer] 23.17.6.2
[time.point.arithmetic] 23.17.6.3
[time.point.special] 23.17.6.4
[time.point.nonmember] 23.17.6.5
[time.point.comparisons] 23.17.6.6
[time.point.cast] 23.17.6.7
[time.clock] 23.17.7
[time.clock.system] 23.17.7.1
[time.clock.utc]
[time.clock.tai]
[time.clock.gps]
[time.clock.file]
[time.clock.steady] 23.17.7.2
[time.clock.hires] 23.17.7.3
[time.clock.local_time]
[time.clock.clock_cast]
[time.format]
[time.parse]
[time.calendar]
[time.calendar.last]
[time.calendar.day]
[time.calendar.month]
[time.calendar.year]
[time.calendar.weekday]
[time.calendar.weekday_indexed]
[time.calendar.weekday_last]
[time.calendar.month_day]
[time.calendar.month_day_last]
[time.calendar.month_weekday]
[time.calendar.month_weekday_last]
[time.calendar.year_month]
[time.calendar.year_month_day]
[time.calendar.year_month_day_last]
[time.calendar.year_month_weekday]
[time.calendar.year_month_weekday_last]
[time.calendar.operators]
[time.time_of_day]
[time.timezone]
[time.timezone.database]
[time.timezone.database.remote]
[time.timezone.exception]
[time.timezone.info]
[time.timezone.time_zone]
[time.timezone.zoned_traits]
[time.timezone.zoned_time]
[time.timezone.leap]
[time.timezone.link]
[ctime.syn] 23.17.8

[fs.filesystem.syn] 30.10.6

[thread.req.paramname] 33.2.1

Text in grey boxes is not proposed wording.

Insert into synopsis in 23.17.2 Header <chrono> synopsis [time.syn]:

```
namespace std {
namespace chrono {

// ...
// customization traits
// ...

template <class T> struct is_clock;
template <class T> inline constexpr bool is_clock_v = is_clock<T>::value;

// duration I/O
template <class charT, class traits, class Rep, class Period>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os,
               const duration<Rep, Period>& d);

template <class charT, class traits, class Rep, class Period>
basic_ostream<charT, traits>&
```

```
to_stream(basic_ostream<charT, traits>& os, const charT* fmt,
          const duration<Rep, Period>& d);

template <class charT, class traits, class Rep, class Period, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            duration<Rep, Period>& d,
            basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);

// ...
// convenience typedefs
// ...
using days   = duration<signed integer type of at least 25 bits, ratio_multiply<ratio<24>, hours::period>>;
using weeks  = duration<signed integer type of at least 22 bits, ratio_multiply<ratio<7>, days::period>>;
using years  = duration<signed integer type of at least 17 bits, ratio_multiply<ratio<146097, 400>, days::period>>;
using months = duration<signed integer type of at least 20 bits, ratio_divide<years::period, ratio<12>>>;

// ...
// clocks
// ...
class utc_clock;
class tai_clock;
class gps_clock;
class file_clock;

// time_point families
template <class Duration>
    using sys_time = time_point<system_clock, Duration>;
using sys_seconds  = sys_time<seconds>;
using sys_days     = sys_time<days>;

struct local_t {};
template <class Duration>
    using local_time = time_point<local_t, Duration>;
using local_seconds  = local_time<seconds>;
using local_days     = local_time<days>;

template <class Duration>
    using utc_time = time_point<utc_clock, Duration>;
using utc_seconds  = utc_time<seconds>;

template <class Duration>
    using tai_time = time_point<tai_clock, Duration>;
using tai_seconds  = tai_time<seconds>;

template <class Duration>
    using gps_time = time_point<gps_clock, Duration>;
using gps_seconds  = gps_time<seconds>;

template <class Duration>
    using file_time = time_point<file_clock, Duration>;

// time_point conversions

template <class DestClock, class SourceClock> struct clock_time_conversion;

template <class DestClock, class SourceClock, class Duration>
    time_point<DestClock, see below>
    clock_cast(const time_point<SourceClock, Duration>& t);

// time_point I/O

// operator<<
template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const sys_time<Duration>& tp);

template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const local_time<Duration>& tp);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const sys_days& dp);

template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const utc_time<Duration>& t);

template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const tai_time<Duration>& t);

template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
```

```
    operator<<(basic_ostream<charT, traits>& os, const gps_time<Duration>& t);

template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const file_time<Duration>& tp);

// to_stream
template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const sys_time<Duration>& tp);

template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const local_time<Duration>& tp,
              const string* abbrev = nullptr, const seconds* offset_sec = nullptr);

template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const utc_time<Duration>& tp);

template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const tai_time<Duration>& tp);

template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const gps_time<Duration>& tp);

template <class charT, class traits, class Duration>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const file_time<Duration>& tp);

// from_stream
template <class charT, class traits, class Duration, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                sys_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

template <class charT, class traits, class Duration, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                local_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

template <class charT, class traits, class Duration, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                utc_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

template <class charT, class traits, class Duration, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                tai_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

template <class charT, class traits, class Duration, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                gps_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

template <class charT, class traits, class Duration, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                file_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

// Calendrical types

struct last_spec;

class day;

constexpr bool operator==(const day& x, const day& y) noexcept;
constexpr bool operator!=(const day& x, const day& y) noexcept;
constexpr bool operator< (const day& x, const day& y) noexcept;
constexpr bool operator> (const day& x, const day& y) noexcept;
constexpr bool operator<=(const day& x, const day& y) noexcept;
constexpr bool operator>=(const day& x, const day& y) noexcept;

constexpr day  operator+(const day&  x, const days& y) noexcept;
constexpr day  operator+(const days& x, const day&  y) noexcept;
constexpr day  operator-(const day&  x, const days& y) noexcept;
constexpr days operator-(const day&  x, const day&  y) noexcept;
```

```
template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const day& d);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const day& d);

template <class charT, class traits, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                day& d, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

class month;

constexpr bool operator==(const month& x, const month& y) noexcept;
constexpr bool operator!=(const month& x, const month& y) noexcept;
constexpr bool operator< (const month& x, const month& y) noexcept;
constexpr bool operator> (const month& x, const month& y) noexcept;
constexpr bool operator<=(const month& x, const month& y) noexcept;
constexpr bool operator>=(const month& x, const month& y) noexcept;

constexpr month  operator+(const month&  x, const months& y) noexcept;
constexpr month  operator+(const months& x,  const month& y) noexcept;
constexpr month  operator-(const month&  x, const months& y) noexcept;
constexpr months operator-(const month&  x,  const month& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month& m);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const month& m);

template <class charT, class traits, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                month& m, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

class year;

constexpr bool operator==(const year& x, const year& y) noexcept;
constexpr bool operator!=(const year& x, const year& y) noexcept;
constexpr bool operator< (const year& x, const year& y) noexcept;
constexpr bool operator> (const year& x, const year& y) noexcept;
constexpr bool operator<=(const year& x, const year& y) noexcept;
constexpr bool operator>=(const year& x, const year& y) noexcept;

constexpr year  operator+(const year&  x, const years& y) noexcept;
constexpr year  operator+(const years& x, const year&  y) noexcept;
constexpr year  operator-(const year&  x, const years& y) noexcept;
constexpr years operator-(const year&  x, const year&  y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year& y);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const year& y);

template <class charT, class traits, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                year& y, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

class weekday;

constexpr bool operator==(const weekday& x, const weekday& y) noexcept;
constexpr bool operator!=(const weekday& x, const weekday& y) noexcept;

constexpr weekday operator+(const weekday& x, const days&    y) noexcept;
constexpr weekday operator+(const days&    x, const weekday& y) noexcept;
constexpr weekday operator-(const weekday& x, const days&    y) noexcept;
constexpr days    operator-(const weekday& x, const weekday& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const weekday& wd);

template <class charT, class traits>
```

```
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const weekday& wd);

template <class charT, class traits, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                weekday& wd, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

class weekday_indexed;

constexpr bool operator==(const weekday_indexed& x, const weekday_indexed& y) noexcept;
constexpr bool operator!=(const weekday_indexed& x, const weekday_indexed& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const weekday_indexed& wdi);

class weekday_last;

constexpr bool operator==(const weekday_last& x, const weekday_last& y) noexcept;
constexpr bool operator!=(const weekday_last& x, const weekday_last& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const weekday_last& wdl);

class month_day;

constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
constexpr bool operator!=(const month_day& x, const month_day& y) noexcept;
constexpr bool operator< (const month_day& x, const month_day& y) noexcept;
constexpr bool operator> (const month_day& x, const month_day& y) noexcept;
constexpr bool operator<=(const month_day& x, const month_day& y) noexcept;
constexpr bool operator>=(const month_day& x, const month_day& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_day& md);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const month_day& md);

template <class charT, class traits, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                month_day& md, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

class month_day_last;

constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator!=(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator< (const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator> (const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator<=(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator>=(const month_day_last& x, const month_day_last& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_day_last& mdl);

class month_weekday;

constexpr bool operator==(const month_weekday& x, const month_weekday& y) noexcept;
constexpr bool operator!=(const month_weekday& x, const month_weekday& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_weekday& mwd);

class month_weekday_last;

constexpr bool operator==(const month_weekday_last& x, const month_weekday_last& y) noexcept;
constexpr bool operator!=(const month_weekday_last& x, const month_weekday_last& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_weekday_last& mwdl);

class year_month;

constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
constexpr bool operator!=(const year_month& x, const year_month& y) noexcept;
constexpr bool operator< (const year_month& x, const year_month& y) noexcept;
```

```
constexpr bool operator> (const year_month& x, const year_month& y) noexcept;
constexpr bool operator<=(const year_month& x, const year_month& y) noexcept;
constexpr bool operator>=(const year_month& x, const year_month& y) noexcept;

constexpr year_month operator+(const year_month& ym, const months& dm) noexcept;
constexpr year_month operator+(const months& dm, const year_month& ym) noexcept;
constexpr year_month operator-(const year_month& ym, const months& dm) noexcept;
constexpr months operator-(const year_month& x, const year_month& y) noexcept;
constexpr year_month operator+(const year_month& ym, const years& dy) noexcept;
constexpr year_month operator+(const years& dy, const year_month& ym) noexcept;
constexpr year_month operator-(const year_month& ym, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month& ym);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const year_month& ym);

template <class charT, class traits, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                year_month& ym, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

class year_month_day;

constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator!=(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator< (const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator> (const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator<=(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator>=(const year_month_day& x, const year_month_day& y) noexcept;

constexpr year_month_day operator+(const year_month_day& ymd, const months& dm) noexcept;
constexpr year_month_day operator+(const months& dm, const year_month_day& ymd) noexcept;
constexpr year_month_day operator+(const year_month_day& ymd, const years& dy) noexcept;
constexpr year_month_day operator+(const years& dy, const year_month_day& ymd) noexcept;
constexpr year_month_day operator-(const year_month_day& ymd, const months& dm) noexcept;
constexpr year_month_day operator-(const year_month_day& ymd, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_day& ymd);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const year_month_day& ymd);

template <class charT, class traits, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                year_month_day& ymd, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);

class year_month_day_last;

constexpr bool operator==(const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator!=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator< (const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator> (const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator<=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator>=(const year_month_day_last& x, const year_month_day_last& y) noexcept;

constexpr year_month_day_last operator+(const year_month_day_last& ymdl, const months& dm) noexcept;
constexpr year_month_day_last operator+(const months& dm, const year_month_day_last& ymdl) noexcept;
constexpr year_month_day_last operator+(const year_month_day_last& ymdl, const years& dy) noexcept;
constexpr year_month_day_last operator+(const years& dy, const year_month_day_last& ymdl) noexcept;
constexpr year_month_day_last operator-(const year_month_day_last& ymdl, const months& dm) noexcept;
constexpr year_month_day_last operator-(const year_month_day_last& ymdl, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_day_last& ymdl);

class year_month_weekday;

constexpr bool operator==(const year_month_weekday& x, const year_month_weekday& y) noexcept;
constexpr bool operator!=(const year_month_weekday& x, const year_month_weekday& y) noexcept;

constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const months& dm) noexcept;
constexpr year_month_weekday operator+(const months& dm, const year_month_weekday& ymwd) noexcept;
constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const years& dy) noexcept;
constexpr year_month_weekday operator+(const years& dy, const year_month_weekday& ymwd) noexcept;
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const months& dm) noexcept;
```

```
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_weekday& ymwdi);

class year_month_weekday_last;

constexpr bool operator==(const year_month_weekday_last& x, const year_month_weekday_last& y) noexcept;
constexpr bool operator!=(const year_month_weekday_last& x, const year_month_weekday_last& y) noexcept;

constexpr year_month_weekday_last operator+(const year_month_weekday_last& ymwdl, const months& dm) noexcept;
constexpr year_month_weekday_last operator+(const months& dm, const year_month_weekday_last& ymwdl) noexcept;
constexpr year_month_weekday_last operator+(const year_month_weekday_last& ymwdl, const years& dy) noexcept;
constexpr year_month_weekday_last operator+(const years& dy, const year_month_weekday_last& ymwdl) noexcept;
constexpr year_month_weekday_last operator-(const year_month_weekday_last& ymwdl, const months& dm) noexcept;
constexpr year_month_weekday_last operator-(const year_month_weekday_last& ymwdl, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_weekday_last& ymwdl);

// civil calendar conventional syntax operators
constexpr year_month operator/(const year& y, const month& m) noexcept;
constexpr year_month operator/(const year& y, int   m) noexcept;
constexpr month_day operator/(const month& m, const day& d) noexcept;
constexpr month_day operator/(const month& m, int d) noexcept;
constexpr month_day operator/(int m, const day& d) noexcept;
constexpr month_day operator/(const day& d, const month& m) noexcept;
constexpr month_day operator/(const day& d, int m) noexcept;
constexpr month_day_last operator/(const month& m, last_spec) noexcept;
constexpr month_day_last operator/(int m, last_spec) noexcept;
constexpr month_day_last operator/(last_spec, const month& m) noexcept;
constexpr month_day_last operator/(last_spec, int m) noexcept;
constexpr month_weekday operator/(const month& m, const weekday_indexed& wdi) noexcept;
constexpr month_weekday operator/(int m, const weekday_indexed& wdi) noexcept;
constexpr month_weekday operator/(const weekday_indexed& wdi, const month& m) noexcept;
constexpr month_weekday operator/(const weekday_indexed& wdi, int m) noexcept;
constexpr month_weekday_last operator/(const month& m, const weekday_last& wdl) noexcept;
constexpr month_weekday_last operator/(int m, const weekday_last& wdl) noexcept;
constexpr month_weekday_last operator/(const weekday_last& wdl, const month& m) noexcept;
constexpr month_weekday_last operator/(const weekday_last& wdl, int m) noexcept;
constexpr year_month_day operator/(const year_month& ym, const day& d) noexcept;
constexpr year_month_day operator/(const year_month& ym, int d) noexcept;
constexpr year_month_day operator/(const year& y, const month_day& md) noexcept;
constexpr year_month_day operator/(int y, const month_day& md) noexcept;
constexpr year_month_day operator/(const month_day& md, const year& y) noexcept;
constexpr year_month_day operator/(const month_day& md, int y) noexcept;
constexpr year_month_day_last operator/(const year_month& ym, last_spec) noexcept;
constexpr year_month_day_last operator/(const year& y, const month_day_last& mdl) noexcept;
constexpr year_month_day_last operator/(int y, const month_day_last& mdl) noexcept;
constexpr year_month_day_last operator/(const month_day_last& mdl, const year& y) noexcept;
constexpr year_month_day_last operator/(const month_day_last& mdl, int y) noexcept;
constexpr year_month_weekday operator/(const year_month& ym, const weekday_indexed& wdi) noexcept;
constexpr year_month_weekday operator/(const year& y, const month_weekday& mwd) noexcept;
constexpr year_month_weekday operator/(int y, const month_weekday& mwd) noexcept;
constexpr year_month_weekday operator/(const month_weekday& mwd, const year& y) noexcept;
constexpr year_month_weekday operator/(const month_weekday& mwd, int y) noexcept;
constexpr year_month_weekday_last operator/(const year_month& ym, const weekday_last& wdl) noexcept;
constexpr year_month_weekday_last operator/(const year& y, const month_weekday_last& mwdl) noexcept;
constexpr year_month_weekday_last operator/(int y, const month_weekday_last& mwdl) noexcept;
constexpr year_month_weekday_last operator/(const month_weekday_last& mwdl, const year& y) noexcept;
constexpr year_month_weekday_last operator/(const month_weekday_last& mwdl, int y) noexcept;

// time_of_day
template <class Duration> class time_of_day;
template <> class time_of_day<hours>;
template <> class time_of_day<minutes>;
template <> class time_of_day<seconds>;
template <class Rep, class Period> class time_of_day<duration<Rep, Period>>;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const time_of_day<hours>& t);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const time_of_day<minutes>& t);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const time_of_day<seconds>& t);

template <class charT, class traits, class Rep, class Period>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const time_of_day<duration<Rep, Period>>& t);
```

```
// time zone database

struct tzdb;
class tzdb_list;
const tzdb& get_tzdb();
tzdb_list& get_tzdb_list();
const time_zone* locate_zone(string_view tz_name);
const time_zone* current_zone();

// Remote time zone database -- Needs discussion

const tzdb&  reload_tzdb();
string       remote_version();

// exception classes
class nonexistent_local_time;
class ambiguous_local_time;

// information classes
struct sys_info;
template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const sys_info& si);

struct local_info;
template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const local_info& li);

// time_zone
enum class choose {earliest, latest};
class time_zone;

bool operator==(const time_zone& x, const time_zone& y) noexcept;
bool operator!=(const time_zone& x, const time_zone& y) noexcept;

bool operator<(const time_zone& x, const time_zone& y) noexcept;
bool operator>(const time_zone& x, const time_zone& y) noexcept;
bool operator<=(const time_zone& x, const time_zone& y) noexcept;
bool operator>=(const time_zone& x, const time_zone& y) noexcept;

// zoned_time
template <class Duration, class TimeZonePtr = const time_zone*> class zoned_time;

using zoned_seconds = zoned_time<seconds>;

template <class Duration1, class Duration2, class TimeZonePtr>
    bool
    operator==(const zoned_time<Duration1, TimeZonePtr>& x,
               const zoned_time<Duration2, TimeZonePtr>& y);

template <class Duration1, class Duration, class TimeZonePtr2>
    bool
    operator!=(const zoned_time<Duration1, TimeZonePtr>& x,
               const zoned_time<Duration2, TimeZonePtr>& y);

template <class charT, class traits, class Duration, class TimeZonePtr>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os,
               const zoned_time<Duration, TimeZonePtr>& t);

template <class charT, class traits, class Duration, class TimeZonePtr>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt,
              const zoned_time<Duration, TimeZonePtr>& tp);

// format

template <class charT, class Streamable>
    basic_string<charT>
    format(const charT* fmt, const Streamable& s);

template <class charT, class Streamable>
    basic_string<charT>
    format(const locale& loc, const charT* fmt, const Streamable& s);

template <class charT, class traits, class Alloc, class Streamable>
    basic_string<charT, traits, Alloc>
    format(const basic_string<charT, traits, Alloc>& fmt, const Streamable& s);

template <class charT, class traits, class Alloc, class Streamable>
    basic_string<charT, traits, Alloc>
    format(const locale& loc, const basic_string<charT, traits, Alloc>& fmt, const Streamable& s);

// parse
```

```
template <class charT, class traits, class Alloc, class Parsable>
    unspecified
    parse(const basic_string<charT, traits, Alloc>& format, Parsable& tp);

template <class charT, class traits, class Alloc, class Parsable>
    unspecified
    parse(const basic_string<charT, traits, Alloc>& format, Parsable& tp,
          basic_string<charT, traits, Alloc>& abbrev);

template <class charT, class traits, class Alloc, class Parsable>
    unspecified
    parse(const basic_string<charT, traits, Alloc>& format, Parsable& tp,
          minutes& offset);

template <class charT, class traits, class Alloc, class Parsable>
    unspecified
    parse(const basic_string<charT, traits, Alloc>& format, Parsable& tp,
          basic_string<charT, traits, Alloc>& abbrev, minutes& offset);

// leap second support

class leap;

bool operator==(const leap& x, const leap& y);
bool operator!=(const leap& x, const leap& y);
bool operator< (const leap& x, const leap& y);
bool operator> (const leap& x, const leap& y);
bool operator<=(const leap& x, const leap& y);
bool operator>=(const leap& x, const leap& y);

template <class Duration> bool operator==(const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator==(const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator!=(const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator!=(const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator< (const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator< (const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator> (const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator> (const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator<=(const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator<=(const sys_time<Duration>& x, const leap&               y);
template <class Duration> bool operator>=(const leap&               x, const sys_time<Duration>& y);
template <class Duration> bool operator>=(const sys_time<Duration>& x, const leap&               y);

class link;

bool operator==(const link& x, const link& y);
bool operator!=(const link& x, const link& y);
bool operator< (const link& x, const link& y);
bool operator> (const link& x, const link& y);
bool operator<=(const link& x, const link& y);
bool operator>=(const link& x, const link& y);

inline constexpr chrono::last_spec last{};

inline constexpr chrono::weekday Sunday{0};
inline constexpr chrono::weekday Monday{1};
inline constexpr chrono::weekday Tuesday{2};
inline constexpr chrono::weekday Wednesday{3};
inline constexpr chrono::weekday Thursday{4};
inline constexpr chrono::weekday Friday{5};
inline constexpr chrono::weekday Saturday{6};

inline constexpr chrono::month January{1};
inline constexpr chrono::month February{2};
inline constexpr chrono::month March{3};
inline constexpr chrono::month April{4};
inline constexpr chrono::month May{5};
inline constexpr chrono::month June{6};
inline constexpr chrono::month July{7};
inline constexpr chrono::month August{8};
inline constexpr chrono::month September{9};
inline constexpr chrono::month October{10};
inline constexpr chrono::month November{11};
inline constexpr chrono::month December{12};

} // namespace chrono

inline namespace literals {
inline namespace chrono_literals {
// ...
constexpr chrono::day  operator "" d(unsigned long long d) noexcept;
constexpr chrono::year operator "" y(unsigned long long y) noexcept;
}
}
```

Back to TOC

Add new section [time.is_clock] after 23.17.4.3 Specializations of `common_type` [time.traits.specializations]:

23.17.4.4 `is_clock` [time.traits.is_clock]

```
template <class T> struct is_clock;
```

`is_clock` is a `UnaryTypeTrait` ([meta.rqmts]) with a base characteristic of `true_type` if `T` meets the `Clock` requirements ([time.clock.req]), otherwise `false_type`. For the purposes of the specification of this trait, the extent to which an implementation determines that a type cannot meet the clock requirements is unspecified, except that as a minimum a type `T` shall not qualify as a clock unless it satisfies all of the following conditions:

- the *qualified-id*s `T::rep`, `T::period`, `T::duration`, and `T::time_point` are valid and each denotes a type ([temp.deduct]),
- the expression `T::is_steady` is well-formed when treated as an unevaluated operand,
- the expression `T::now()` is well-formed when treated as an unevaluated operand.

The behavior of a program that adds specializations for `is_clock` is undefined.

Back to TOC

Add new section [time.duration.io] after 23.17.5.9 duration algorithms [time.duration.alg]:

23.17.5.10 duration stream insertion [time.duration.io]

```
template <class charT, class traits, class Rep, class Period>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const duration<Rep, Period>& d);
```

Requires: `Rep` is an integral type with rank `short` or greater, or a floating point type. `charT` is `char` or `wchar_t`.

Effects: Forms a `basic_string<charT, traits>` from `d.count()` using `to_string` if `charT` is `char`, or `to_wstring` if `charT` is `wchar_t`. This `basic_string<charT, traits>` is appended with `get_units<charT, traits>(typename Period::type{})` (described below) and inserts that `basic_string` into `os`. [*Note:* this specification assures that the result of this streaming operation will obey the width and alignment properties of the stream. — *end note*]

`get_units<charT, traits>(typename Period::type{})` is an exposition-only function which returns a NTCTS which depends on `Period::type` as follows (let `period` be the type `Period::type`):

- If `period` is type `atto`, `as`, else
- if `period` is type `femto`, `fs`, else
- if `period` is type `pico`, `ps`, else
- if `period` is type `nano`, `ns`, else
- if `period` is type `micro`, `μs` (U+00B5), else
- if `period` is type `milli`, `ms`, else
- if `period` is type `centi`, `cs`, else
- if `period` is type `deci`, `ds`, else
- if `period` is type `ratio<1>`, `s`, else
- if `period` is type `deca`, `das`, else
- if `period` is type `hecto`, `hs`, else
- if `period` is type `kilo`, `ks`, else
- if `period` is type `mega`, `Ms`, else
- if `period` is type `giga`, `Gs`, else
- if `period` is type `tera`, `Ts`, else
- if `period` is type `peta`, `Ps`, else
- if `period` is type `exa`, `Es`, else
- if `period` is type `ratio<60>`, `min`, else
- if `period` is type `ratio<3600>`, `h`, else
- if `period::den == 1`, `[num]s`, else
- `[num/den]s`.

In the list above the use of `num` and `den` refer to the static data members of `period` which are converted to arrays of `charT` using a decimal conversion with no leading zeroes.

For streams with `charT` which has a representation of 8 bits `μs` should be encoded as UTF-8. Otherwise UTF-16 or UTF-32 is encouraged. The implementation may substitute other encodings, including `us`.

Returns: `os`.

```
template <class charT, class traits, class Rep, class Period>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt,
          const duration<Rep, Period>& d);
```

Effects: Streams `d` into `os` using the format specified by the NTCTS `fmt`. `fmt` encoding follows the rules specified by [time.format].

Returns: `os`.

```
template <class charT, class traits, class Rep, class Period, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            duration<Rep, Period>& d,
            basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream `is` into the duration `d` using the format flags given in the NTCTS `fmt` as specified in [time.parse].

If the parse parses everything specified by the parsing format flags without error, and yet none of the flags impacts a duration, `d` will be assigned a zero value.

If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

Returns: `is`.

Back to TOC

Add to synopsis in section [time.point] 23.17.6 Class template `time_point`:

```
template <class Clock, class Duration = typename Clock::duration>
class time_point {
public:
...
// 23.17.6.3, arithmetic
constexpr time_point& operator++();
constexpr time_point operator++(int);
constexpr time_point& operator--();
constexpr time_point operator--(int);

constexpr time_point& operator+=(const duration& d);
constexpr time_point& operator-=(const duration& d);
...
};
```

Back to TOC

Modify section 23.17.6 Class template `time_point` [time.point]/p1:

1 `Clock` shall meet the Clock requirements ([time.clock.req]) or `Clock` shall be `local_t`.

Back to TOC

Add to section [time.point.arithmetic] 23.17.6.3 `time_point` arithmetic:

```
constexpr time_point& operator++();
```

Effects: `++d_`.

Returns: `*this`.

```
constexpr time_point operator++(int);
```

Returns: `time_point{d_++}`.

```
constexpr time_point& operator--();
```

Effects: `--d_`.

Returns: `*this`.

```
constexpr time_point operator--(int);
```

Returns: `time_point{d_--}`.

Back to TOC

Modify 23.17.7 [time.clock]:

1 The types defined in this subclause shall satisfy the TrivialClock requirements (23.17.3) unless otherwise specified.

Back to TOC

Modify 23.17.7.1 [time.clock.system]:

1 Objects of class `system_clock` represent wall clock time from the system-wide realtime clock. `sys_time<Duration>` measures time since (and before) 1970-01-01 00:00:00 UTC excluding leap seconds. This measure is commonly referred to as *Unix Time*. This measure facilitates an efficient mapping between `sys_time` and calendar types ([time.calendar]).

[*Example:*

```
sys_seconds{sys_days{1970y/January/1}}.time_since_epoch() is 0s
sys_seconds{sys_days{2000y/January/1}}.time_since_epoch() is 946'684'800s which is 10'957 * 86'400s
```

—end example]

Back to TOC

Append new paragraphs after 23.17.7.1 [time.clock.system]/p4:

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const sys_time<Duration>& tp);
```

Remarks: This operator shall not participate in overload resolution if `treat_as_floating_point_v<typename Duration::rep>` is true, or if `Duration{1} >= days{1}`.

Effects:

```
auto const dp = floor<days>(tp);
os << year_month_day{dp} << ' ' << time_of_day{tp-dp};
```

Returns: `os`.

[*Example:*

```
cout << sys_seconds{0s} << '\n';             // 1970-01-01 00:00:00
cout << sys_seconds{946'684'800s} << '\n';   // 2000-01-01 00:00:00
cout << sys_seconds{946'688'523s} << '\n';   // 2000-01-01 01:02:03
```

— *end example*]

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const sys_days& dp);
```

Effects: `os << year_month_day{dp}`.

Returns: `os`.

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const sys_time<Duration>& tp);
```

Effects: Streams `tp` into `os` using the format specified by the NTCTS `fmt`. `fmt` encoding follows the rules specified by [time.format]. If `%Z` is used, it will be replaced with `"UTC"` widened to `charT`. If `%z` is used (or a modified variant of `%z`), an offset of `0min` will be formatted.

Returns: `os`.

```
template <class charT, class traits, class Duration, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            sys_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream `is` into the `sys_time tp` using the format flags given in the NTCTS `fmt` as specified in [time.parse].

If the parse fails to decode a valid date, `is.setstate(ios_base::failbit)` shall be called and `tp` shall not be modified.

If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null. Additionally, the parsed offset will be subtracted from the successfully parsed timestamp prior to assigning that difference to `tp`.

Returns: `is`.

Back to TOC

Add new section [time.clock.utc] after 23.17.7.1 Class system_clock [time.clock.system]:

23.17.7.2 Class utc_clock [time.clock.utc]

```
class utc_clock
{
public:
    using rep                       = a signed arithmetic type;
    using period                    = ratio<unspecified, unspecified>;
    using duration                  = chrono::duration<rep, period>;
    using time_point                = chrono::time_point<utc_clock>;
    static constexpr bool is_steady = unspecified;

    static time_point now();

    template <class Duration>
    static
    sys_time<common_type_t<Duration, seconds>>
    to_sys(const utc_time<Duration>& t);

    template <class Duration>
    static
    utc_time<common_type_t<Duration, seconds>>
    from_sys(const sys_time<Duration>& t);
};
```

In contrast to `sys_time` which does not take leap seconds into account, `utc_clock` and its associated `time_point`, `utc_time`, count time, including leap seconds, since 1970-01-01 00:00:00 UTC.

[*Example:*

```
clock_cast<utc_clock>(sys_seconds{sys_days{1970y/January/1}}).time_since_epoch() is 0s
clock_cast<utc_clock>(sys_seconds{sys_days{2000y/January/1}}).time_since_epoch() is 946'684'822s which is 10'957 * 86'400s + 22s
```

—*end example*]

`utc_clock` is not a `TrivialClock` unless the implementation can guarantee that `utc_clock::now()` does not propagate an exception. [*Note:* `noexcept(from_sys(system_clock::now()))` is `false`. — *end note*]

```
static utc_clock::time_point utc_clock::now();
```

Returns: `from_sys(system_clock::now())`, or a more accurate value of `utc_time`.

```
template <typename Duration>
static
sys_time<common_type_t<Duration, seconds>>
utc_clock::to_sys(const utc_time<Duration>& u);
```

Returns: A `sys_time` `t`, such that `from_sys(t) == u` if such a mapping exists. Otherwise `u` represents a `time_point` during a leap second insertion and the last representable value of `sys_time` prior to the insertion of the leap second is returned.

```
template <typename Duration>
static
utc_time<common_type_t<Duration, seconds>>
utc_clock::from_sys(const sys_time<Duration>& t);
```

Returns: A `utc_time` `u`, such that `u.time_since_epoch() - t.time_since_epoch()` is equal to the number of leap seconds that were inserted between `t` and 1970-01-01. If `t` is exactly the date of leap second insertion, then the conversion counts that leap second as inserted.

[*Example:*

```
auto t = sys_days{July/1/2015} - 2ns;
auto u = utc_clock::from_sys(t);
assert(u.time_since_epoch() - t.time_since_epoch() == 25s);
t += 1ns;
u = utc_clock::from_sys(t);
assert(u.time_since_epoch() - t.time_since_epoch() == 25s);
t += 1ns;
u = utc_clock::from_sys(t);
assert(u.time_since_epoch() - t.time_since_epoch() == 26s);
t += 1ns;
u = utc_clock::from_sys(t);
assert(u.time_since_epoch() - t.time_since_epoch() == 26s);
```

— *end example*]

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const utc_time<Duration>& t);
```

Effects: Calls `to_stream(os, fmt, t)`, where `fmt` is a string containing `"%F %T"` widened to `charT`.

Returns: `os`.

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const utc_time<Duration>& tp);
```

Effects: Streams `tp` into `os` using the format specified by the NTCTS `fmt`. `fmt` encoding follows the rules specified by [time.format]. If `%Z` is used, it will be replaced with `"UTC"` widened to `charT`. If `%z` is used (or a modified variant of `%z`), an offset of `0min` will be formatted. If `tp` represents a time during a leap second insertion, and if a seconds field is formatted, the integral portion of that format shall be `"60"` widened to `charT`.

Returns: `os`.

[*Example:*

```
auto t = sys_days{July/1/2015} - 500ms;
auto u = clock_cast<utc_clock>(t);
for (auto i = 0; i < 8; ++i, u += 250ms)
    cout << u << " UTC\n";
```

Output:

```
2015-06-30 23:59:59.500 UTC
2015-06-30 23:59:59.750 UTC
2015-06-30 23:59:60.000 UTC
2015-06-30 23:59:60.250 UTC
2015-06-30 23:59:60.500 UTC
2015-06-30 23:59:60.750 UTC
2015-07-01 00:00:00.000 UTC
2015-07-01 00:00:00.250 UTC
```

— *end example*]

```
template <class charT, class traits, class Duration, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            utc_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream `is` into the `utc_time` `tp` using the format flags given in the NTCTS `fmt` as specified in [time.parse].

If the parse fails to decode a valid date, `is.setstate(ios_base::failbit)` shall be called and `tp` shall not be modified.

If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null. Additionally, the parsed offset will be subtracted from the successfully parsed timestamp prior to assigning that difference to `tp`.

Returns: `is`.

Back to TOC

Add new section [time.clock.tai] after 23.17.7.2 Class utc_clock [time.clock.utc]:

23.17.7.3 Class tai_clock [time.clock.tai]

```
class tai_clock
{
public:
    using rep                       = a signed arithmetic type;
    using period                    = ratio<unspecified, unspecified>;
    using duration                  = chrono::duration<rep, period>;
    using time_point                = chrono::time_point<tai_clock>;
    static constexpr bool is_steady = unspecified;

    static time_point now();

    template <class Duration>
    static
    utc_time<common_type_t<Duration, seconds>>
    to_utc(const tai_time<Duration>&) noexcept;

    template <class Duration>
    static
    tai_time<common_type_t<Duration, seconds>>
    from_utc(const utc_time<Duration>&) noexcept;
};
```

The clock `tai_clock` measures seconds since 1958-01-01 00:00:00 and is offset 10s ahead of UTC at this date. That is, 1958-01-01 00:00:00 TAI is equivalent to 1957-12-31 23:59:50 UTC. Leap seconds are not inserted into TAI. Therefore every time a leap second is inserted into UTC, UTC falls another second behind TAI. For example by 2000-01-01 there had been 22 leap seconds inserted so

2000-01-01 00:00:00 UTC is equivalent to 2000-01-01 00:00:32 TAI (22s plus the initial 10s offset).

`tai_clock` is not a `TrivialClock` unless the implementation can guarantee that `tai_clock::now()` does not propagate an exception. [*Note:* `noexcept(from_utc(utc_clock::now()))` is `false`. — *end note*]

```
static tai_clock::time_point tai_clock::now();
```

Returns: `from_utc(utc_clock::now())`, or a more accurate value of `tai_time`.

```
template <class Duration>
static
utc_time<common_type_t<Duration, seconds>>
to_utc(const tai_time<Duration>& t) noexcept;
```

Returns: `utc_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} - 378691210s`

[*Note:* `378691210s == sys_days{1970y/January/1} - sys_days{1958y/January/1} + 10s` — *end note*]

```
template <class Duration>
static
tai_time<common_type_t<Duration, seconds>>
tai_clock::from_utc(const utc_time<Duration>& t) noexcept;
```

Returns: `tai_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} + 378691210s`

[*Note:* `378691210s == sys_days{1970y/January/1} - sys_days{1958y/January/1} + 10s` — *end note*]

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const tai_time<Duration>& t);
```

Effects: Calls `to_stream(os, fmt, t)`, where `fmt` is a string containing `"%F %T"` widened to `charT`.

Returns: `os`.

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const tai_time<Duration>& tp);
```

Effects: Streams `tp` into `os` using the format specified by the NTCTS `fmt`. `fmt` encoding follows the rules specified by [time.format]. If `%Z` is used, it will be replaced with `"TAI"`. If `%z` is used (or a modified variant of `%z`), an offset of `0min` will be formatted. The date and time formatted shall be equivalent to that formatted by a `sys_time` initialized with:

```
sys_time<Duration>{tp.time_since_epoch()} - (sys_days{1970y/January/1} - sys_days{1958y/January/1})
```

Returns: `os`.

[*Example:*

```
auto st = sys_days{2000y/January/1};
auto tt = clock_cast<tai_clock>(st);
cout << format("%F %T %Z == ", st) << format("%F %T %Z\n", tt);
```

Output:

```
2000-01-01 00:00:00 UTC == 2000-01-01 00:00:32 TAI
```

— *end example*]

```
template <class charT, class traits, class Duration, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            tai_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream `is` into the `tai_time tp` using the format flags given in the NTCTS `fmt` as specified in [time.parse].

If the parse fails to decode a valid date, `is.setstate(ios_base::failbit)` shall be called and `tp` shall not be modified.

If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null. Additionally, the parsed offset will be subtracted from the successfully parsed timestamp prior to assigning that difference to `tp`.

Returns: `is`.

Back to TOC

Add new section [time.clock.gps] after 23.17.7.3 Class tai_clock [time.clock.tai]:

23.17.7.4 Class gps_clock [time.clock.gps]

```
class gps_clock
{
public:
    using rep                       = a signed arithmetic type;
    using period                    = ratio<unspecified, unspecified>;
    using duration                  = chrono::duration<rep, period>;
    using time_point                = chrono::time_point<gps_clock>;
    static constexpr bool is_steady = unspecified;

    static time_point now();

    template <class Duration>
    static
    utc_time<common_type_t<Duration, seconds>>
    to_utc(const gps_time<Duration>&) noexcept;

    template <class Duration>
    static
    gps_time<common_type_t<Duration, seconds>>
    from_utc(const utc_time<Duration>&) noexcept;
};
```

The clock `gps_clock` measures seconds since the first Sunday of January, 1980 00:00:00 UTC. Leap seconds are not inserted into GPS. Therefore every time a leap second is inserted into UTC, UTC falls another second behind GPS. Aside from the offset from 1958y/January/1 to 1980y/January/Sunday[1] GPS is behind TAI by 19s due to the 10s offset between 1958 and 1970 and the additional 9 leap seconds inserted between 1970 and 1980.

`gps_clock` is not a `TrivialClock` unless the implementation can guarantee that `gps_clock::now()` does not propagate an exception. [*Note:* `noexcept(from_utc(utc_clock::now()))` is `false`. — *end note*]

```
static gps_clock::time_point gps_clock::now();
```

Returns: `from_utc(utc_clock::now())`, or a more accurate value of `gps_time`.

```
template <class Duration>
static
utc_time<common_type_t<Duration, seconds>>
gps_clock::to_utc(const gps_time<Duration>& t) noexcept;
```

Returns: `gps_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} + 315964809s`

[*Note:* `315964809s == sys_days{1980y/January/Sunday[1]} - sys_days{1970y/January/1} + 9s` — *end note*]

```
template <class Duration>
static
gps_time<common_type_t<Duration, seconds>>
gps_clock::from_utc(const utc_time<Duration>& t) noexcept;
```

Returns: `gps_time<common_type_t<Duration, seconds>>{t.time_since_epoch()} - 315964809s`

[*Note:* `315964809s == sys_days{1980y/January/Sunday[1]} - sys_days{1970y/January/1} + 9s` — *end note*]

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const gps_time<Duration>& t);
```

Effects: Calls `to_stream(os, fmt, t)`, where `fmt` is a string containing `"%F %T"` widened to `charT`.

Returns: `os`.

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const gps_time<Duration>& tp);
```

Effects: Streams `tp` into `os` using the format specified by the NTCTS `fmt`. `fmt` encoding follows the rules specified by [time.format]. If `%Z` is used, it will be replaced with `"GPS"`. If `%z` is used (or a modified variant of `%z`), an offset of `0min` will be formatted. The date and time formatted shall be equivalent to that formatted by a `sys_time` initialized with:

```
sys_time<Duration>{tp.time_since_epoch()} + (sys_days{1980y/January/Sunday[1]} - sys_days{1970y/January/1})
```

Returns: `os`.

[*Example:*

```
auto st = sys_days{2000y/January/1};
auto gt = clock_cast<gps_clock>(st);
cout << format("%F %T %Z == ", st) << format("%F %T %Z\n", gt);
```

Output:

```
2000-01-01 00:00:00 UTC == 2000-01-01 00:00:13 GPS
```

— *end example*]

```
template <class charT, class traits, class Duration, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            gps_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream `is` into the `gps_time tp` using the format flags given in the NTCTS `fmt` as specified in [time.parse].

If the parse fails to decode a valid date, `is.setstate(ios_base::failbit)` shall be called and `tp` shall not be modified.

If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null. Additionally, the parsed offset will be subtracted from the successfully parsed timestamp prior to assigning that difference to `tp`.

Returns: `is`.

Back to TOC

Add new section [time.clock.file] after 23.17.7.4 Class gps_clock [time.clock.gps]:

23.17.7.5 Class file_clock [time.clock.file]

```
class file_clock
{
public:
    using rep                       = a signed arithmetic type;
    using period                    = ratio<unspecified, unspecified>;
    using duration                  = chrono::duration<rep, period>;
    using time_point                = chrono::time_point<file_clock>;
    static constexpr bool is_steady = unspecified;

    static time_point now() noexcept;

    // Conversion functions, see below
};
```

The clock `file_clock` is used to create the `time_point` system used for `file_time_type` ([filesystems]). Its epoch is unspecified.

```
static file_clock::time_point file_clock::now();
```

Returns: A `file_clock::time_point` indicating the current time.

The class `file_clock` shall provide precisely one of the following two sets of static member functions:

```
template <class Duration>
static
sys_time<see below>
to_sys(const file_time<Duration>&);

template <class Duration>
static
file_time<see below>
from_sys(const sys_time<Duration>&);
```

or:

```
template <class Duration>
static
utc_time<see below>
to_utc(const file_time<Duration>&);

template <class Duration>
static
file_time<see below>
from_utc(const utc_time<Duration>&);
```

These member functions shall provide `time_point` conversions consistent with those specified by `utc_clock`, `tai_clock`, and `gps_clock`. The `duration` of the resultant `time_point` is computed from the `Duration` of the input `time_point`.

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const file_time<Duration>& t);
```

Effects: Calls `to_stream(os, fmt, t)`, where `fmt` is a string containing `"%F %T"` widened to `charT`.

Returns: os.

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const file_time<Duration>& tp);
```

Effects: Streams tp into os using the format specified by the NTCTS fmt. fmt encoding follows the rules specified by [time.format]. If %Z is used, it will be replaced with "UTC" widened to charT. If %z is used (or a modified variant of %z), an offset of 0min will be formatted. The date and time formatted shall be equivalent to that formatted by a sys_time initialized with clock_cast<system_clock>(tp), or by a utc_time initialized with clock_cast<utc_clock>(tp).

Returns: os.

```
template <class charT, class traits, class Duration, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            file_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream is into the file_time tp using the format flags given in the NTCTS fmt as specified in [time.parse].

If the parse fails to decode a valid date, is.setstate(ios_base::failbit) shall be called and tp shall not be modified.

If %Z is used and successfully parsed, that value will be assigned to *abbrev if abbrev is non-null.

If %z (or a modified variant) is used and successfully parsed, that value will be assigned to *offset if offset is non-null. Additionally, the parsed offset will be subtracted from the successfully parsed timestamp prior to assigning that difference to tp.

Returns: is.

Back to TOC

Add new section [time.clock.local_time] after 23.17.7.3 Class high_resolution_clock [time.clock.hres]:

23.17.7.8 local_time [time.clock.local_time]

The family of time points denoted by local_time<Duration> are based on the pseudo clock local_t. local_t has no member now() and thus does not meet the clock requirements. Nevertheless local_time<Duration> serves the vital role of representing local time with respect to a not-yet-specified time zone. Aside from being able to get the current time, the complete time_point algebra is available for local_time<Duration> (just as for sys_time<Duration>).

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const local_time<Duration>& lt);
```

Effects:

```
os << sys_time<Duration>{lt.time_since_epoch()};
```

Returns: os.

```
template <class charT, class traits, class Duration>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const local_time<Duration>& tp,
          const string* abbrev = nullptr, const seconds* offset_sec = nullptr);
```

Effects: Streams tp into os using the format specified by the NTCTS fmt. fmt encoding follows the rules specified by [time.format]. If %Z is used, it will be replaced with *abbrev if abbrev is not equal to nullptr. If abbrev is equal to nullptr (and %Z is used), os.setstate(ios_base::failbit) shall be called. If %z is used (or a modified variant of %z), it will be formatted with the value of *offset_sec if offset_sec is not equal to nullptr. If %z (or a modified variant of %z) is used, and offset_sec is equal to nullptr, then os.setstate(ios_base::failbit) shall be called.

Returns: os.

```
template <class charT, class traits, class Duration, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            local_time<Duration>& tp, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream is into the local_time tp using the format flags given in the NTCTS fmt as specified in [time.parse].

If the parse fails to decode a valid date, is.setstate(ios_base::failbit) shall be called and tp shall not be modified.

If %Z is used and successfully parsed, that value will be assigned to *abbrev if abbrev is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

Returns: `is`.

Back to TOC

Add new section [time.clock.clock_cast] after 23.17.7.9 local_time [time.clock.local_time]:

23.17.7.9 clock_cast [time.clock.clock_cast]

```
template <class DestClock, class SourceClock>
struct clock_time_conversion
{};
```

`clock_time_conversion` serves as a trait which can be used to specify how to convert `time_point<SourceClock, Duration>` to `time_point<DestClock, Duration>` via a specialization: `clock_time_conversion<DestClock, SourceClock>`. A specialization of `clock_time_conversion<DestClock, SourceClock>` shall provide a const-qualified `operator()` that takes a parameter of type `time_point<SourceClock, Duration>` and returns a `time_point<DestClock, *OtherDuration*>` representing an equivalent point in time. *`OtherDuration`* is a `chrono::duration` whose specialization is computed from the input `Duration` in a manner which can vary for each `clock_time_conversion` specialization. A program may specialize `clock_time_conversion` if at least one of the template parameters is a user-defined clock type.

Several specializations are provided by the implementation:

```
// Identity

template <typename Clock>
struct clock_time_conversion<Clock, Clock>
{
    template <class Duration>
    time_point<Clock, Duration>
    operator()(const time_point<Clock, Duration>& t) const;
};

template <class Duration>
time_point<Clock, Duration>
operator()(const time_point<Clock, Duration>& t) const;
```

Returns: `t`.

```
template <>
struct clock_time_conversion<system_clock, system_clock>
{
    template <class Duration>
    sys_time<Duration>
    operator()(const sys_time<Duration>& t) const;
};

template <class Duration>
sys_time<Duration>
operator()(const sys_time<Duration>& t) const;
```

Returns: `t`.

```
template <>
struct clock_time_conversion<utc_clock, utc_clock>
{
    template <class Duration>
    utc_time<Duration>
    operator()(const utc_time<Duration>& t) const;
};

template <class Duration>
utc_time<Duration>
operator()(const utc_time<Duration>& t) const;
```

Returns: `t`.

```
// system_clock <-> utc_clock

template <>
struct clock_time_conversion<utc_clock, system_clock>
{
    template <class Duration>
    utc_time<common_type_t<Duration, seconds>>
    operator()(const sys_time<Duration>& t) const;
};

template <class Duration>
utc_time<common_type_t<Duration, seconds>>
operator()(const sys_time<Duration>& t) const;
```

Returns: `utc_clock::from_sys(t)`.

```
template <>
struct clock_time_conversion<system_clock, utc_clock>
{
    template <class Duration>
    sys_time<common_type_t<Duration, seconds>>
    operator()(const utc_time<Duration>& t) const;
};

template <class Duration>
sys_time<common_type_t<Duration, seconds>>
operator()(const utc_time<Duration>& t) const;
```

Returns: `utc_clock::to_sys(t)`.

```
// Clock <-> system_clock

template <class SourceClock>
struct clock_time_conversion<system_clock, SourceClock>
{
    template <class Duration>
    auto
    operator()(const time_point<SourceClock, Duration>& t) const
        -> decltype(SourceClock::to_sys(t));
};

template <class Duration>
auto
operator()(const time_point<SourceClock, Duration>& t) const
    -> decltype(SourceClock::to_sys(t));
```

Remarks: This function does not participate in overload resolution unless `SourceClock::to_sys(t)` is well-formed. If `SourceClock::to_sys(t)` does not return `sys_time<Duration>`, where `Duration` is a valid `chrono::duration` specialization, the program is ill-formed.

Returns: `SourceClock::to_sys(t)`.

```
template <class DestClock>
struct clock_time_conversion<DestClock, system_clock>
{
    template <class Duration>
    auto
    operator()(const sys_time<Duration>& t) const
        -> decltype(DestClock::from_sys(t));
};

template <class Duration>
auto
operator()(const sys_time<Duration>& t) const
    -> decltype(DestClock::from_sys(t));
```

Remarks: This function does not participate in overload resolution unless `DestClock::from_sys(t)` is well-formed. If `DestClock::from_sys(t)` does not return `time_point<DestClock, Duration>`, where `Duration` is a valid `chrono::duration` specialization, the program is ill-formed.

Returns: `DestClock::from_sys(t)`.

```
// Clock <-> utc_clock

template <class SourceClock>
struct clock_time_conversion<utc_clock, SourceClock>
{
    template <class Duration>
    auto
    operator()(const time_point<SourceClock, Duration>& t) const
        -> decltype(SourceClock::to_utc(t));
};

template <class Duration>
auto
operator()(const time_point<SourceClock, Duration>& t) const
    -> decltype(SourceClock::to_utc(t));
```

Remarks: This function does not participate in overload resolution unless `SourceClock::to_utc(t)` is well-formed. If `SourceClock::to_utc(t)` does not return `utc_time<Duration>`, where `Duration` is a valid `chrono::duration` specialization, the program is ill-formed.

Returns: `SourceClock::to_utc(t)`.

```
template <class DestClock>
struct clock_time_conversion<DestClock, utc_clock>
{
    template <class Duration>
```

```
    auto
    operator()(const utc_time<Duration>& t) const
        -> decltype(DestClock::from_utc(t));
};

template <class Duration>
auto
operator()(const utc_time<Duration>& t) const
    -> decltype(DestClock::from_utc(t));
```

Remarks: This function does not participate in overload resolution unless `DestClock::from_utc(t)` is well-formed. If `DestClock::from_utc(t)` does not return `time_point<DestClock, Duration>`, where `Duration` is a valid `chrono::duration` specialization, the program is ill-formed. .

Returns: `DestClock::from_utc(t)`.

```
// clock_cast

template <class DestClock, class SourceClock, class Duration>
auto
clock_cast(const time_point<SourceClock, Duration>& t);
```

Remarks: This function does not participate in overload resolution unless at least one of the following expressions are well-formed:

1. `clock_time_conversion<DestClock, SourceClock>{}(t)`
2. Exactly one of:
    - `clock_time_conversion<DestClock, system_clock>{}(`
      `clock_time_conversion<system_clock, SourceClock>{}(t))`
    - `clock_time_conversion<DestClock, utc_clock>{}(`
      `clock_time_conversion<utc_clock, SourceClock>{}(t))`
3. Exactly one of:
    - `clock_time_conversion<DestClock, utc_clock>{}(`
      `clock_time_conversion<utc_clock, system_clock>{}(`
      `clock_time_conversion<system_clock, SourceClock>{}(t)))`
    - `clock_time_conversion<DestClock, system_clock>{}(`
      `clock_time_conversion<system_clock, utc_clock>{}(`
      `clock_time_conversion<utc_clock, SourceClock>{}(t)))`

Returns: The first expression in the above list that is well-formed. If item 1 is not well-formed and both expressions in item 2 are well-formed, the `clock_cast` is ambiguous and the program is ill-formed. If items 1 and 2 are not well-formed and both expressions in item 3 are well-formed, the `clock_cast` is ambiguous and the program is ill-formed.

Back to TOC

Add a new section 23.17.8 Formatting [time.format]:

If `fmt` (P0645) moves forward within the LEWG, this section can easily be reworked to plug into that facility without loss of functionality. This will avoid two unrelated `format` facilities in the standard.

23.17.8 Formatting [time.format]

Each `format` overload specified in this section calls `to_stream` unqualified, so as to enable argument dependent lookup ([basic.lookup.argdep]).

```
template <class charT, class Streamable>
basic_string<charT>
format(const charT* fmt, const Streamable& s);
```

Remarks: This function shall not participate in overload resolution unless `to_stream(declval<basic_ostream<charT>&>(), fmt, s)` is a valid expression.

Effects: Constructs a local variable of type `basic_ostringstream<charT>` (for exposition purposes, named `os`). Executes `os.exceptions(ios::failbit | ios::badbit)`. Then calls `to_stream(os, fmt, s)`.

Returns: `os.str()`.

```
template <class charT, class Streamable>
basic_string<charT>
format(const locale& loc, const charT* fmt, const Streamable& s);
```

Remarks: This function shall not participate in overload resolution unless `to_stream(declval<basic_ostream<charT>&>(), fmt, s)` is a valid expression.

Effects: Constructs a local variable of type `basic_ostringstream<charT>` (for exposition purposes, named `os`). Executes `os.exceptions(ios::failbit | ios::badbit)`. Then calls `os.imbue(loc)`. Then calls `to_stream(os, fmt, s)`.

Returns: `os.str()`.

```
template <class charT, class traits, class Alloc, class Streamable>
basic_string<charT, traits, Alloc>
format(const basic_string<charT, traits, Alloc>& fmt, const Streamable& s);
```

Remarks: This function shall not participate in overload resolution unless `to_stream(declval<basic_ostringstream<charT, traits, Alloc>&>(), fmt.c_str(), s)` is a valid expression.

Effects: Constructs a local variable of type `basic_ostringstream<charT, traits, Alloc>` (for exposition purposes, named `os`). Executes `os.exceptions(ios::failbit | ios::badbit)`. Then calls `to_stream(os, fmt.c_str(), s)`.

Returns: `os.str()`.

```
template <class charT, class traits, class Alloc, class Streamable>
basic_string<charT, traits, Alloc>
format(const locale& loc, const basic_string<charT, traits, Alloc>& fmt, const Streamable& s);
```

Remarks: This function shall not participate in overload resolution unless `to_stream(declval<basic_ostringstream<charT, traits, Alloc>&>(), fmt.c_str(), s)` is a valid expression.

Effects: Constructs a local variable of type `basic_ostringstream<charT, traits, Alloc>` (for exposition purposes, named `os`). Then calls `os.imbue(loc)`. Executes `os.exceptions(ios::failbit | ios::badbit)`. Then calls `to_stream(os, fmt.c_str(), s)`.

Returns: `os.str()`.

The `format` functions call a `to_stream` function with a `basic_ostream`, a formatting string specifier, and a `Streamable` argument. Each `to_stream` overload is customized for each `Streamable` type. However all `to_stream` overloads treat the formatting string specifier according to the following specification:

The `fmt` string consists of zero or more conversion specifiers and ordinary multibyte characters. A conversion specifier consists of a `%` character, possibly followed by an `E` or `O` modifier character (described below), followed by a character that determines the behavior of the conversion specifier. All ordinary multibyte characters (excluding the terminating null character) are streamed unchanged into the `basic_ostream`.

Each conversion specifier is replaced by appropriate characters as described in the following list. Some of the conversion specifiers depend on the locale which is imbued to the `basic_ostream`. If the `Streamable` object does not contain the information the conversion specifier refers to, the value streamed to the `basic_ostream` is unspecified.

Unless explicitly specified, `Streamable` types will not contain time zone abbreviation and time zone offset information. If available, the conversion specifiers `%Z` and `%z` will format this information (respectively). If the information is not available, and `%Z` or `%z` are contained in `fmt`, `os.setstate(ios_base::failbit)` shall be called.

- `%a` The locale's abbreviated weekday name. If the value does not contain a valid `weekday`, `setstate(ios::failbit)` is called.
- `%A` The locale's full weekday name. If the value does not contain a valid `weekday`, `setstate(ios::failbit)` is called.
- `%b` The locale's abbreviated month name. If the value does not contain a valid `month`, `setstate(ios::failbit)` is called.
- `%B` The locale's full month name. If the value does not contain a valid `month`, `setstate(ios::failbit)` is called.
- `%c` The locale's date and time representation. The modified command `%Ec` produces the locale's alternate date and time representation.
- `%C` The year divided by 100 using floored division. If the result is a single decimal digit, it is prefixed with `0`. The modified command `%EC` produces the locale's alternative representation of the century.
- `%d` The day of month as a decimal number. If the result is a single decimal digit, it is prefixed with `0`. The modified command `%Od` produces the locale's alternative representation.
- `%D` Equivalent to `%m/%d/%y`.
- `%e` The day of month as a decimal number. If the result is a single decimal digit, it is prefixed with a space. The modified command `%Oe` produces the locale's alternative representation.
- `%F` Equivalent to `%Y-%m-%d`.
- `%g` The last two decimal digits of the ISO week-based year. If the result is a single digit it is prefixed by `0`.
- `%G` The ISO week-based year as a decimal number. If the result is less than four digits it is left-padded with `0` to four digits.
- `%h` Equivalent to `%b`.
- `%H` The hour (24-hour clock) as a decimal number. If the result is a single digit, it is prefixed with `0`. The modified command `%OH` produces the locale's alternative representation.

`%I` The hour (12-hour clock) as a decimal number. If the result is a single digit, it is prefixed with `0`. The modified command `%OI` produces the locale's alternative representation.

`%j` The day of the year as a decimal number. Jan 1 is `001`. If the result is less than three digits, it is left-padded with `0` to three digits.

`%m` The month as a decimal number. Jan is `01`. If the result is a single digit, it is prefixed with `0`. The modified command `%Om` produces the locale's alternative representation.

`%M` The minute as a decimal number. If the result is a single digit, it is prefixed with `0`. The modified command `%OM` produces the locale's alternative representation.

`%n` A new-line character.

`%p` The locale's equivalent of the AM/PM designations associated with a 12-hour clock.

`%r` The locale's 12-hour clock time.

`%R` Equivalent to `%H:%M`.

`%S` Seconds as a decimal number. If the number of seconds is less than 10, the result is prefixed with `0`. If the precision of the input can not be exactly represented with seconds, then the format is a decimal floating point number with a fixed format and a precision matching that of the precision of the input (or to a microseconds precision if the conversion to floating point decimal seconds can not be made within 18 fractional digits). The character for the decimal point is localized according to the locale. The modified command `%OS` produces the locale's alternative representation.

`%t` A horizontal-tab character.

`%T` Equivalent to `%H:%M:%S`.

`%u` The ISO weekday as a decimal number (1-7), where Monday is 1. The modified command `%Ou` produces the locale's alternative representation.

`%U` The week number of the year as a decimal number. The first Sunday of the year is the first day of week `01`. Days of the same year prior to that are in week `00`. If the result is a single digit, it is prefixed with `0`. The modified command `%OU` produces the locale's alternative representation.

`%V` The ISO week-based week number as a decimal number. If the result is a single digit, it is prefixed with `0`. The modified command `%OV` produces the locale's alternative representation.

`%w` The weekday as a decimal number (0-6), where Sunday is 0. The modified command `%Ow` produces the locale's alternative representation.

`%W` The week number of the year as a decimal number. The first Monday of the year is the first day of week `01`. Days of the same year prior to that are in week `00`. If the result is a single digit, it is prefixed with `0`. The modified command `%OW` produces the locale's alternative representation.

`%x` The locale's date representation. The modified command `%Ex` produces the locale's alternate date representation.

`%X` The locale's time representation. The modified command `%EX` produces the locale's alternate time representation.

`%y` The last two decimal digits of the year. If the result is a single digit it is prefixed by `0`.

`%Y` The year as a decimal number. If the result is less than four digits it is left-padded with `0` to four digits.

`%z` The offset from UTC in the ISO 8601 format. For example `-0430` refers to 4 hours 30 minutes behind UTC. If the offset is zero, `+0000` is used. The modified commands `%Ez` and `%Oz` insert a `:` between the hours and minutes: `-04:30`. If the offset information is not available, `setstate(ios_base::failbit)` shall be called.

`%Z` The time zone abbreviation. If the time zone abbreviation is not available, `setstate(ios_base::failbit)` shall be called.

`%%` A `%` character.

Back to TOC

Add a new section 23.17.9 Parsing [time.parse]:

23.17.9 Parsing [time.parse]

Each `parse` overload specified in this section calls `from_stream` unqualified, so as to enable argument dependent lookup ([basic.lookup.argdep]).

```
template <class charT, class traits, class Alloc, class Parsable>
unspecified
parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp);
```

Remarks: This function shall not participate in overload resolution unless

`from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp)` is a valid expression.

Returns: A manipulator that when extracted from a `basic_istream<charT, traits>` is calls `from_stream(is, fmt.c_str(), tp)`.

```
template <class charT, class traits, class Alloc, class Parsable>
unspecified
parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
      basic_string<charT, traits, Alloc>& abbrev);
```

Remarks: This function shall not participate in overload resolution unless `from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp, &abbrev)` is a valid expression.

Returns: A manipulator that when extracted from a `basic_istream<charT, traits>` is calls `from_stream(is, fmt.c_str(), tp, &abbrev)`.

```
template <class charT, class traits, class Alloc, class Parsable>
unspecified
parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
      minutes& offset);
```

Remarks: This function shall not participate in overload resolution unless `from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp, nullptr, &offset)` is a valid expression.

Returns: A manipulator that when extracted from a `basic_istream<charT, traits>` is calls `from_stream(is, fmt.c_str(), tp, nullptr, &offset)`.

```
template <class charT, class traits, class Alloc, class Parsable>
unspecified
parse(const basic_string<charT, traits, Alloc>& fmt, Parsable& tp,
      basic_string<charT, traits, Alloc>& abbrev, minutes& offset);
```

Remarks: This function shall not participate in overload resolution unless `from_stream(declval<basic_istream<charT, traits>&>(), fmt.c_str(), tp, &abbrev, &offset)` is a valid expression.

Returns: A manipulator that when extracted from a `basic_istream<charT, traits>` is calls `from_stream(is, fmt.c_str(), tp, &abbrev, &offset)`.

All `from_stream` overloads behave as an unformatted input function, except that they have an unspecified effect on the value returned by subsequent calls to `basic_istream<>::gcount()`. Each overload takes a format string containing ordinary characters and flags which have special meaning. Each flag begins with a `%`. Some flags can be modified by `E` or `O`. During parsing each flag interprets characters as parts of date and time type according to the table below. Some flags can be modified by a width parameter given as a positive decimal integer called out as *N* below which governs how many characters are parsed from the stream in interpreting the flag. All characters in the format string which are not represented in the table below, except for white space, are parsed unchanged from the stream. A white space character matches zero or more white space characters in the input stream.

If the `from_stream` overload fails to parse everything specified by the format string, or if insufficient information is parsed to specify a complete duration, time point, or calendrical data structure, `setstate(ios_base::failbit)` is called on the `basic_istream`.

- `%a` The locale's full or abbreviated case-insensitive weekday name.
- `%A` Equivalent to `%a`.
- `%b` The locale's full or abbreviated case-insensitive month name.
- `%B` Equivalent to `%b`.
- `%c` The locale's date and time representation. The modified command `%Ec` interprets the locale's alternate date and time representation.
- `%C` The century as a decimal number. The modified command `%NC` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required. The modified commands `%EC` and `%OC` interpret the locale's alternative representation of the century.
- `%d` The day of the month as a decimal number. The modified command `%Nd` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command `%Ed` interprets the locale's alternative representation of the day of the month.
- `%D` Equivalent to `%m/%d/%y`.
- `%e` Equivalent to `%d` and can be modified like `%d`.
- `%F` Equivalent to `%Y-%m-%d`. If modified with a width *N*, the width is applied to only `%Y`.
- `%g` The last two decimal digits of the ISO week-based year. The modified command `%Ng` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required.

`%G` The ISO week-based year as a decimal number. The modified command `%`*N*`G` specifies the maximum number of characters to read. If *N* is not specified, the default is 4. Leading zeroes are permitted but not required.

`%h` Equivalent to `%b`.

`%H` The hour (24-hour clock) as a decimal number. The modified command `%`*N*`H` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command `%OH` interprets the locale's alternative representation.

`%I` The hour (12-hour clock) as a decimal number. The modified command `%`*N*`I` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required.

`%j` The day of the year as a decimal number. Jan 1 is `1`. The modified command `%`*N*`j` specifies the maximum number of characters to read. If *N* is not specified, the default is 3. Leading zeroes are permitted but not required.

`%m` The month as a decimal number. Jan is `1`. The modified command `%`*N*`m` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command `%Om` interprets the locale's alternative representation.

`%M` The minutes as a decimal number. The modified command `%`*N*`M` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required. The modified command `%OM` interprets the locale's alternative representation.

`%n` Matches one white space character. [*Note:* `%n`, `%t` and a space, can be combined to match a wide range of white-space patterns. For example `"%n "` matches one or more white space characters, and `"%n%t%t"` matches one to three white space characters. — *end note*]

`%p` The locale's equivalent of the AM/PM designations associated with a 12-hour clock. The command `%I` must precede `%p` in the format string.

`%r` The locale's 12-hour clock time.

`%R` Equivalent to `%H:%M`.

`%S` The seconds as a decimal number. The modified command `%`*N*`S` specifies the maximum number of characters to read. If *N* is not specified, the default is 2 if the input time has a precision convertible to seconds. Otherwise the default width is determined by the decimal precision of the input and the field is interpreted as a long double in a fixed format. If encountered, the locale determines the decimal point character. Leading zeroes are permitted but not required. The modified command `%OS` interprets the locale's alternative representation.

`%t` Matches zero or one white space characters.

`%T` Equivalent to `%H:%M:%S`.

`%u` The ISO weekday as a decimal number (1-7), where Monday is 1. The modified command `%`*N*`u` specifies the maximum number of characters to read. If *N* is not specified, the default is 1. Leading zeroes are permitted but not required. The modified command `%Ou` interprets the locale's alternative representation.

`%U` The week number of the year as a decimal number. The first Sunday of the year is the first day of week `01`. Days of the same year prior to that are in week `00`. The modified command `%`*N*`U` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required.

`%V` The ISO week-based week number as a decimal number. The modified command `%`*N*`V` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required.

`%w` The weekday as a decimal number (0-6), where Sunday is 0. The modified command `%`*N*`w` specifies the maximum number of characters to read. If *N* is not specified, the default is 1. Leading zeroes are permitted but not required. The modified command `%Ow` interprets the locale's alternative representation.

`%W` The week number of the year as a decimal number. The first Monday of the year is the first day of week `01`. Days of the same year prior to that are in week `00`. The modified command `%`*N*`W` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required.

`%x` The locale's date representation. The modified command `%Ex` produces the locale's alternate date representation.

`%X` The locale's time representation. The modified command `%EX` produces the locale's alternate time representation.

`%y` The last two decimal digits of the year. If the century is not otherwise specified (e.g. with `%C`), values in the range [69 - 99] are presumed to refer to the years [1969 - 1999], and values in the range [00 - 68] are presumed to refer to the years [2000 - 2068]. The modified command `%`*N*`y` specifies the maximum number of characters to read. If *N* is not specified, the default is 2. Leading zeroes are permitted but not required. The modified commands `%Ey` and `%Oy` interpret the locale's alternative representation.

`%Y` The year as a decimal number. The modified command `%`*N*`Y` specifies the maximum number of characters to read. If *N* is not specified, the default is 4. Leading zeroes are permitted but not required. The modified command `%EY` interprets the locale's alternative representation.

The offset from UTC in the format `[+|-]hh[mm]`. For example `-0430` refers to 4 hours 30 minutes behind UTC. And

`%z` `04` refers to 4 hours ahead of UTC. The modified commands `%Ez` and `%Oz` parse a `:` between the hours and minutes and leading zeroes on the hour field are optional: `[+|-]h[h][:mm]`. For example `-04:30` refers to 4 hours 30 minutes behind UTC. And `4` refers to 4 hours ahead of UTC.

`%Z` The time zone abbreviation or name. A single word is parsed. This word can only contain characters from the *basic source character set* ([lex.charset] in the C++ standard) that are alphanumeric, or one of `'_'`, `'/'`, `'-'` or `'+'`.

`%%` A `%` character is extracted.

Back to TOC

Add new section [time.calendar] after 23.17.9 Clocks [time.parse]:

23.17.10 The civil calendar [time.calendar]

The types in this subclause describe the civil (Gregorian) calendar and its relationship to `sys_days` and `local_days`.

23.17.10.1 Class `last_spec` [time.calendar.last]

The struct `last_spec` is used in conjunction with other calendar types to specify the last in a sequence. For example, depending on context, it can represent the last day of a month, or the last day of the week of a month.

There is a `constexpr` object of this type named `last` in the `chrono` namespace.

```
struct last_spec
{
    explicit last_spec() = default;
};
```

23.17.10.2 Class `day` [time.calendar.day]

`day` represents a day of a month. It normally holds values in the range 1 to 31. However it may hold non-negative values outside this range. It can be constructed with any `unsigned` value, which will be subsequently truncated to fit into `day` object's unspecified internal storage. `day` is equality and less-than comparable, and participates in basic arithmetic with `days` representing the quantity between any two `day` objects. One can form a `day` literal with `d`. And one can stream out a `day`. `day` has explicit conversions to and from `unsigned`.

```
class day
{
    unsigned char d_;  // exposition only
public:
    day() = default;
    explicit constexpr day(unsigned d) noexcept;

    constexpr day& operator++()    noexcept;
    constexpr day  operator++(int) noexcept;
    constexpr day& operator--()    noexcept;
    constexpr day  operator--(int) noexcept;

    constexpr day& operator+=(const days& d) noexcept;
    constexpr day& operator-=(const days& d) noexcept;

    explicit constexpr operator unsigned() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const day& x, const day& y) noexcept;
constexpr bool operator!=(const day& x, const day& y) noexcept;
constexpr bool operator< (const day& x, const day& y) noexcept;
constexpr bool operator> (const day& x, const day& y) noexcept;
constexpr bool operator<=(const day& x, const day& y) noexcept;
constexpr bool operator>=(const day& x, const day& y) noexcept;

constexpr day  operator+(const day&  x, const days& y) noexcept;
constexpr day  operator+(const days& x, const day&  y) noexcept;
constexpr day  operator-(const day&  x, const days& y) noexcept;
constexpr days operator-(const day&  x, const day&  y) noexcept;

template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const day& d);

template <class charT, class traits>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const day& d);

template <class charT, class traits, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            day& d, basic_string<charT, traits, Alloc>* abbrev = nullptr,
```

```
            minutes* offset = nullptr);
```

`day` is a trivially copyable class type.
`day` is a standard-layout class type.

```
explicit constexpr day::day(unsigned d) noexcept;
```

Effects: Constructs an object of type `day` by constructing `d_` with `d`. The value held is unspecified if `d` is not in the range `[0, 255]`.

```
constexpr day& day::operator++() noexcept;
```

Effects: `++d_`.

Returns: `*this`.

```
constexpr day day::operator++(int) noexcept;
```

Effects: `++(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr day& day::operator--() noexcept;
```

Effects: `--d_`.

Returns: `*this`.

```
constexpr day day::operator--(int) noexcept;
```

Effects: `--(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr day& day::operator+=(const days& d) noexcept;
```

Effects: `*this = *this + d`.

Returns: `*this`.

```
constexpr day& day::operator-=(const days& d) noexcept;
```

Effects: `*this = *this - d`.

Returns: `*this`.

```
explicit constexpr day::operator unsigned() const noexcept;
```

Returns: `d_`.

```
constexpr bool day::ok() const noexcept;
```

Returns: `1 <= d_ && d_ <= 31`.

```
constexpr bool operator==(const day& x, const day& y) noexcept;
```

Returns: `unsigned{x} == unsigned{y}`.

```
constexpr bool operator!=(const day& x, const day& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const day& x, const day& y) noexcept;
```

Returns: `unsigned{x} < unsigned{y}`.

```
constexpr bool operator> (const day& x, const day& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const day& x, const day& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const day& x, const day& y) noexcept;
```

Returns: `!(x < y)`.

```
constexpr day  operator+(const day&  x, const days& y) noexcept;
```

Returns: `day(unsigned{x} + y.count())`.

```
constexpr day  operator+(const days& x, const day&  y) noexcept;
```

Returns: `y + x`.

```
constexpr day  operator-(const day&  x, const days& y) noexcept;
```

Returns: `x + -y`.

```
constexpr days operator-(const day&  x, const day&  y) noexcept;
```

Returns: `days{int(unsigned{x}) - int(unsigned{y})}`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const day& d);
```

Effects: Inserts `format(fmt, d)` where `fmt` is `"%d"` widened to `charT`. If `!d.ok()`, appends with `" is not a valid day"`.

Returns: `os`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const day& d);
```

Effects: Streams `d` into `os` using the format specified by the NTCTS `fmt`. `fmt` encoding follows the rules specified by [time.format].

Returns: `os`.

```
template <class charT, class traits, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            day& d, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream `is` into the `day d` using the format flags given in the NTCTS `fmt` as specified in [time.parse].

If the parse fails to decode a valid day, `is.setstate(ios_base::failbit)` shall be called and `d` shall not be modified.

If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

Returns: `is`.

```
constexpr day operator "" d(unsigned long long d) noexcept;
```

Returns: `day{static_cast<unsigned>(d)}`.

23.17.10.3 Class `month` [time.calendar.month]

`month` represents a month of a year. It normally holds values in the range 1 to 12. However it may hold non-negative values outside this range. It can be constructed with any `unsigned` value, which will be subsequently truncated to fit into `month` object's unspecified internal storage. `month` is equality and less-than comparable, and participates in basic arithmetic with `months` representing the quantity between any two `month` objects. One can stream out a `month`. `month` has explicit conversions to and from `unsigned`. There are 12 `month` constants, one for each month of the year.

```
class month
{
    unsigned char m_;  // exposition only
public:
    month() = default;
    explicit constexpr month(unsigned m) noexcept;

    constexpr month& operator++()    noexcept;
    constexpr month  operator++(int) noexcept;
    constexpr month& operator--()    noexcept;
    constexpr month  operator--(int) noexcept;

    constexpr month& operator+=(const months& m) noexcept;
    constexpr month& operator-=(const months& m) noexcept;

    explicit constexpr operator unsigned() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const month& x, const month& y) noexcept;
constexpr bool operator!=(const month& x, const month& y) noexcept;
constexpr bool operator< (const month& x, const month& y) noexcept;
constexpr bool operator> (const month& x, const month& y) noexcept;
constexpr bool operator<=(const month& x, const month& y) noexcept;
constexpr bool operator>=(const month& x, const month& y) noexcept;
```

```
constexpr month  operator+(const month&  x, const months& y) noexcept;
constexpr month  operator+(const months& x,  const month& y) noexcept;
constexpr month  operator-(const month&  x, const months& y) noexcept;
constexpr months operator-(const month&  x,  const month& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month& m);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const month& m);

template <class charT, class traits, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                month& m, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);
```

`month` is a trivially copyable class type.
`month` is a standard-layout class type.

```
explicit constexpr month::month(unsigned m) noexcept;
```

Effects: Constructs an object of type `month` by constructing `m_` with `m`. The value held is unspecified if `m` is not in the range `[0, 255]`.

```
constexpr month& month::operator++() noexcept;
```

Effects: `*this += months{1}`.

Returns: `*this`.

```
constexpr month month::operator++(int) noexcept;
```

Effects: `++(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr month& month::operator--() noexcept;
```

Effects: `*this -= months{1}`.

Returns: `*this`.

```
constexpr month month::operator--(int) noexcept;
```

Effects: `--(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr month& month::operator+=(const months& m) noexcept;
```

Effects: `*this = *this + m`.

Returns: `*this`.

```
constexpr month& month::operator-=(const months& m) noexcept;
```

Effects: `*this = *this - m`.

Returns: `*this`.

```
explicit constexpr month::operator unsigned() const noexcept;
```

Returns: `m_`.

```
constexpr bool month::ok() const noexcept;
```

Returns: `1 <= m_ && m_ <= 12`.

```
constexpr bool operator==(const month& x, const month& y) noexcept;
```

Returns: `unsigned{x} == unsigned{y}`.

```
constexpr bool operator!=(const month& x, const month& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const month& x, const month& y) noexcept;
```

Returns: `unsigned{x} < unsigned{y}`.

```
constexpr bool operator> (const month& x, const month& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const month& x, const month& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const month& x, const month& y) noexcept;
```

Returns: `!(x < y)`.

```
constexpr month  operator+(const month&  x, const months& y) noexcept;
```

Returns: `month{modulo(static_cast<long long>(unsigned{x}) + (y.count() - 1), 12) + 1}` where `modulo(n, 12)` computes the remainder of `n` divided by `12` using Euclidean division. [*Note:* Given a divisor of 12, Euclidean division truncates towards negative infinity and always produces a remainder in the range of `[0, 11]`. Assuming no overflow in the signed summation, this operation results in a `month` holding a value in the range `[1, 12]` even if `!x.ok()`. — *end note*]

[*Example:* `February + months{11} == January`. — *end example*]

```
constexpr month  operator+(const months& x, const month&  y) noexcept;
```

Returns: `y + x`.

```
constexpr month  operator-(const month&  x, const months& y) noexcept;
```

Returns: `x + -y`.

```
constexpr months operator-(const month&  x, const month&  y) noexcept;
```

Returns: If `x.ok() == true` and `y.ok() == true`, returns a value of `months` in the range of `months{0}` to `months{11}` inclusive. Otherwise the value returned is unspecified.

Remarks: If `x.ok() == true` and `y.ok() == true`, the returned value `m` shall satisfy the equality: `y + m == x`.

[*Example:* `January - February == months{11}` . — *end example*]

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const month& m);
```

Effects: If `m.ok() == true` inserts `format(os.getloc(), fmt, m)` where `fmt` is `"%b"` widened to `charT`. Otherwise inserts `unsigned{m} << " is not a valid month"`.

Returns: `os`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const month& m);
```

Effects: Streams `m` into `os` using the format specified by the NTCTS `fmt`. `fmt` encoding follows the rules specified by [time.format].

Returns: `os`.

```
template <class charT, class traits, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            month& m, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream `is` into the `month m` using the format flags given in the NTCTS `fmt` as specified in [time.parse].

If the parse fails to decode a valid month, `is.setstate(ios_base::failbit)` shall be called and `m` shall not be modified.

If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

Returns: `is`.

23.17.10.4 Class `year` [time.calendar.year]

`year` represents a year in the civil calendar. It shall represent values in the range `[min(), max()]`. It can be constructed with any `int` value, which will be subsequently truncated to fit into `year` object's internal unspecified storage. `year` is equality and less-than comparable, and participates in basic arithmetic with `years` representing the quantity between any two `year` objects. One can form a `year` literal with `y`. And one can stream out a `year`. `year` has explicit conversions to and from `int`.

```
class year
{
```

```
    short y_;  // exposition only
public:
    year() = default;
    explicit constexpr year(int y) noexcept;

    constexpr year& operator++()    noexcept;
    constexpr year  operator++(int) noexcept;
    constexpr year& operator--()    noexcept;
    constexpr year  operator--(int) noexcept;

    constexpr year& operator+=(const years& y) noexcept;
    constexpr year& operator-=(const years& y) noexcept;

    constexpr year operator+() const noexcept;
    constexpr year operator-() const noexcept;

    constexpr bool is_leap() const noexcept;

    explicit constexpr operator int() const noexcept;
    constexpr bool ok() const noexcept;

    static constexpr year min() noexcept;
    static constexpr year max() noexcept;
};

constexpr bool operator==(const year& x, const year& y) noexcept;
constexpr bool operator!=(const year& x, const year& y) noexcept;
constexpr bool operator< (const year& x, const year& y) noexcept;
constexpr bool operator> (const year& x, const year& y) noexcept;
constexpr bool operator<=(const year& x, const year& y) noexcept;
constexpr bool operator>=(const year& x, const year& y) noexcept;

constexpr year  operator+(const year&  x, const years& y) noexcept;
constexpr year  operator+(const years& x, const year&  y) noexcept;
constexpr year  operator-(const year&  x, const years& y) noexcept;
constexpr years operator-(const year&  x, const year&  y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year& y);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const year& y);

template <class charT, class traits, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                year& y, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);
```

`year` is a trivially copyable class type.
`year` is a standard-layout class type.

```
explicit constexpr year::year(int y) noexcept;
```

Effects: Constructs an object of type `year` by constructing `y_` with `y`. The value held is unspecified if `y` is not in the range `[-32767, 32767]`.

```
constexpr year& year::operator++() noexcept;
```

Effects: `++y_`.

Returns: `*this`.

```
constexpr year year::operator++(int) noexcept;
```

Effects: `++(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr year& year::operator--() noexcept;
```

Effects: `--y_`.

Returns: `*this`.

```
constexpr year year::operator--(int) noexcept;
```

Effects: `--(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr year& year::operator+=(const years& y) noexcept;
```

Effects: `*this = *this + y`.

Returns: `*this`.

```
constexpr year& year::operator-=(const years& y) noexcept;
```

Effects: `*this = *this - y`.

Returns: `*this`.

```
constexpr year year::operator+() const noexcept;
```

Returns: `*this`.

```
constexpr year year::operator-() const noexcept;
```

Returns: `year{-y_}`.

```
constexpr bool year::is_leap() const noexcept;
```

Returns: `y_ % 4 == 0 && (y_ % 100 != 0 || y_ % 400 == 0)`.

```
explicit constexpr year::operator int() const noexcept;
```

Returns: `y_`.

```
constexpr bool year::ok() const noexcept;
```

Returns: `min() <= y_ && y_ <= max()`.

```
static constexpr year year::min() noexcept;
```

Returns: `year{-32767}`.

```
static constexpr year year::max() noexcept;
```

Returns: `year{32767}`.

```
constexpr bool operator==(const year& x, const year& y) noexcept;
```

Returns: `int{x} == int{y}`.

```
constexpr bool operator!=(const year& x, const year& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const year& x, const year& y) noexcept;
```

Returns: `int{x} < int{y}`.

```
constexpr bool operator> (const year& x, const year& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const year& x, const year& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const year& x, const year& y) noexcept;
```

Returns: `!(x < y)`.

```
constexpr year  operator+(const year&  x, const years& y) noexcept;
```

Returns: `year{int{x} + y.count()}`.

```
constexpr year  operator+(const years& x, const year&  y) noexcept;
```

Returns: `y + x`.

```
constexpr year  operator-(const year&  x, const years& y) noexcept;
```

Returns: `x + -y`.

```
constexpr years operator-(const year&  x, const year&  y) noexcept;
```

Returns: `years{int{x} - int{y}}`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year& y);
```

Effects: Inserts `format(fmt, y)` where `fmt` is `"%Y"` widened to `charT`. If `!y.ok()`, appends with `" is not a valid year"`.

Returns: `os`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const year& y);
```

Effects: Streams `y` into `os` using the format specified by the NTCTS `fmt`. `fmt` encoding follows the rules specified by [time.format].

Returns: `os`.

```
template <class charT, class traits, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            year& y, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream `is` into the `year y` using the format flags given in the NTCTS `fmt` as specified in [time.parse].

If the parse fails to decode a valid year, `is.setstate(ios_base::failbit)` shall be called and `y` shall not be modified.

If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

Returns: `is`.

```
constexpr year operator "" y(unsigned long long y) noexcept;
```

Returns: `year{static_cast<int>(y)}`.

23.17.10.5 Class `weekday` [time.calendar.weekday]

`weekday` represents a day of the week in the civil calendar. It normally holds values in the range 0 to 6, corresponding to Sunday through Saturday. However it may hold non-negative values outside this range. It can be constructed with any `unsigned` value, which will be subsequently truncated to fit into `weekday` object's unspecified internal storage. `weekday` is equality comparable. `weekday` is not less-than comparable because there is no universal consensus on which day is the first day of the week. This design chooses the encoding of 0 to 6 to represent Sunday through Saturday only because this is consistent with existing C and C++ practice. However `weekday` object's arithmetic operations treat the days of the week as a circular range, with no beginning and no end. One can stream out a `weekday`. `weekday` has explicit conversions to and from `unsigned`. There are 7 `weekday` constants, one for each day of the week.

A `weekday` can be implicitly constructed from a `sys_days`. This is the computation that discovers the day of the week of an arbitrary date.

A `weekday` can be indexed with either `unsigned` or `last`. This produces new types which represent the first, second, third, fourth, fifth, or last weekdays of a month.

```
class weekday
{
    unsigned char wd_;  // exposition only
public:
    weekday() = default;
    explicit constexpr weekday(unsigned wd) noexcept;
    constexpr weekday(const sys_days& dp) noexcept;
    explicit constexpr weekday(const local_days& dp) noexcept;

    constexpr weekday& operator++()    noexcept;
    constexpr weekday  operator++(int) noexcept;
    constexpr weekday& operator--()    noexcept;
    constexpr weekday  operator--(int) noexcept;

    constexpr weekday& operator+=(const days& d) noexcept;
    constexpr weekday& operator-=(const days& d) noexcept;

    explicit constexpr operator unsigned() const noexcept;
    constexpr bool ok() const noexcept;

    constexpr weekday_indexed operator[](unsigned index) const noexcept;
    constexpr weekday_last    operator[](last_spec) const noexcept;
};

constexpr bool operator==(const weekday& x, const weekday& y) noexcept;
constexpr bool operator!=(const weekday& x, const weekday& y) noexcept;

constexpr weekday operator+(const weekday& x, const days&    y) noexcept;
constexpr weekday operator+(const days&    x, const weekday& y) noexcept;
constexpr weekday operator-(const weekday& x, const days&    y) noexcept;
constexpr days    operator-(const weekday& x, const weekday& y) noexcept;
```

```
template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const weekday& wd);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const weekday& wd);

template <class charT, class traits, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                weekday& wd, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);
```

`weekday` is a trivially copyable class type.
`weekday` is a standard-layout class type.

```
explicit constexpr weekday::weekday(unsigned wd) noexcept;
```

Effects: Constructs an object of type `weekday` by constructing `wd_` with `wd`. The value held is unspecified if `wd` is not in the range `[0, 255]`.

```
constexpr weekday(const sys_days& dp) noexcept;
```

Effects: Constructs an object of type `weekday` by computing what day of the week corresponds to the `sys_days dp`, and representing that day of the week in `wd_`.

[*Example:* If `dp` represents 1970-01-01, the constructed `weekday` represents Thursday by storing 4 in `wd_`. — *end example*]

```
explicit constexpr weekday(const local_days& dp) noexcept;
```

Effects: Constructs an object of type `weekday` by computing what day of the week corresponds to the `local_days dp`, and representing that day of the week in `wd_`.

The value after construction shall be identical to that constructed from `sys_days{dp.time_since_epoch()}`.

```
constexpr weekday& weekday::operator++() noexcept;
```

Effects: `*this += days{1}`.

Returns: `*this`.

```
constexpr weekday weekday::operator++(int) noexcept;
```

Effects: `++(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr weekday& weekday::operator--() noexcept;
```

Effects: `*this -= days{1}`.

Returns: `*this`.

```
constexpr weekday weekday::operator--(int) noexcept;
```

Effects: `--(*this)`.

Returns: A copy of `*this` as it existed on entry to this member function.

```
constexpr weekday& weekday::operator+=(const days& d) noexcept;
```

Effects: `*this = *this + d`.

Returns: `*this`.

```
constexpr weekday& weekday::operator-=(const days& d) noexcept;
```

Effects: `*this = *this - d`.

Returns: `*this`.

```
explicit constexpr weekday::operator unsigned() const noexcept;
```

Returns: `wd_`.

```
constexpr bool weekday::ok() const noexcept;
```

Returns: `wd_ <= 6`.

```
constexpr weekday_indexed weekday::operator[](unsigned index) const noexcept;
```

Returns: `{*this, index}`.

```
constexpr weekday_last weekday::operator[](last_spec) const noexcept;
```

Returns: `weekday_last{*this}`.

```
constexpr bool operator==(const weekday& x, const weekday& y) noexcept;
```

Returns: `unsigned{x} == unsigned{y}`.

```
constexpr bool operator!=(const weekday& x, const weekday& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr weekday  operator+(const weekday&  x, const days& y) noexcept;
```

Returns: `weekday{modulo(static_cast<long long>(unsigned{x}) + y.count(), 7)}` where `modulo(n, 7)` computes the remainder of `n` divided by `7` using Euclidean division. [*Note:* Given a divisor of 7, Euclidean division truncates towards negative infinity and always produces a remainder in the range of `[0, 6]`. Assuming no overflow in the signed summation, this operation results in a `weekday` holding a value in the range `[0, 6]` even if `!x.ok()`. — *end note*]

— *end example*]*Example:* `Monday + days{6} == Sunday`. — *end example*]

```
constexpr weekday  operator+(const days& x, const weekday&  y) noexcept;
```

Returns: `y + x`.

```
constexpr weekday  operator-(const weekday&  x, const days& y) noexcept;
```

Returns: `x + -y`.

```
constexpr days operator-(const weekday&  x, const weekday&  y) noexcept;
```

Returns: If `x.ok() == true` and `y.ok() == true`, returns a value of `days` in the range of `days{0}` to `days{6}` inclusive. Otherwise the value returned is unspecified.

Remarks: If `x.ok() == true` and `y.ok() == true`, the returned value `d` shall satisfy the equality: `y + d == x`.

[*Example:* `Sunday - Monday == days{6}`. — *end example*]

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const weekday& wd);
```

Effects: If `wd.ok() == true` inserts `format(os.getloc(), fmt, wd)` where `fmt` is `"%a"` widened to `charT`. Otherwise inserts `unsigned{wd} << " is not a valid weekday"`.

Returns: `os`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const weekday& wd);
```

Effects: Streams `wd` into `os` using the format specified by the NTCTS `fmt`. `fmt` encoding follows the rules specified by [time.format].

Returns: `os`.

```
template <class charT, class traits, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            weekday& wd, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream `is` into the `weekday wd` using the format flags given in the NTCTS `fmt` as specified in [time.parse].

If the parse fails to decode a valid weekday, `is.setstate(ios_base::failbit)` shall be called and `wd` shall not be modified.

If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

Returns: `is`.

23.17.10.6 Class `weekday_indexed` [time.calendar.weekday_indexed]

`weekday_indexed` represents a `weekday` and a small index in the range 1 to 5. This class is used to represent the first, second, third, fourth, or fifth weekday of a month. It is most easily constructed by indexing a `weekday`.

[*Example:*

```
constexpr auto wdi = Sunday[2];  // wdi is the second Sunday of an as yet unspecified month
static_assert(wdi.weekday() == Sunday);
static_assert(wdi.index() == 2);
```

— *end example*]

```
class weekday_indexed
{
    chrono::weekday  wd_;     // exposition only
    unsigned char    index_;  // exposition only

public:
    weekday_indexed() = default;
    constexpr weekday_indexed(const chrono::weekday& wd, unsigned index) noexcept;

    constexpr chrono::weekday weekday() const noexcept;
    constexpr unsigned        index()   const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const weekday_indexed& x, const weekday_indexed& y) noexcept;
constexpr bool operator!=(const weekday_indexed& x, const weekday_indexed& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const weekday_indexed& wdi);
```

`weekday_indexed` is a trivially copyable class type.
`weekday_indexed` is a standard-layout class type.

```
constexpr weekday_indexed::weekday_indexed(const chrono::weekday& wd, unsigned index) noexcept;
```

Effects: Constructs an object of type `weekday_indexed` by constructing `wd_` with `wd` and `index_` with `index`. The values held are unspecified if `!wd.ok()` or `index` is not in the range `[1, 5]`.

```
constexpr weekday weekday_indexed::weekday() const noexcept;
```

Returns: `wd_`.

```
constexpr unsigned weekday_indexed::index() const noexcept;
```

Returns: `index_`.

```
constexpr bool weekday_indexed::ok() const noexcept;
```

Returns: `wd_.ok() && 1 <= index_ && index_ <= 5`.

```
constexpr bool operator==(const weekday_indexed& x, const weekday_indexed& y) noexcept;
```

Returns: `x.weekday() == y.weekday() && x.index() == y.index()`.

```
constexpr bool operator!=(const weekday_indexed& x, const weekday_indexed& y) noexcept;
```

Returns: `!(x == y)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const weekday_indexed& wdi);
```

Effects: `os << wdi.weekday() << '[' << wdi.index()`. If `wdi.index()` is in the range `[1, 5]`, appends with `']'`, otherwise appends with `" is not a valid index]"`.

Returns: `os`.

23.17.10.7 Class `weekday_last` [time.calendar.weekday_last]

`weekday_last` represents the last `weekday` of a month. It is most easily constructed by indexing a `weekday` with `last`.

[*Example:*

```
constexpr auto wdl = Sunday[last];  // wdl is the last Sunday of an as yet unspecified month
static_assert(wdl.weekday() == Sunday);
```

— *end example*]

```
class weekday_last
{
    chrono::weekday wd_;  // exposition only

public:
    explicit constexpr weekday_last(const chrono::weekday& wd) noexcept;
```

```
    constexpr chrono::weekday weekday() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const weekday_last& x, const weekday_last& y) noexcept;
constexpr bool operator!=(const weekday_last& x, const weekday_last& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const weekday_last& wdl);
```

`weekday_last` is a trivially copyable class type.
`weekday_last` is a standard-layout class type.

```
explicit constexpr weekday_last::weekday_last(const chrono::weekday& wd) noexcept;
```

Effects: Constructs an object of type `weekday_last` by constructing `wd_` with `wd`.

```
constexpr weekday weekday_last::weekday() const noexcept;
```

Returns: `wd_`.

```
constexpr bool weekday_last::ok() const noexcept;
```

Returns: `wd_.ok()`.

```
constexpr bool operator==(const weekday_last& x, const weekday_last& y) noexcept;
```

Returns: `x.weekday() == y.weekday()`.

```
constexpr bool operator!=(const weekday_last& x, const weekday_last& y) noexcept;
```

Returns: `!(x == y)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const weekday_last& wdl);
```

Returns: `os << wdl.weekday() << "[last]"`.

23.17.10.8 Class `month_day` [time.calendar.month_day]

`month_day` represents a specific `day` of a specific `month`, but with an unspecified `year`. One can observe the different components. One can assign a new value. `month_day` is equality comparable and less-than comparable. One can stream out a `month_day`.

```
class month_day
{
    chrono::month m_;  // exposition only
    chrono::day   d_;  // exposition only

public:
    month_day() = default;
    constexpr month_day(const chrono::month& m, const chrono::day& d) noexcept;

    constexpr chrono::month month() const noexcept;
    constexpr chrono::day   day()   const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
constexpr bool operator!=(const month_day& x, const month_day& y) noexcept;
constexpr bool operator< (const month_day& x, const month_day& y) noexcept;
constexpr bool operator> (const month_day& x, const month_day& y) noexcept;
constexpr bool operator<=(const month_day& x, const month_day& y) noexcept;
constexpr bool operator>=(const month_day& x, const month_day& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_day& md);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const month_day& md);

template <class charT, class traits, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                month_day& md, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);
```

`month_day` is a trivially copyable class type.
`month_day` is a standard-layout class type.

```
constexpr month_day::month_day(const chrono::month& m, const chrono::day& d) noexcept;
```

Effects: Constructs an object of type `month_day` by constructing `m_` with `m`, and `d_` with `d`.

```
constexpr month month_day::month() const noexcept;
```

Returns: `m_`.

```
constexpr day month_day::day() const noexcept;
```

Returns: `d_`.

```
constexpr bool month_day::ok() const noexcept;
```

Returns: `true` if `m_.ok()` is true, and if `1d <= d_`, and if `d_ <=` the number of days in month `m_`. For `m_ == February` the number of days is considered to be 29. Otherwise returns `false`.

```
constexpr bool operator==(const month_day& x, const month_day& y) noexcept;
```

Returns: `x.month() == y.month() && x.day() == y.day()`.

```
constexpr bool operator!=(const month_day& x, const month_day& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const month_day& x, const month_day& y) noexcept;
```

Returns: If `x.month() < y.month()` returns `true`. Else if `x.month() > y.month()` returns `false`. Else returns `x.day() < y.day()`.

```
constexpr bool operator> (const month_day& x, const month_day& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const month_day& x, const month_day& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const month_day& x, const month_day& y) noexcept;
```

Returns: `!(x < y)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const month_day& md);
```

Returns: `os << md.month() << '/' << md.day()`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const month_day& md);
```

Effects: Streams `md` into `os` using the format specified by the NTCTS `fmt`. `fmt` encoding follows the rules specified by [time.format].

Returns: `os`.

```
template <class charT, class traits, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            month_day& md, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream `is` into the `month_day md` using the format flags given in the NTCTS `fmt` as specified in [time.parse].

If the parse fails to decode a valid `month_day`, `is.setstate(ios_base::failbit)` shall be called and `md` shall not be modified.

If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

Returns: `is`.

23.17.10.9 Class `month_day_last` [time.calendar.month_day_last]

`month_day_last` represents the last `day` of a `month`. It is most easily constructed using the expression `m/last` or `last/m`, where `m` is an expression with type `month`.

[*Example:*

```
constexpr auto mdl = February/last;  // mdl is the last day of February of an as yet unspecified year
static_assert(mdl.month() == February);
```

— *end example*]

```
class month_day_last
{
    chrono::month m_;  // exposition only

public:
    explicit constexpr month_day_last(const chrono::month& m) noexcept;

    constexpr chrono::month month() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator!=(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator< (const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator> (const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator<=(const month_day_last& x, const month_day_last& y) noexcept;
constexpr bool operator>=(const month_day_last& x, const month_day_last& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_day_last& mdl);
```

`month_day_last` is a trivially copyable class type.
`month_day_last` is a standard-layout class type.

```
explicit constexpr month_day_last::month_day_last(const chrono::month& m) noexcept;
```

Effects: Constructs an object of type `month_day_last` by constructing `m_` with `m`.

```
constexpr month month_day_last::month() const noexcept;
```

Returns: `m_`.

```
constexpr bool month_day_last::ok() const noexcept;
```

Returns: `m_.ok()`.

```
constexpr bool operator==(const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `x.month() == y.month()`.

```
constexpr bool operator!=(const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `x.month() < y.month()`.

```
constexpr bool operator> (const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const month_day_last& x, const month_day_last& y) noexcept;
```

Returns: `!(x < y)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const month_day_last& mdl);
```

Effects: `os << mdl.month() << "/last"`.

23.17.10.10 Class `month_weekday` [time.calendar.month_weekday]

`month_weekday` represents the nth `weekday` of a `month`, of an as yet unspecified `year`. To do this the `month_weekday` stores a `month` and a `weekday_indexed`.

[*Example:*

```
constexpr auto mwd = February/Tueday[3];  // mwd is the third Tuesday of February of an as yet unspecified year
static_assert(mwd.month() == February);
static_assert(mwd.weekday_indexed() == Tueday[3]);
```

— *end example*]

```
class month_weekday
```

```
{
    chrono::month           m_;    // exposition only
    chrono::weekday_indexed wdi_;  // exposition only
public:
    constexpr month_weekday(const chrono::month& m, const chrono::weekday_indexed& wdi) noexcept;

    constexpr chrono::month           month()           const noexcept;
    constexpr chrono::weekday_indexed weekday_indexed() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const month_weekday& x, const month_weekday& y) noexcept;
constexpr bool operator!=(const month_weekday& x, const month_weekday& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_weekday& mwd);
```

`month_weekday` is a trivially copyable class type.
`month_weekday` is a standard-layout class type.

```
constexpr month_weekday::month_weekday(const chrono::month& m, const chrono::weekday_indexed& wdi) noexcept;
```

Effects: Constructs an object of type `month_weekday` by constructing `m_` with `m`, and `wdi_` with `wdi`.

```
constexpr month month_weekday::month() const noexcept;
```

Returns: `m_`.

```
constexpr weekday_indexed month_weekday::weekday_indexed() const noexcept;
```

Returns: `wdi_`.

```
constexpr bool month_weekday::ok() const noexcept;
```

Returns: `m_.ok() && wdi_.ok()`.

```
constexpr bool operator==(const month_weekday& x, const month_weekday& y) noexcept;
```

Returns: `x.month() == y.month() && x.weekday_indexed() == y.weekday_indexed()`.

```
constexpr bool operator!=(const month_weekday& x, const month_weekday& y) noexcept;
```

Returns: `!(x == y)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const month_weekday& mwd);
```

Returns: `os << mwd.month() << '/' << mwd.weekday_indexed()`.

23.17.10.11 Class `month_weekday_last` [time.calendar.month_weekday_last]

`month_weekday_last` represents the last `weekday` of a `month`, of an as yet unspecified `year`. To do this the `month_weekday_last` stores a `month` and a `weekday_last`.

[*Example:*

```
constexpr auto mwd = February/Tueday[last];  // mwd is the last Tuesday of February of an as yet unspecified year
static_assert(mwd.month() == February);
static_assert(mwd.weekday_last() == Tueday[last]);
```

— *end example*]

```
class month_weekday_last
{
    chrono::month        m_;     // exposition only
    chrono::weekday_last wdl_;   // exposition only
public:
    constexpr month_weekday_last(const chrono::month& m,
                                 const chrono::weekday_last& wdl) noexcept;

    constexpr chrono::month        month()        const noexcept;
    constexpr chrono::weekday_last weekday_last() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const month_weekday_last& x, const month_weekday_last& y) noexcept;
constexpr bool operator!=(const month_weekday_last& x, const month_weekday_last& y) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const month_weekday_last& mwdl);
```

`month_weekday_last` is a trivially copyable class type.
`month_weekday_last` is a standard-layout class type.

```
constexpr month_weekday_last::month_weekday_last(const chrono::month& m,
                                                 const chrono::weekday_last& wdl) noexcept;
```

Effects: Constructs an object of type `month_weekday_last` by constructing `m_` with `m`, and `wdl_` with `wdl`.

```
constexpr month month_weekday_last::month() const noexcept;
```

Returns: `m_`.

```
constexpr weekday_last month_weekday_last::weekday_last() const noexcept;
```

Returns: `wdl_`.

```
constexpr bool month_weekday_last::ok() const noexcept;
```

Returns: `m_.ok() && wdl_.ok()`.

```
constexpr bool operator==(const month_weekday_last& x, const month_weekday_last& y) noexcept;
```

Returns: `x.month() == y.month() && x.weekday_last() == y.weekday_last()`.

```
constexpr bool operator!=(const month_weekday_last& x, const month_weekday_last& y) noexcept;
```

Returns: `!(x == y)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const month_weekday_last& mwdl);
```

Effects: `os << mwdl.month() << '/' << mwdl.weekday_last()`.

23.17.10.12 Class `year_month` [time.calendar.year_month]

`year_month` represents a specific `month` of a specific `year`, but with an unspecified `day`. `year_month` is a field-based time point with a resolution of `months`. One can observe the different components. One can assign a new value. `year_month` is equality comparable and less-than comparable. One can stream out a `year_month`.

```
class year_month
{
    chrono::year  y_;  // exposition only
    chrono::month m_;  // exposition only

public:
    year_month() = default;
    constexpr year_month(const chrono::year& y, const chrono::month& m) noexcept;

    constexpr chrono::year  year()  const noexcept;
    constexpr chrono::month month() const noexcept;

    constexpr year_month& operator+=(const months& dm) noexcept;
    constexpr year_month& operator-=(const months& dm) noexcept;
    constexpr year_month& operator+=(const years& dy)  noexcept;
    constexpr year_month& operator-=(const years& dy)  noexcept;

    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
constexpr bool operator!=(const year_month& x, const year_month& y) noexcept;
constexpr bool operator< (const year_month& x, const year_month& y) noexcept;
constexpr bool operator> (const year_month& x, const year_month& y) noexcept;
constexpr bool operator<=(const year_month& x, const year_month& y) noexcept;
constexpr bool operator>=(const year_month& x, const year_month& y) noexcept;

constexpr year_month operator+(const year_month& ym, const months& dm) noexcept;
constexpr year_month operator+(const months& dm, const year_month& ym) noexcept;
constexpr year_month operator-(const year_month& ym, const months& dm) noexcept;
constexpr months operator-(const year_month& x, const year_month& y) noexcept;
constexpr year_month operator+(const year_month& ym, const years& dy) noexcept;
constexpr year_month operator+(const years& dy, const year_month& ym) noexcept;
constexpr year_month operator-(const year_month& ym, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month& ym);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const year_month& ym);

template <class charT, class traits, class Alloc = allocator<charT>>
```

```
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                year_month& ym, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);
```

`year_month` is a trivially copyable class type.
`year_month` is a standard-layout class type.

```
constexpr year_month::year_month(const chrono::year& y, const chrono::month& m) noexcept;
```

Effects: Constructs an object of type `year_month` by constructing `y_` with `y`, and `m_` with `m`.

```
constexpr year year_month::year() const noexcept;
```

Returns: `y_`.

```
constexpr month year_month::month() const noexcept;
```

Returns: `m_`.

```
constexpr year_month& operator+=(const months& dm) noexcept;
```

Effects: `*this = *this + dm`.

Returns: `*this`.

```
constexpr year_month& operator-=(const months& dm) noexcept;
```

Effects: `*this = *this - dm`.

Returns: `*this`.

```
constexpr year_month& operator+=(const years& dy) noexcept;
```

Effects: `*this = *this + dy`.

Returns: `*this`.

```
constexpr year_month& operator-=(const years& dy) noexcept;
```

Effects: `*this = *this - dy`.

Returns: `*this`.

```
constexpr bool year_month::ok() const noexcept;
```

Returns: `y_.ok() && m_.ok()`.

```
constexpr bool operator==(const year_month& x, const year_month& y) noexcept;
```

Returns: `x.year() == y.year() && x.month() == y.month()`.

```
constexpr bool operator!=(const year_month& x, const year_month& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const year_month& x, const year_month& y) noexcept;
```

Returns: If `x.year() < y.year()` returns `true`. Else if `x.year() > y.year()` returns `false`. Else returns `x.month() < y.month()`.

```
constexpr bool operator> (const year_month& x, const year_month& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const year_month& x, const year_month& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const year_month& x, const year_month& y) noexcept;
```

Returns: `!(x < y)`.

```
constexpr year_month operator+(const year_month& ym, const months& dm) noexcept;
```

Returns: A `year_month` value `z` such that `z - ym == dm`.

Complexity: O(1) with respect to the value of `dm`.

```
constexpr year_month operator+(const months& dm, const year_month& ym) noexcept;
```

Returns: `ym + dm`.

```
constexpr year_month operator-(const year_month& ym, const months& dm) noexcept;
```

Returns: `ym + -dm`.

```
constexpr months operator-(const year_month& x, const year_month& y) noexcept;
```

Returns: `x.year() - y.year() + months{static_cast<int>(unsigned{x.month()}) - static_cast<int>(unsigned{y.month()})}`.

```
constexpr year_month operator+(const year_month& ym, const years& dy) noexcept;
```

Returns: `(ym.year() + dy) / ym.month()`.

```
constexpr year_month operator+(const years& dy, const year_month& ym) noexcept;
```

Returns: `ym + dy`.

```
constexpr year_month operator-(const year_month& ym, const years& dy) noexcept;
```

Returns: `ym + -dy`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month& ym);
```

Effects: `os << ym.year() << '/' << ym.month()`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const year_month& ym);
```

Effects: Streams `ym` into `os` using the format specified by the NTCTS `fmt`. `fmt` encoding follows the rules specified by [time.format].

Returns: `os`.

```
template <class charT, class traits, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            year_month& ym, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream `is` into the `year_month ym` using the format flags given in the NTCTS `fmt` as specified in [time.parse].

If the parse fails to decode a valid `year_month`, `is.setstate(ios_base::failbit)` shall be called and `ym` shall not be modified.

If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

Returns: `is`.

23.17.10.13 Class `year_month_day` [time.calendar.year_month_day]

`year_month_day` represents a specific `year`, `month`, and `day`. `year_month_day` is a field-based time point with a resolution of `days`. One can observe each data member. `year_month_day` supports `years` and `months` oriented arithmetic, but not `days` oriented arithmetic. For the latter, there is a conversion to `sys_days` which efficiently supports `days` oriented arithmetic. There is also a conversion *from* `sys_days`. `year_month_day` is equality and less-than comparable.

```
class year_month_day
{
    chrono::year  y_;  // exposition only
    chrono::month m_;  // exposition only
    chrono::day   d_;  // exposition only

public:
    year_month_day() = default;
    constexpr year_month_day(const chrono::year& y, const chrono::month& m, const chrono::day& d) noexcept;
    constexpr year_month_day(const year_month_day_last& ymdl) noexcept;
    constexpr year_month_day(const sys_days& dp) noexcept;
    explicit constexpr year_month_day(const local_days& dp) noexcept;

    constexpr year_month_day& operator+=(const months& m) noexcept;
    constexpr year_month_day& operator-=(const months& m) noexcept;
    constexpr year_month_day& operator+=(const years& y)  noexcept;
    constexpr year_month_day& operator-=(const years& y)  noexcept;

    constexpr chrono::year  year()  const noexcept;
    constexpr chrono::month month() const noexcept;
    constexpr chrono::day   day()   const noexcept;

    constexpr          operator sys_days()   const noexcept;
```

```
    explicit constexpr operator local_days() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator!=(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator< (const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator> (const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator<=(const year_month_day& x, const year_month_day& y) noexcept;
constexpr bool operator>=(const year_month_day& x, const year_month_day& y) noexcept;

constexpr year_month_day operator+(const year_month_day& ymd, const months& dm) noexcept;
constexpr year_month_day operator+(const months& dm, const year_month_day& ymd) noexcept;
constexpr year_month_day operator+(const year_month_day& ymd, const years& dy) noexcept;
constexpr year_month_day operator+(const years& dy, const year_month_day& ymd) noexcept;
constexpr year_month_day operator-(const year_month_day& ymd, const months& dm) noexcept;
constexpr year_month_day operator-(const year_month_day& ymd, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_day& ymd);

template <class charT, class traits>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const year_month_day& ymd);

template <class charT, class traits, class Alloc = allocator<charT>>
    basic_istream<charT, traits>&
    from_stream(basic_istream<charT, traits>& is, const charT* fmt,
                year_month_day& ymd, basic_string<charT, traits, Alloc>* abbrev = nullptr,
                minutes* offset = nullptr);
```

`year_month_day` is a trivially copyable class type.
`year_month_day` is a standard-layout class type.

```
constexpr year_month_day::year_month_day(const chrono::year& y, const chrono::month& m, const chrono::day& d) noexcept;
```

Effects: Constructs an object of type `year_month_day` by constructing `y_` with `y`, `m_` with `m`, and, `d_` with `d`.

```
constexpr year_month_day::year_month_day(const year_month_day_last& ymdl) noexcept;
```

Effects: Constructs an object of type `year_month_day` by constructing `y_` with `ymdl.year()`, `m_` with `ymdl.month()`, and, `d_` with `ymdl.day()`.

[*Note:* This conversion from `year_month_day_last` to `year_month_day` is more efficient than converting a `year_month_day_last` to a `sys_days`, and then converting that `sys_days` to a `year_month_day`. — *end note*]

```
constexpr year_month_day::year_month_day(const sys_days& dp) noexcept;
```

Effects: Constructs an object of type `year_month_day` which corresponds to the date represented by `dp`.

Remarks: For any value of `year_month_day`, `ymd`, for which `ymd.ok()` is `true`, this equality will also be `true`: `ymd == year_month_day{sys_days{ymd}}`.

```
explicit constexpr year_month_day::year_month_day(const local_days& dp) noexcept;
```

Effects: Constructs an object of type `year_month_day` which corresponds to the date represented by `dp`.

Remarks: Equivalent to constructing with `sys_days{dp.time_since_epoch()}`.

```
constexpr year_month_day& year_month_day::operator+=(const months& m) noexcept;
```

Effects: `*this = *this + m`.

Returns: `*this`.

```
constexpr year_month_day& year_month_day::operator-=(const months& m) noexcept;
```

Effects: `*this = *this - m`.

Returns: `*this`.

```
constexpr year_month_day& year_month_day::operator+=(const years& y) noexcept;
```

Effects: `*this = *this + y`.

Returns: `*this`.

```
constexpr year_month_day& year_month_day::operator-=(const years& y) noexcept;
```

Effects: `*this = *this - y`.

Returns: `*this`.

```
constexpr year year_month_day::year() const noexcept;
```

Returns: `y_`.

```
constexpr month year_month_day::month() const noexcept;
```

Returns: `m_`.

```
constexpr day year_month_day::day() const noexcept;
```

Returns: `d_`.

```
constexpr year_month_day::operator sys_days() const noexcept;
```

Returns: If `ok()`, returns a `sys_days` holding a count of `days` from the `sys_days` epoch to `*this` (a negative value if `*this` represents a date prior to the `sys_days` epoch). Otherwise if `y_.ok() && m_.ok() == true` returns a `sys_days` which is offset from `sys_days{y_/m_/last}` by the number of `days d_` is offset from `sys_days{y_/m_/last}.day()`. Otherwise the value returned is unspecified.

Remarks: A `sys_days` in the range `[days{-12687428}, days{11248737}]` which is converted to a `year_month_day`, shall have the same value when converted back to a `sys_days`.

[*Example*:

```
static_assert(year_month_day{sys_days{2017y/January/0}}  == 2016y/December/31);
static_assert(year_month_day{sys_days{2017y/January/31}} == 2017y/January/31);
static_assert(year_month_day{sys_days{2017y/January/32}} == 2017y/February/1);
```

—*end example*]

```
explicit constexpr year_month_day::operator local_days() const noexcept;
```

Returns: `local_days{sys_days{*this}.time_since_epoch()}`.

```
constexpr bool year_month_day::ok() const noexcept;
```

Returns: If `y_.ok()` is `true`, and `m_.ok()` is `true`, and `d_` is in the range `[1d, (y_/m_/last).day()]`, then returns `true`, else returns `false`.

```
constexpr bool operator==(const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: `x.year() == y.year() && x.month() == y.month() && x.day() == y.day()`.

```
constexpr bool operator!=(const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: If `x.year() < y.year()`, returns `true`. Else if `x.year() > y.year()` returns `false`. Else if `x.month() < y.month()`, returns `true`. Else if `x.month() > y.month()`, returns `false`. Else returns `x.day() < y.day()`.

```
constexpr bool operator> (const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const year_month_day& x, const year_month_day& y) noexcept;
```

Returns: `!(x < y)`.

```
constexpr year_month_day operator+(const year_month_day& ymd, const months& dm) noexcept;
```

Returns: `(ymd.year() / ymd.month() + dm) / ymd.day()`.

[*Note:* If `ymd.day()` is in the range `[1d, 28d]`, the resultant `year_month_day` shall return `true` from `ok()`. — *end note*]

```
constexpr year_month_day operator+(const months& dm, const year_month_day& ymd) noexcept;
```

Returns: `ymd + dm`.

```
constexpr year_month_day operator-(const year_month_day& ymd, const months& dm) noexcept;
```

Returns: `ymd + (-dm)`.

```
constexpr year_month_day operator+(const year_month_day& ymd, const years& dy) noexcept;
```

Returns: `(ymd.year() + dy) / ymd.month() / ymd.day()`.

[*Note:* If `ymd.month()` is `February` and `ymd.day()` is not in the range `[1d, 28d]`, the resultant `year_month_day` may return `false` from `ok()`. — *end note*]

```
constexpr year_month_day operator+(const years& dy, const year_month_day& ymd) noexcept;
```

Returns: `ymd + dy`.

```
constexpr year_month_day operator-(const year_month_day& ymd, const years& dy) noexcept;
```

Returns: `ymd + (-dy)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month_day& ymd);
```

Inserts `format(fmt, ymd)` where `fmt` is `"%F"` widened to `charT`. If `!ymd.ok()`, appends with `" is not a valid date"`.

Returns: `os`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt, const year_month_day& ymd);
```

Effects: Streams `ymd` into `os` using the format specified by the NTCTS `fmt`. `fmt` encoding follows the rules specified by [time.format].

Returns: `os`.

```
template <class charT, class traits, class Alloc = allocator<charT>>
basic_istream<charT, traits>&
from_stream(basic_istream<charT, traits>& is, const charT* fmt,
            year_month_day& ymd, basic_string<charT, traits, Alloc>* abbrev = nullptr,
            minutes* offset = nullptr);
```

Effects: Attempts to parse the input stream `is` into the `year_month_day ymd` using the format flags given in the NTCTS `fmt` as specified in [time.parse].

If the parse fails to decode a valid `year_month_day`, `is.setstate(ios_base::failbit)` shall be called and `ymd` shall not be modified.

If `%Z` is used and successfully parsed, that value will be assigned to `*abbrev` if `abbrev` is non-null.

If `%z` (or a modified variant) is used and successfully parsed, that value will be assigned to `*offset` if `offset` is non-null.

Returns: `is`.

23.17.10.14 Class `year_month_day_last` [time.calendar.year_month_day_last]

`year_month_day_last` represents a specific `year`, `month`, and the last `day` of the `month`. `year_month_day_last` is a field-based time point with a resolution of `days`, except that it is restricted to pointing to the last day of a year and month. One can observe each data member. The `day` member is computed on demand. `year_month_day_last` supports `years` and `months` oriented arithmetic, but not `days` oriented arithmetic. For the latter, there is a conversion to `sys_days` which efficiently supports `days` oriented arithmetic. `year_month_day_last` is equality and less-than comparable.

```
class year_month_day_last
{
    chrono::year           y_;    // exposition only
    chrono::month_day_last mdl_;  // exposition only

public:
    constexpr year_month_day_last(const chrono::year& y,
                                  const chrono::month_day_last& mdl) noexcept;

    constexpr year_month_day_last& operator+=(const months& m) noexcept;
    constexpr year_month_day_last& operator-=(const months& m) noexcept;
    constexpr year_month_day_last& operator+=(const years& y)  noexcept;
    constexpr year_month_day_last& operator-=(const years& y)  noexcept;

    constexpr chrono::year           year()           const noexcept;
    constexpr chrono::month          month()          const noexcept;
    constexpr chrono::month_day_last month_day_last() const noexcept;
    constexpr chrono::day            day()            const noexcept;

    constexpr          operator sys_days()   const noexcept;
    explicit constexpr operator local_days() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator!=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator< (const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

```
constexpr bool operator> (const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator<=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
constexpr bool operator>=(const year_month_day_last& x, const year_month_day_last& y) noexcept;

constexpr year_month_day_last operator+(const year_month_day_last& ymdl, const months& dm) noexcept;
constexpr year_month_day_last operator+(const months& dm, const year_month_day_last& ymdl) noexcept;
constexpr year_month_day_last operator+(const year_month_day_last& ymdl, const years& dy) noexcept;
constexpr year_month_day_last operator+(const years& dy, const year_month_day_last& ymdl) noexcept;
constexpr year_month_day_last operator-(const year_month_day_last& ymdl, const months& dm) noexcept;
constexpr year_month_day_last operator-(const year_month_day_last& ymdl, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_day_last& ymdl);
```

`year_month_day_last` is a trivially copyable class type.
`year_month_day_last` is a standard-layout class type.

```
constexpr year_month_day_last::year_month_day_last(const chrono::year& y,
                                                   const chrono::month_day_last& mdl) noexcept;
```

Effects: Constructs an object of type `year_month_day_last` by constructing `y_` with `y` and `mdl_` with `mdl`.

```
constexpr year_month_day_last& year_month_day_last::operator+=(const months& m) noexcept;
```

Effects: `*this = *this + m`.

Returns: `*this`.

```
constexpr year_month_day_last& year_month_day_last::operator-=(const months& m) noexcept;
```

Effects: `*this = *this - m`.

Returns: `*this`.

```
constexpr year_month_day_last& year_month_day_last::operator+=(const years& y) noexcept;
```

Effects: `*this = *this + y`.

Returns: `*this`.

```
constexpr year_month_day_last& year_month_day_last::operator-=(const years& y) noexcept;
```

Effects: `*this = *this - y`.

Returns: `*this`.

```
constexpr year year_month_day_last::year() const noexcept;
```

Returns: `y_`.

```
constexpr month year_month_day_last::month() const noexcept;
```

Returns: `mdl_.month()`.

```
constexpr month_day_last year_month_day_last::month_day_last() const noexcept;
```

Returns: `mdl_`.

```
constexpr day year_month_day_last::day() const noexcept;
```

Returns: A `day` representing the last day of the `year`, `month` pair represented by `*this`.

```
constexpr year_month_day_last::operator sys_days() const noexcept;
```

Returns: `sys_days{year()/month()/day()}`.

```
explicit constexpr year_month_day_last::operator local_days() const noexcept;
```

Returns: `local_days{sys_days{*this}.time_since_epoch()}`.

```
constexpr bool year_month_day_last::ok() const noexcept;
```

Returns: `y_.ok() && mdl_.ok()`.

```
constexpr bool operator==(const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: `x.year() == y.year() && x.month_day_last() == y.month_day_last()`.

```
constexpr bool operator!=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr bool operator< (const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: If `x.year() < y.year()`, returns `true`. Else if `x.year() > y.year()` returns `false`. Else returns `x.month_day_last() < y.month_day_last()`.

```
constexpr bool operator> (const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: `y < x`.

```
constexpr bool operator<=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const year_month_day_last& x, const year_month_day_last& y) noexcept;
```

Returns: `!(x < y)`.

```
constexpr year_month_day_last operator+(const year_month_day_last& ymdl, const months& dm) noexcept;
```

Returns: `(ymdl.year() / ymdl.month() + dm) / last`.

```
constexpr year_month_day_last operator+(const months& dm, const year_month_day_last& ymdl) noexcept;
```

Returns: `ymdl + dm`.

```
constexpr year_month_day_last operator-(const year_month_day_last& ymdl, const months& dm) noexcept;
```

Returns: `ymdl + (-dm)`.

```
constexpr year_month_day_last operator+(const year_month_day_last& ymdl, const years& dy) noexcept;
```

Returns: `{ymdl.year()+dy, ymdl.month_day_last()}`.

```
constexpr year_month_day_last operator+(const years& dy, const year_month_day_last& ymdl) noexcept;
```

Returns: `ymdl + dy`.

```
constexpr year_month_day_last operator-(const year_month_day_last& ymdl, const years& dy) noexcept;
```

Returns: `ymdl + (-dy)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month_day_last& ymdl);
```

Returns: `os << ymdl.year() << '/' << ymdl.month_day_last()`.

23.17.10.15 Class `year_month_weekday` [time.calendar.year_month_weekday]

`year_month_weekday` represents a specific `year`, `month`, and nth `weekday` of the `month`. `year_month_weekday` is a field-based time point with a resolution of `days`. One can observe each data member. `year_month_weekday` supports `years` and `months` oriented arithmetic, but not `days` oriented arithmetic. For the latter, there is a conversion to `sys_days` which efficiently supports `days` oriented arithmetic. `year_month_weekday` is equality comparable.

```
class year_month_weekday
{
    chrono::year            y_;    // exposition only
    chrono::month           m_;    // exposition only
    chrono::weekday_indexed wdi_;  // exposition only

public:
    year_month_weekday() = default;
    constexpr year_month_weekday(const chrono::year& y, const chrono::month& m,
                                 const chrono::weekday_indexed& wdi) noexcept;
    constexpr year_month_weekday(const sys_days& dp) noexcept;
    explicit constexpr year_month_weekday(const local_days& dp) noexcept;

    constexpr year_month_weekday& operator+=(const months& m) noexcept;
    constexpr year_month_weekday& operator-=(const months& m) noexcept;
    constexpr year_month_weekday& operator+=(const years& y)  noexcept;
    constexpr year_month_weekday& operator-=(const years& y)  noexcept;

    constexpr chrono::year            year()            const noexcept;
    constexpr chrono::month           month()           const noexcept;
    constexpr chrono::weekday         weekday()         const noexcept;
    constexpr unsigned                index()           const noexcept;
    constexpr chrono::weekday_indexed weekday_indexed() const noexcept;

    constexpr          operator sys_days()   const noexcept;
    explicit constexpr operator local_days() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr bool operator==(const year_month_weekday& x, const year_month_weekday& y) noexcept;
constexpr bool operator!=(const year_month_weekday& x, const year_month_weekday& y) noexcept;
```

```
constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const months& dm) noexcept;
constexpr year_month_weekday operator+(const months& dm, const year_month_weekday& ymwd) noexcept;
constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const years& dy) noexcept;
constexpr year_month_weekday operator+(const years& dy, const year_month_weekday& ymwd) noexcept;
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const months& dm) noexcept;
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_weekday& ymwdi);
```

`year_month_weekday` is a trivially copyable class type.
`year_month_weekday` is a standard-layout class type.

```
constexpr year_month_weekday::year_month_weekday(const chrono::year& y, const chrono::month& m,
                                                 const chrono::weekday_indexed& wdi) noexcept;
```

Effects: Constructs an object of type `year_month_weekday` by constructing `y_` with `y`, `m_` with `m`, and `wdi_` with `wdi`.

```
constexpr year_month_weekday(const sys_days& dp) noexcept;
```

Effects: Constructs an object of type `year_month_weekday` which corresponds to the date represented by `dp`.

Remarks: For any value of `year_month_weekday`, `ymdl`, for which `ymdl.ok()` is `true`, this equality will also be `true`: `ymdl == year_month_weekday{sys_days{ymdl}}`.

```
explicit constexpr year_month_weekday(const local_days& dp) noexcept;
```

Effects: Constructs an object of type `year_month_weekday` which corresponds to the date represented by `dp`.

Remarks: Equivalent to constructing with `sys_days{dp.time_since_epoch()}`.

```
constexpr year_month_weekday& year_month_weekday::operator+=(const months& m) noexcept;
```

Effects: `*this = *this + m`.

Returns: `*this`.

```
constexpr year_month_weekday& year_month_weekday::operator-=(const months& m) noexcept;
```

Effects: `*this = *this - m`.

Returns: `*this`.

```
constexpr year_month_weekday& year_month_weekday::operator+=(const years& y) noexcept;
```

Effects: `*this = *this + y`.

Returns: `*this`.

```
constexpr year_month_weekday& year_month_weekday::operator-=(const years& y) noexcept;
```

Effects: `*this = *this - y`.

Returns: `*this`.

```
constexpr year year_month_weekday::year() const noexcept;
```

Returns: `y_`.

```
constexpr month year_month_weekday::month() const noexcept;
```

Returns: `m_`.

```
constexpr weekday year_month_weekday::weekday() const noexcept;
```

Returns: `wdi_.weekday()`.

```
constexpr unsigned year_month_weekday::index() const noexcept;
```

Returns: `wdi_.index()`.

```
constexpr weekday_indexed year_month_weekday::weekday_indexed() const noexcept;
```

Returns: `wdi_`.

```
constexpr year_month_weekday::operator sys_days() const noexcept;
```

Returns: If `y_.ok() && m_.ok() && wdi_.weekday().ok()`, returns a `sys_days` which represents the date `(index() - 1)*7` days after the first `weekday()` of `year()/month()`. If `index()` is `0` the returned `sys_days` represents the date 7 days prior to the first `weekday()` of `year()/month()`. Otherwise the returned value is unspecified.

```
explicit constexpr year_month_weekday::operator local_days() const noexcept;
```

Returns: `local_days{sys_days{*this}.time_since_epoch()}`.

```
constexpr bool year_month_weekday::ok() const noexcept;
```

Returns: If any of `y_.ok()` or `m_.ok()` or `wdi_.ok()` is `false`, returns `false`. Else if `*this` represents a valid date, returns `true`, else returns `false`.

```
constexpr bool operator==(const year_month_weekday& x, const year_month_weekday& y) noexcept;
```

Returns: `x.year() == y.year() && x.month() == y.month() && x.weekday_indexed() == y.weekday_indexed()`.

```
constexpr bool operator!=(const year_month_weekday& x, const year_month_weekday& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const months& dm) noexcept;
```

Returns: `(ymwd.year() / ymwd.month() + dm) / ymwd.weekday_indexed()`.

```
constexpr year_month_weekday operator+(const months& dm, const year_month_weekday& ymwd) noexcept;
```

Returns: `ymwd + dm`.

```
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const months& dm) noexcept;
```

Returns: `ymwd + (-dm)`.

```
constexpr year_month_weekday operator+(const year_month_weekday& ymwd, const years& dy) noexcept;
```

Returns: `{ymwd.year()+dy, ymwd.month(), ymwd.weekday_indexed()}`.

```
constexpr year_month_weekday operator+(const years& dy, const year_month_weekday& ymwd) noexcept;
```

Returns: `ymwd + dm`.

```
constexpr year_month_weekday operator-(const year_month_weekday& ymwd, const years& dy) noexcept;
```

Returns: `ymwd + (-dm)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month_weekday& ymwd);
```

Returns: `os << ymwdi.year() << '/' << ymwdi.month() << '/' << ymwdi.weekday_indexed()`.

23.17.10.16 Class `year_month_weekday_last` [time.calendar.year_month_weekday_last]

`year_month_weekday_last` represents a specific `year`, `month`, and last `weekday` of the `month`. `year_month_weekday_last` is a field-based time point with a resolution of `days`, except that it is restricted to pointing to the last weekday of a year and month. One can observe each data member. `year_month_weekday_last` supports `years` and `months` oriented arithmetic, but not `days` oriented arithmetic. For the latter, there is a conversion to `sys_days` which efficiently supports `days` oriented arithmetic. `year_month_weekday_last` is equality comparable.

```
class year_month_weekday_last
{
    chrono::year         y_;    // exposition only
    chrono::month        m_;    // exposition only
    chrono::weekday_last wdl_;  // exposition only

public:
    constexpr year_month_weekday_last(const chrono::year& y, const chrono::month& m,
                                      const chrono::weekday_last& wdl) noexcept;

    constexpr year_month_weekday_last& operator+=(const months& m) noexcept;
    constexpr year_month_weekday_last& operator-=(const months& m) noexcept;
    constexpr year_month_weekday_last& operator+=(const years& y)  noexcept;
    constexpr year_month_weekday_last& operator-=(const years& y)  noexcept;

    constexpr chrono::year         year()         const noexcept;
    constexpr chrono::month        month()        const noexcept;
    constexpr chrono::weekday      weekday()      const noexcept;
    constexpr chrono::weekday_last weekday_last() const noexcept;

    constexpr          operator sys_days()   const noexcept;
    explicit constexpr operator local_days() const noexcept;
    constexpr bool ok() const noexcept;
};

constexpr
bool
operator==(const year_month_weekday_last& x, const year_month_weekday_last& y) noexcept;
```

```
constexpr
bool
operator!=(const year_month_weekday_last& x, const year_month_weekday_last& y) noexcept;

constexpr
year_month_weekday_last
operator+(const year_month_weekday_last& ymwdl, const months& dm) noexcept;

constexpr
year_month_weekday_last
operator+(const months& dm, const year_month_weekday_last& ymwdl) noexcept;

constexpr
year_month_weekday_last
operator+(const year_month_weekday_last& ymwdl, const years& dy) noexcept;

constexpr
year_month_weekday_last
operator+(const years& dy, const year_month_weekday_last& ymwdl) noexcept;

constexpr
year_month_weekday_last
operator-(const year_month_weekday_last& ymwdl, const months& dm) noexcept;

constexpr
year_month_weekday_last
operator-(const year_month_weekday_last& ymwdl, const years& dy) noexcept;

template <class charT, class traits>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const year_month_weekday_last& ymwdl);
```

`year_month_weekday_last` is a trivially copyable class type.
`year_month_weekday_last` is a standard-layout class type.

```
constexpr year_month_weekday_last::year_month_weekday_last(const chrono::year& y, const chrono::month& m,
                                                           const chrono::weekday_last& wdl) noexcept;
```

Effects: Constructs an object of type `year_month_weekday_last` by constructing `y_` with `y`, `m_` with `m`, and `wdl_` with `wdl`.

```
constexpr year_month_weekday_last& year_month_weekday_last::operator+=(const months& m) noexcept;
```

Effects: `*this = *this + m`.

Returns: `*this`.

```
constexpr year_month_weekday_last& year_month_weekday_last::operator-=(const months& m) noexcept;
```

Effects: `*this = *this - m`.

Returns: `*this`.

```
constexpr year_month_weekday_last& year_month_weekday_last::operator+=(const years& y) noexcept;
```

Effects: `*this = *this + y`.

Returns: `*this`.

```
constexpr year_month_weekday_last& year_month_weekday_last::operator-=(const years& y) noexcept;
```

Effects: `*this = *this - y`.

Returns: `*this`.

```
constexpr year year_month_weekday_last::year() const noexcept;
```

Returns: `y_`.

```
constexpr month year_month_weekday_last::month() const noexcept;
```

Returns: `m_`.

```
constexpr weekday year_month_weekday_last::weekday() const noexcept;
```

Returns: `wdl_.weekday()`.

```
constexpr weekday_last year_month_weekday_last::weekday_last() const noexcept;
```

Returns: `wdl_`.

```
constexpr year_month_weekday_last::operator sys_days() const noexcept;
```

Returns: If `ok() == true`, returns a `sys_days` which represents the last `weekday()` of `year()/month()`. Otherwise the

returned value is unspecified.

```
explicit constexpr year_month_weekday_last::operator local_days() const noexcept;
```

Returns: `local_days{sys_days{*this}.time_since_epoch()}`.

```
constexpr bool year_month_weekday_last::ok() const noexcept;
```

Returns: `y_.ok() && m_.ok() && wdl_.ok()`.

```
constexpr bool operator==(const year_month_weekday_last& x, const year_month_weekday_last& y) noexcept;
```

Returns: `x.year() == y.year() && x.month() == y.month() && x.weekday_last() == y.weekday_last()`.

```
constexpr bool operator!=(const year_month_weekday_last& x, const year_month_weekday_last& y) noexcept;
```

Returns: `!(x == y)`.

```
constexpr year_month_weekday_last operator+(const year_month_weekday_last& ymwdl, const months& dm) noexcept;
```

Returns: `(ymwdl.year() / ymwdl.month() + dm) / ymwdl.weekday_last()`.

```
constexpr year_month_weekday_last operator+(const months& dm, const year_month_weekday_last& ymwdl) noexcept;
```

Returns: `ymwdl + dm`.

```
constexpr year_month_weekday_last operator-(const year_month_weekday_last& ymwdl, const months& dm) noexcept;
```

Returns: `ymwdl + (-dm)`.

```
constexpr year_month_weekday_last operator+(const year_month_weekday_last& ymwdl, const years& dy) noexcept;
```

Returns: `{ymwdl.year()+dy, ymwdl.month(), ymwdl.weekday_last()}`.

```
constexpr year_month_weekday_last operator+(const years& dy, const year_month_weekday_last& ymwdl) noexcept;
```

Returns: `ymwdl + dy`.

```
constexpr year_month_weekday_last operator-(const year_month_weekday_last& ymwdl, const years& dy) noexcept;
```

Returns: `ymwdl + (-dy)`.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const year_month_weekday_last& ymwdl);
```

Returns: `os << ymwdl.year() << '/' << ymwdl.month() << '/' << ymwdl.weekday_last()`.

23.17.10.17 civil calendar conventional syntax operators [time.calendar.operators]

A set of overloaded `operator/()` provide a conventional syntax for the creation of civil calendar dates. The year, month and day ordering are accepted in any of the following 3 orders:

1. `y/m/d`
2. `m/d/y`
3. `d/m/y`

Anywhere a "day" is required one can also specify one of:

- `last`
- `weekday[i]`
- `weekday[last]`

Partial-date-types such as `year_month` and `month_day` can be created by simply not applying the second division operator for any of the three orders. For example:

```
year_month ym = 2015y/April;
month_day md1 = April/4;
month_day md2 = 4d/April;
```

Everything not intended as above is ill-formed, with the notable exception of an expression that consists of nothing but `int`, which has type `int`.

```
auto a = 2015/4/4;         // a == int(125)
auto b = 2015y/4/4;        // b == year_month_day{year(2015), month(4), day(4)}
auto c = 2015y/4d/April;   // error: invalid operands to binary expression ('chrono::year' and 'chrono::day')
auto d = 2015/April/4;     // error: invalid operands to binary expression ('int' and 'const chrono::month')
```

```
constexpr year_month operator/(const year& y, const month& m) noexcept;
```

Returns: `{y, m}`.

```
constexpr year_month operator/(const year& y, int   m) noexcept;
```

Returns: `y / month(m)`.

```
constexpr month_day operator/(const month& m, const day& d) noexcept;
```

Returns: `{m, d}`.

```
constexpr month_day operator/(const month& m, int d) noexcept;
```

Returns: `m / day(d)`.

```
constexpr month_day operator/(int m, const day& d) noexcept;
```

Returns: `month(m) / d`.

```
constexpr month_day operator/(const day& d, const month& m) noexcept;
```

Returns: `m / d`.

```
constexpr month_day operator/(const day& d, int m) noexcept;
```

Returns: `month(m) / d`.

```
constexpr month_day_last operator/(const month& m, last_spec) noexcept;
```

Returns: `month_day_last{m}`.

```
constexpr month_day_last operator/(int m, last_spec) noexcept;
```

Returns: `month(m) / last`.

```
constexpr month_day_last operator/(last_spec, const month& m) noexcept;
```

Returns: `m / last`.

```
constexpr month_day_last operator/(last_spec, int m) noexcept;
```

Returns: `month(m) / last`.

```
constexpr month_weekday operator/(const month& m, const weekday_indexed& wdi) noexcept;
```

Returns: `{m, wdi}`.

```
constexpr month_weekday operator/(int m, const weekday_indexed& wdi) noexcept;
```

Returns: `month(m) / wdi`.

```
constexpr month_weekday operator/(const weekday_indexed& wdi, const month& m) noexcept;
```

Returns: `m / wdi`.

```
constexpr month_weekday operator/(const weekday_indexed& wdi, int m) noexcept;
```

Returns: `month(m) / wdi`.

```
constexpr month_weekday_last operator/(const month& m, const weekday_last& wdl) noexcept;
```

Returns: `{m, wdl}`.

```
constexpr month_weekday_last operator/(int m, const weekday_last& wdl) noexcept;
```

Returns: `month(m) / wdl`.

```
constexpr month_weekday_last operator/(const weekday_last& wdl, const month& m) noexcept;
```

Returns: `m / wdl`.

```
constexpr month_weekday_last operator/(const weekday_last& wdl, int m) noexcept;
```

Returns: `month(m) / wdl`.

```
constexpr year_month_day operator/(const year_month& ym, const day& d) noexcept;
```

Returns: `{ym.year(), ym.month(), d}`.

```
constexpr year_month_day operator/(const year_month& ym, int d) noexcept;
```

Returns: `ym / day(d)`.

```
constexpr year_month_day operator/(const year& y, const month_day& md) noexcept;
```

Returns: `y / md.month() / md.day()`.

```
constexpr year_month_day operator/(int y, const month_day& md) noexcept;
```

Returns: `year(y) / md`.

```
constexpr year_month_day operator/(const month_day& md, const year& y) noexcept;
```

Returns: `y / md`.

```
constexpr year_month_day operator/(const month_day& md, int y) noexcept;
```

Returns: `year(y) / md`.

```
constexpr year_month_day_last operator/(const year_month& ym, last_spec) noexcept;
```

Returns: `{ym.year(), month_day_last{ym.month()}}`.

```
constexpr year_month_day_last operator/(const year& y, const month_day_last& mdl) noexcept;
```

Returns: `{y, mdl}`.

```
constexpr year_month_day_last operator/(int y, const month_day_last& mdl) noexcept;
```

Returns: `year(y) / mdl`.

```
constexpr year_month_day_last operator/(const month_day_last& mdl, const year& y) noexcept;
```

Returns: `y / mdl`.

```
constexpr year_month_day_last operator/(const month_day_last& mdl, int y) noexcept;
```

Returns: `year(y) / mdl`.

```
constexpr year_month_weekday operator/(const year_month& ym, const weekday_indexed& wdi) noexcept;
```

Returns: `{ym.year(), ym.month(), wdi}`.

```
constexpr year_month_weekday operator/(const year& y, const month_weekday& mwd) noexcept;
```

Returns: `{y, mwd.month(), mwd.weekday_indexed()}`.

```
constexpr year_month_weekday operator/(int y, const month_weekday& mwd) noexcept;
```

Returns: `year(y) / mwd`.

```
constexpr year_month_weekday operator/(const month_weekday& mwd, const year& y) noexcept;
```

Returns: `y / mwd`.

```
constexpr year_month_weekday operator/(const month_weekday& mwd, int y) noexcept;
```

Returns: `year(y) / mwd`.

```
constexpr year_month_weekday_last operator/(const year_month& ym, const weekday_last& wdl) noexcept;
```

Returns: `{ym.year(), ym.month(), wdl}`.

```
constexpr year_month_weekday_last operator/(const year& y, const month_weekday_last& mwdl) noexcept;
```

Returns: `{y, mwdl.month(), mwdl.weekday_last()}`.

```
constexpr year_month_weekday_last operator/(int y, const month_weekday_last& mwdl) noexcept;
```

Returns: `year(y) / mwdl`.

```
constexpr year_month_weekday_last operator/(const month_weekday_last& mwdl, const year& y) noexcept;
```

Returns: `y / mwdl`.

```
constexpr year_month_weekday_last operator/(const month_weekday_last& mwdl, int y) noexcept;
```

Returns: `year(y) / mwdl`.

Back to TOC

Add new section [time.time_of_day] after 23.17.10 The civil calendar [time.calendar]:

23.17.11 Class `time_of_day` [time.time_of_day]

The `time_of_day` class breaks a `duration` which represents the time elapsed since midnight, into a "broken" down time such as hours:minutes:seconds. The `Duration` template parameter dictates the precision to which the time is broken down. This can vary from a coarse precision of hours to a very fine precision of nanoseconds. `time_of_day` is primarily a formatting tool.

```
template <class Duration> class time_of_day;
```

There are four specializations of `time_of_day` to handle four precisions:

1. `template <> class time_of_day<hours>`

   This specialization handles hours since midnight.

2. `template <> class time_of_day<minutes>`

   This specialization handles hours:minutes since midnight.

3. `template <> class time_of_day<seconds>`

   This specialization handles hours:minutes:seconds since midnight.

4. `template <class Rep, class Period> class time_of_day<duration<Rep, Period>>`

   This specialization is restricted to `Rep` types that are integral, and `Period`s that are not an integral number of seconds. Typical uses are with milliseconds, microseconds and nanoseconds. This specialization handles hours:minute:seconds.fractional_seconds since midnight.

Each specialization of `time_of_day` is a trivially copyable class type.
Each specialization of `time_of_day` is a standard-layout class type.

```
template <>
class time_of_day<hours>
{
public:
    using precision = chrono::hours;

    time_of_day() = default;
    explicit constexpr time_of_day(chrono::hours since_midnight) noexcept;

    constexpr chrono::hours hours() const noexcept;

    explicit constexpr operator  precision()   const noexcept;
    constexpr          precision to_duration() const noexcept;

    constexpr void make24() noexcept;
    constexpr void make12() noexcept;
};
```

```
explicit constexpr time_of_day<hours>::time_of_day(chrono::hours since_midnight) noexcept;
```

Effects: Constructs an object of type `time_of_day` in 24-hour format corresponding to `since_midnight` hours after 00:00:00.

Postconditions: `hours()` returns the integral number of hours `since_midnight` is after 00:00:00.

```
constexpr hours time_of_day<hours>::hours() const noexcept;
```

Returns: The stored hour of `*this`.

```
explicit constexpr time_of_day<hours>::operator precision() const noexcept;
```

Returns: The number of hours since midnight.

```
constexpr precision to_duration() const noexcept;
```

Returns: `precision{*this}`.

```
constexpr void time_of_day<hours>::make24() noexcept;
```

Effects: If `*this` is a 12-hour time, converts to a 24-hour time. Otherwise, no effects.

```
constexpr void time_of_day<hours>::make12() noexcept;
```

Effects: If `*this` is a 24-hour time, converts to a 12-hour time. Otherwise, no effects.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const time_of_day<hours>& t);
```

Effects: If `t` is a 24-hour time, outputs to `os` according to the format: "%H00" ([time.format]). Otherwise outputs to `os` according to the format: "%I%p" ([time.format]).

Returns: `os`.

[*Example:*

```
0100  // 1 in the morning in 24-hour format
1800  // 6 in the evening in 24-hour format
1am   // 1 in the morning in 12-hour format
6pm   // 6 in the evening in 12-hour format
```

— end example]

```
template <>
class time_of_day<minutes>
{
public:
    using precision = chrono::minutes;

    time_of_day() = default;
    explicit constexpr time_of_day(chrono::minutes since_midnight) noexcept;

    constexpr chrono::hours    hours()   const noexcept;
    constexpr chrono::minutes  minutes() const noexcept;

    explicit constexpr operator precision()    const noexcept;
    constexpr          precision to_duration() const noexcept;

    constexpr void make24() noexcept;
    constexpr void make12() noexcept;
};
```

```
explicit constexpr time_of_day<minutes>::time_of_day(minutes since_midnight) noexcept;
```

Effects: Constructs an object of type `time_of_day` in 24-hour format corresponding to `since_midnight` minutes after 00:00:00.

Postconditions: `hours()` returns the integral number of hours `since_midnight` is after 00:00:00. `minutes()` returns the integral number of minutes `since_midnight` is after (00:00:00 + `hours()`).

```
constexpr hours time_of_day<minutes>::hours() const noexcept;
```

Returns: The stored hour of `*this`.

```
constexpr minutes time_of_day<minutes>::minutes() const noexcept;
```

Returns: The stored minute of `*this`.

```
explicit constexpr time_of_day<minutes>::operator precision() const noexcept;
```

Returns: The number of minutes since midnight.

```
constexpr precision to_duration() const noexcept;
```

Returns: `precision{*this}`.

```
constexpr void time_of_day<minutes>::make24() noexcept;
```

Effects: If `*this` is a 12-hour time, converts to a 24-hour time. Otherwise, no effects.

```
constexpr void time_of_day<minutes>::make12() noexcept;
```

Effects: If `*this` is a 24-hour time, converts to a 12-hour time. Otherwise, no effects.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const time_of_day<minutes>& t);
```

Effects: If `t` is a 24-hour time, outputs to `os` according to the format: "%H:%M" ([time.format]). Otherwise outputs to `os` according to the format: "%I:%M%p" ([time.format]).

Returns: `os`.

[*Example:*

```
01:08   // 1:08 in the morning in 24-hour format
18:15   // 6:15 in the evening in 24-hour format
1:08am  // 1:08 in the morning in 12-hour format
6:15pm  // 6:15 in the evening in 12-hour format
```

— *end example*]

```
template <>
class time_of_day<seconds>
{
public:
    using precision = chrono::seconds;

    time_of_day() = default;
    explicit constexpr time_of_day(chrono::seconds since_midnight) noexcept;

    constexpr chrono::hours    hours()   const noexcept;
    constexpr chrono::minutes  minutes() const noexcept;
    constexpr chrono::seconds  seconds() const noexcept;

    explicit constexpr operator  precision()   const noexcept;
    constexpr          precision to_duration() const noexcept;
```

```
    constexpr void make24() noexcept;
    constexpr void make12() noexcept;
};
```

```
explicit constexpr time_of_day<seconds>::time_of_day(seconds since_midnight) noexcept;
```

Effects: Constructs an object of type `time_of_day` in 24-hour format corresponding to `since_midnight` seconds after 00:00:00.

Postconditions: `hours()` returns the integral number of hours `since_midnight` is after 00:00:00. `minutes()` returns the integral number of minutes `since_midnight` is after (00:00:00 + `hours()`). `seconds()` returns the integral number of seconds `since_midnight` is after (00:00:00 + `hours()` + `minutes()`).

```
constexpr hours time_of_day<seconds>::hours() const noexcept;
```

Returns: The stored hour of `*this`.

```
constexpr minutes time_of_day<seconds>::minutes() const noexcept;
```

Returns: The stored minute of `*this`.

```
constexpr seconds time_of_day<seconds>::seconds() const noexcept;
```

Returns: The stored second of `*this`.

```
explicit constexpr time_of_day<seconds>::operator precision() const noexcept;
```

Returns: The number of seconds since midnight.

```
constexpr precision to_duration() const noexcept;
```

Returns: `precision{*this}`.

```
constexpr void time_of_day<seconds>::make24() noexcept;
```

Effects: If `*this` is a 12-hour time, converts to a 24-hour time. Otherwise, no effects.

```
constexpr void time_of_day<seconds>::make12() noexcept;
```

Effects: If `*this` is a 24-hour time, converts to a 12-hour time. Otherwise, no effects.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const time_of_day<seconds>& t);
```

Effects: If `t` is a 24-hour time, outputs to `os` according to the format: "%T" ([time.format]). Otherwise outputs to `os` according to the format: "%I:%M:%S%p" ([time.format]).

Returns: `os`.

[*Example:*

```
01:08:03   // 1:08:03 in the morning in 24-hour format
18:15:45   // 6:15:45 in the evening in 24-hour format
1:08:03am  // 1:08:03 in the morning in 12-hour format
6:15:45pm  // 6:15:45 in the evening in 12-hour format
```

— *end example*]

```
template <class Rep, class Period>
class time_of_day<duration<Rep, Period>>
{
public:
    using precision = duration<Rep, Period>;

    time_of_day() = default;
    explicit constexpr time_of_day(precision since_midnight) noexcept;

    constexpr chrono::hours     hours()      const noexcept;
    constexpr chrono::minutes   minutes()    const noexcept;
    constexpr chrono::seconds   seconds()    const noexcept;
    constexpr precision subseconds() const noexcept;

    explicit constexpr operator  precision()   const noexcept;
    constexpr          precision to_duration() const noexcept;

    constexpr void make24() noexcept;
    constexpr void make12() noexcept;
};
```

This specialization shall not exist unless `treat_as_floating_point_v<Rep>` is `false` and `duration<Rep, Period>` is not convertible to `seconds`.

```
explicit constexpr time_of_day<duration<Rep, Period>>::time_of_day(precision since_midnight) noexcept;
```

Effects: Constructs an object of type `time_of_day` in 24-hour format corresponding to `since_midnight precision` fractional seconds after 00:00:00.

Postconditions: `hours()` returns the integral number of hours `since_midnight` is after 00:00:00. `minutes()` returns the integral number of minutes `since_midnight` is after (00:00:00 + `hours()`). `seconds()` returns the integral number of seconds `since_midnight` is after (00:00:00 + `hours()` + `minutes()`). `subseconds()` returns the integral number of fractional precision seconds `since_midnight` is after (00:00:00 + `hours()` + `minutes()` + `seconds()`).

```
constexpr hours time_of_day<duration<Rep, Period>>::hours() const noexcept;
```

Returns: The stored hour of `*this`.

```
constexpr minutes time_of_day<duration<Rep, Period>>::minutes() const noexcept;
```

Returns: The stored minute of `*this`.

```
constexpr seconds time_of_day<duration<Rep, Period>>::seconds() const noexcept;
```

Returns: The stored second of `*this`.

```
constexpr
duration<Rep, Period>
time_of_day<duration<Rep, Period>>::subseconds() const noexcept;
```

Returns: The stored subsecond of `*this`.

```
explicit constexpr time_of_day<duration<Rep, Period>>::operator precision() const noexcept;
```

Returns: The number of subseconds since midnight.

```
constexpr precision to_duration() const noexcept;
```

Returns: `precision{*this}`.

```
constexpr void time_of_day<duration<Rep, Period>>::make24() noexcept;
```

Effects: If `*this` is a 12-hour time, converts to a 24-hour time. Otherwise, no effects.

```
constexpr void time_of_day<duration<Rep, Period>>::make12() noexcept;
```

Effects: If `*this` is a 24-hour time, converts to a 12-hour time. Otherwise, no effects.

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const time_of_day<duration<Rep, Period>>& t);
```

Effects: If `t` is a 24-hour time, outputs to `os` according to the format: "%T" ([time.format]). Otherwise outputs to `os` according to the format: "%I:%M:%S%p" ([time.format]).

Returns: `os`.

[*Example:*

```
01:08:03.007   // 1:08:03.007 in the morning in 24-hour format (assuming millisecond precision)
18:15:45.123   // 6:15:45.123 in the evening in 24-hour format (assuming millisecond precision)
1:08:03.007am  // 1:08:03.007 in the morning in 12-hour format (assuming millisecond precision)
6:15:45.123pm  // 6:15:45.123 in the evening in 12-hour format (assuming millisecond precision)
```

— *end example*]

Back to TOC

Add new section [time.timezone] after 23.17.11 Class `time_of_day` [time.time_of_day]:

23.17.12 Time Zones [time.timezone]

This section creates an API which exposes the IANA Time Zone database (Footnote: RFC 6557 https://tools.ietf.org/html/rfc6557), and interfaces with `sys_time` and `local_time`. By using only this interface, time zone support is provided not only to the civil calendar types, but also to other user-written calendars that interface with `sys_time` and `local_time`.

23.17.12.1 The time zone database [time.timezone.database]

The following data structure contains the time zone database, and the following functions access it.

```
struct tzdb
{
    string            version;
    vector<time_zone> zones;
    vector<link>      links;
```

```
    vector<leap>      leaps;

    const time_zone* locate_zone(string_view tz_name) const;
    const time_zone* current_zone() const;

    tzdb* next;  // exposition only
};

class tzdb_list
{
    atomic<tzdb*> head_{nullptr};  // exposition only

public:
    tzdb_list(const tzdb_list&) = delete;
    tzdb_list& operator=(const tzdb_list&) = delete;

    class const_iterator;

    const tzdb& front() const noexcept;

    const_iterator erase_after(const_iterator p);

    const_iterator begin() const noexcept;
    const_iterator end()   const noexcept;

    const_iterator cbegin() const noexcept;
    const_iterator cend()   const noexcept;
};
```

The `tzdb_list` database is a singleton. Access is granted to it via the `get_tzdb_list()` function which returns a reference to it. However this access is only needed for those applications which need to have long uptimes and have a need to update the time zone database while running. Other applications can implicitly access the `front()` of this list via the *read-only* namespace scope functions `get_tzdb()`, `locate_zone()` and `current_zone()`. Each `vector` in `tzdb` is sorted to enable fast lookup. One can iterate over and inspect this database. And multiple versions of the database can be used at once, via the `tzdb_list`.

```
const time_zone* tzdb::locate_zone(string_view tz_name) const;
```

> *Returns:* If a `time_zone` is found for which `name() == tz_name`, returns a pointer to that `time_zone`. Otherwise if a `link` is found where `tz_name == link.name()`, then a pointer is returned to the `time_zone` for which `zone.name() == link.target()` [*Note:* A `link` is an alternative name for a `time_zone`. — *end note*]
>
> *Throws:* If a `const time_zone*` can not be found as described in the *Returns* clause, throws a `runtime_error`. [*Note:* On non-exceptional return, the return value is *always* a pointer to a valid `time_zone`. — *end note*]

```
const time_zone* tzdb::current_zone() const;
```

> *Returns:* A `const time_zone*` referring to the time zone which the computer has set as its local time zone.

```
tzdb_list& get_tzdb_list();
```

> *Effects:* If this is the first access to the database, will initialize the database. If this call initializes the database, the resulting database will be a `tzdb_list` which holds a single initialized `tzdb`.
>
> *Returns:* A reference to the database.
>
> *Remarks:* It is safe to call this function from multiple threads at one time.
>
> *Throws:* `runtime_error` if for any reason a reference can not be returned to a valid `tzdb_list&` containing one or more valid `tzdb`.

```
const tzdb& get_tzdb();
```

> *Returns:* `get_tzdb_list().front()`.

```
const time_zone* locate_zone(string_view tz_name);
```

> *Returns:* `get_tzdb().locate_zone(tz_name)` which will initialize the timezone database if this is the first reference to the database.

```
const time_zone* current_zone();
```

> *Returns:* `get_tzdb().current_zone()`.

`tzdb_list::const_iterator` is a constant iterator which meets the forward iterator requirements and has a value type of `tzdb`.

```
const tzdb& tzdb_list::front() const noexcept;
```

> *Returns:* `*head_`.
>
> *Remarks:* This operation is thread-safe with respect to `reload_tzdb()`. [*Note:* `reload_tzdb()` pushes a new `tzdb` onto the front of this container. — *end note*]

```
tzdb_list::const_iterator tzdb_list::erase_after(const_iterator p);
```

Requires: The iterator following `p` is dereferenceable.

Effects: Erases the `tzdb` referred to by the iterator following `p`.

Returns: An iterator pointing to the element following the one that was erased, or `end()` if no such element exists.

Remarks: No pointers, references, or iterators are invalidated except those referring to the erased `tzdb`. [*Note:* It is not possible to erase the `tzdb` referred to by `begin()`. — *end note*]

Throws: Nothing.

```
tzdb_list::const_iterator tzdb_list::begin() const noexcept;
```

Returns: An iterator referring to the first `tzdb` in the container.

```
tzdb_list::const_iterator tzdb_list::end() const noexcept;
```

Returns: An iterator referring to the position one past the last `tzdb` in the container.

```
tzdb_list::const_iterator tzdb_list::cbegin() const noexcept;
```

Returns: `begin()`.

```
tzdb_list::const_iterator tzdb_list::cend() const noexcept;
```

Returns: `end()`.

Back to TOC

23.17.12.1.1 Remote time zone database support [time.timezone.database.remote]

The *local* time zone database is that supplied by the implementation when the application first accesses the database, for example via `current_zone()`. While the application is running, the implementation may choose to update the time zone database. This update shall not impact the application in any way unless the application calls the functions in this section. This potentially updated time zone database is referred to as the *remote* time zone database.

```
const tzdb& reload_tzdb();
```

Effects: This function first checks the version of the remote time zone database. If the version of the local and remote databases are the same, there are no effects. Otherwise the remote database is pushed to the front of the `tzdb_list` accessed by `get_tzdb_list()`.

Returns: `get_tzdb_list().front()`.

Remarks: No pointers, references, or iterators are invalidated.

Remarks: This function is thread-safe with respect to `get_tzdb_list().front()` and `get_tzdb_list().erase_after()`.

Throws: `runtime_error` if for any reason a reference can not be returned to a valid `tzdb`.

```
string remote_version();
```

Returns: The latest remote database version.

[*Note:* This can be compared with `get_tzdb().version` to discover if the local and remote databases are equivalent. — *end note*]

23.17.12.2 Exception classes [time.timezone.exception]

`nonexistent_local_time` is thrown when one attempts to convert a non-existent `local_time` to a `sys_time` without specifying `choose::earliest` or `choose::latest`.

```
class nonexistent_local_time
    : public runtime_error
{
public:
    template <class Duration>
        nonexistent_local_time(const local_time<Duration>& tp, const local_info& i);
};
```

```
template <class Duration>
nonexistent_local_time::nonexistent_local_time(const local_time<Duration>& tp,
                                               const local_info& i);
```

Requires: `i.result == local_info::nonexistent`.

Effects: Constructs a `nonexistent_local_time` by initializing the base class with a sequence of `char` equivalent to that produced by `os.str()` initialized as shown below:

```
ostringstream os;
os << tp << " is in a gap between\n"
   << local_seconds{i.first.end.time_since_epoch()} + i.first.offset << ' '
   << i.first.abbrev << " and\n"
   << local_seconds{i.second.begin.time_since_epoch()} + i.second.offset << ' '
   << i.second.abbrev
   << " which are both equivalent to\n"
   << i.first.end << " UTC";
```

[*Example:*

```
#include <chrono>
#include <iostream>

int
main()
{
    using namespace std::chrono;
    try
    {
        auto zt = zoned_time{"America/New_York", local_days{Sunday[2]/March/2016} + 2h + 30min};
    }
    catch (const nonexistent_local_time& e)
    {
        std::cout << e.what() << '\n';
    }
}
```

Which outputs:

```
2016-03-13 02:30:00 is in a gap between
2016-03-13 02:00:00 EST and
2016-03-13 03:00:00 EDT which are both equivalent to
2016-03-13 07:00:00 UTC
```

— *end example*]

`ambiguous_local_time` is thrown when one attempts to convert an ambiguous `local_time` to a `sys_time` without specifying `choose::earliest` or `choose::latest`.

```
class ambiguous_local_time
    : public runtime_error
{
public:
    template <class Duration>
        ambiguous_local_time(const local_time<Duration>& tp, const local_info& i);
};
```

```
template <class Duration>
ambiguous_local_time::ambiguous_local_time(const local_time<Duration>& tp,
                                           const local_info& i);
```

Requires: `i.result == local_info::ambiguous`.

Effects: Constructs an `ambiguous_local_time` by initializing the base class with a sequence of `char` equivalent to that produced by `os.str()` initialized as shown below:

```
ostringstream os;
os << tp << " is ambiguous.  It could be\n"
   << tp << ' ' << i.first.abbrev << " == "
   << tp - i.first.offset << " UTC or\n"
   << tp << ' ' << i.second.abbrev  << " == "
   << tp - i.second.offset  << " UTC";
```

[*Example:*

```
#include <chrono>
#include <iostream>

int
main()
{
    using namespace std::chrono;
    try
    {
        auto zt = zoned_time{"America/New_York", local_days{Sunday[1]/November/2016} + 1h + 30min};
    }
    catch (const ambiguous_local_time& e)
    {
        std::cout << e.what() << '\n';
    }
}
```

Which outputs:

```
    2016-11-06 01:30:00 is ambiguous.  It could be
    2016-11-06 01:30:00 EDT == 2016-11-06 05:30:00 UTC or
    2016-11-06 01:30:00 EST == 2016-11-06 06:30:00 UTC
```

— end example]

23.17.12.3 Information classes [time.timezone.info]

A `sys_info` structure can be obtained from the combination of a `time_zone` and either a `sys_time`, or `local_time`. It can also be obtained from a `zoned_time` which is effectively a `pair` of a `time_zone` and `sys_time`.

This type represents a lower-level API. Typical conversions from `sys_time` to `local_time` will use this structure *implicitly*, not *explicitly*.

```
struct sys_info
{
    sys_seconds   begin;
    sys_seconds   end;
    seconds       offset;
    minutes       save;
    string        abbrev;
};
```

The `begin` and `end` data members indicate that for the associated `time_zone` and `time_point`, the `offset` and `abbrev` are in effect in the range `[begin, end)`. This information can be used to efficiently iterate the transitions of a `time_zone`.

The `offset` data member indicates the UTC offset in effect for the associated `time_zone` and `time_point`. The relationship between `local_time` and `sys_time` is:

```
offset = local_time - sys_time
```

The `save` data member is "extra" information not normally needed for conversion between `local_time` and `sys_time`. If `save != 0min`, this `sys_info` is said to be on "daylight saving" time, and `offset - save` suggests what this `time_zone` *might* use if it were off daylight saving. However this information should not be taken as authoritative. The only sure way to get such information is to query the `time_zone` with a `time_point` that returns an `sys_info` where `save == 0min`. There is no guarantee what `time_point` might return such an `sys_info` except that it is guaranteed *not* to be in the range `[begin, end)` (if `save != 0min` for this `sys_info`).

The `abbrev` data member indicates the current abbreviation used for the associated `time_zone` and `time_point`. Abbreviations are not unique among the `time_zone`s, and so one can not reliably map abbreviations back to a `time_zone` and UTC offset.

A `sys_info` can be streamed out in an unspecified format:

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const sys_info& r);
```

A `local_info` structure represents a lower-level API. Typical conversions from `local_time` to `sys_time` will use this structure *implicitly*, not *explicitly*.

```
struct local_info
{
    static constexpr int unique      = 0;
    static constexpr int nonexistent = 1;
    static constexpr int ambiguous   = 2;

    int result;
    sys_info first;
    sys_info second;
};
```

When a `local_time` to `sys_time` conversion is unique, `result == unique`, `first` will be filled out with the correct `sys_info` and `second` will be zero-initialized. If the conversion stems from a nonexistent `local_time` then `result == nonexistent`, `first` will be filled out with the `sys_info` that ends just prior to the `local_time` and `second` will be filled out with the `sys_info` that begins just after the `local_time`. If the conversion stems from an ambiguous `local_time` then `result == ambiguous`, `first` will be filled out with the `sys_info` that ends just after the `local_time` and `second` will be filled out with the `sys_info` that starts just before the `local_time`.

A `local_info` can be streamed out in an unspecified format:

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const local_info& r);
```

23.17.12.4 Class `time_zone` [time.timezone.time_zone]

A `time_zone` represents all time zone transitions for a specific geographic area. `time_zone` construction is unspecified, and done during the database initialization. One can gain `const` access to a `time_zone` via functions such as `locate_zone`.

```
class time_zone
```

```
{
public:
    time_zone(time_zone&&) = default;
    time_zone& operator=(time_zone&&) = default;

    string_view name() const noexcept;

    template <class Duration> sys_info   get_info(const sys_time<Duration>& st)   const;
    template <class Duration> local_info get_info(const local_time<Duration>& tp) const;

    template <class Duration>
        sys_time<common_type_t<Duration, seconds>>
        to_sys(const local_time<Duration>& tp) const;

    template <class Duration>
        sys_time<common_type_t<Duration, seconds>>
        to_sys(const local_time<Duration>& tp, choose z) const;

    template <class Duration>
        local_time<common_type_t<Duration, seconds>>
        to_local(const sys_time<Duration>& tp) const;
};

bool operator==(const time_zone& x, const time_zone& y) noexcept;
bool operator!=(const time_zone& x, const time_zone& y) noexcept;
bool operator< (const time_zone& x, const time_zone& y) noexcept;
bool operator> (const time_zone& x, const time_zone& y) noexcept;
bool operator<=(const time_zone& x, const time_zone& y) noexcept;
bool operator>=(const time_zone& x, const time_zone& y) noexcept;
```

```
string_view time_zone::name() const noexcept;
```

Returns: The name of the `time_zone`.

[*Example:* "America/New_York". — *end example*]

> Here is an unofficial list of `time_zone` names: https://en.wikipedia.org/wiki/List_of_tz_database_time_zones.

```
template <class Duration> sys_info time_zone::get_info(const sys_time<Duration>& st) const;
```

Returns: A `sys_info i` for which `st` is in the range `[i.begin, i.end)`.

```
template <class Duration> local_info time_zone::get_info(const local_time<Duration>& tp) const;
```

Returns: A `local_info` for `tp`.

```
template <class Duration>
sys_time<common_type_t<Duration, seconds>>
time_zone::to_sys(const local_time<Duration>& tp) const;
```

Returns: A `sys_time` that is at least as fine as `seconds`, and will be finer if the argument `tp` has finer precision. This `sys_time` is the UTC equivalent of `tp` according to the rules of this `time_zone`.

Throws: If the conversion from `tp` to a `sys_time` is ambiguous, throws `ambiguous_local_time`. If the conversion from `tp` to a `sys_time` is nonexistent, throws `nonexistent_local_time`.

```
template <class Duration>
sys_time<common_type_t<Duration, seconds>>
time_zone::to_sys(const local_time<Duration>& tp, choose z) const;
```

Returns: A `sys_time` that is at least as fine as `seconds`, and will be finer if the argument `tp` has finer precision. This `sys_time` is the UTC equivalent of `tp` according to the rules of this `time_zone`. If the conversion from `tp` to a `sys_time` is ambiguous, returns the earlier `sys_time` if `z == choose::earliest`, and returns the later `sys_time` if `z == choose::latest`. If the `tp` represents a non-existent time between two UTC `time_point`s, then the two UTC `time_point`s will be the same, and that UTC `time_point` will be returned.

```
template <class Duration>
local_time<common_type_t<Duration, seconds>>
time_zone::to_local(const sys_time<Duration>& tp) const;
```

Returns: The `local_time` associated with `tp` and this `time_zone`.

```
bool operator==(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `x.name() == y.name()`.

```
bool operator!=(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `!(x == y)`.

```
bool operator<(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `x.name() < y.name()`.

```
bool operator>(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `y < x`.

```
bool operator<=(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `!(y < x)`.

```
bool operator>=(const time_zone& x, const time_zone& y) noexcept;
```

Returns: `!(x < y)`.

23.17.12.5 Class `zoned_traits` [time.timezone.zoned_traits]

`zoned_traits` provides a means for customizing the behavior of `zoned_time<Duration, TimeZonePtr>` for the `zoned_time` default constructor, and constructors taking `string_view`. A specialization for `const time_zone*` is provided by the implementation.

```
template <class T> struct zoned_traits {};

template <>
struct zoned_traits<const time_zone*>
{
    static const time_zone* default_zone();
    static const time_zone* locate_zone(string_view name);
};
```

```
static const time_zone* zoned_traits<const time_zone*>::default_zone();
```

Returns: `std::chrono::locate_zone("UTC")`.

```
static const time_zone* zoned_traits<const time_zone*>::locate_zone(string_view name);
```

Returns: `std::chrono::locate_zone(name)`.

23.17.12.6 Class `zoned_time` [time.timezone.zoned_time]

`zoned_time` represents a logical pairing of `time_zone` and a `time_point` with precision `Duration`.

```
template <class Duration, class TimeZonePtr = const time_zone*>
class zoned_time
{
public:
    using duration = common_type_t<Duration, seconds>;

private:
    TimeZonePtr        zone_;  // exposition only
    sys_time<duration> tp_;    // exposition only

    using traits = zoned_traits<TimeZonePtr>  // exposition only

public:
    zoned_time();
    zoned_time(const zoned_time&) = default;
    zoned_time& operator=(const zoned_time&) = default;

             zoned_time(const sys_time<Duration>& st);
    explicit zoned_time(TimeZonePtr z);
    explicit zoned_time(string_view name);

    template <class Duration2>
        zoned_time(const zoned_time<Duration2>& zt) noexcept;

    zoned_time(TimeZonePtr z,    const sys_time<Duration>& st);
    zoned_time(string_view name, const sys_time<Duration>& st);

    zoned_time(TimeZonePtr z,    const local_time<Duration>& tp);
    zoned_time(string_view name, const local_time<Duration>& tp);
    zoned_time(TimeZonePtr z,    const local_time<Duration>& tp, choose c);
    zoned_time(string_view name, const local_time<Duration>& tp, choose c);

    template <class Duration2, class TimeZonePtr2>
        zoned_time(TimeZonePtr z, const zoned_time<Duration2, TimeZonePtr2>& zt);
    template <class Duration2, class TimeZonePtr2>
        zoned_time(TimeZonePtr z, const zoned_time<Duration2, TimeZonePtr2>& zt, choose);

    zoned_time(string_view name, const zoned_time<Duration>& zt);
    zoned_time(string_view name, const zoned_time<Duration>& zt, choose);

    zoned_time& operator=(const sys_time<Duration>& st);
    zoned_time& operator=(const local_time<Duration>& ut);

             operator sys_time<duration>()   const;
    explicit operator local_time<duration>() const;
```

```
    TimeZonePtr          get_time_zone()  const;
    local_time<duration> get_local_time() const;
    sys_time<duration>   get_sys_time()   const;
    sys_info             get_info()       const;
};

template <class Duration1, class Duration2, class TimeZonePtr>
bool
operator==(const zoned_time<Duration1, TimeZonePtr>& x,
           const zoned_time<Duration2, TimeZonePtr>& y);

template <class Duration1, class Duration2, class TimeZonePtr>
bool
operator!=(const zoned_time<Duration1, TimeZonePtr>& x,
           const zoned_time<Duration2, TimeZonePtr>& y);

template <class charT, class traits, class Duration, class TimeZonePtr>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os,
               const zoned_time<Duration, TimeZonePtr>& t);

template <class charT, class traits, class Duration, class TimeZonePtr>
    basic_ostream<charT, traits>&
    to_stream(basic_ostream<charT, traits>& os, const charT* fmt,
              const zoned_time<Duration, TimeZonePtr>& tp);

zoned_time()
    -> zoned_time<seconds>;

template <class Duration>
    zoned_time(sys_time<Duration>)
        -> zoned_time<common_type_t<Duration, seconds>>;

template <class TimeZonePtr, class Duration>
    zoned_time(TimeZonePtr, sys_time<Duration>)
        -> zoned_time<common_type_t<Duration, seconds>, TimeZonePtr>;

template <class TimeZonePtr, class Duration>
    zoned_time(TimeZonePtr, local_time<Duration>, choose = choose::earliest)
        -> zoned_time<common_type_t<Duration, seconds>, TimeZonePtr>;

template <class TimeZonePtr, class Duration>
    zoned_time(TimeZonePtr, zoned_time<Duration>, choose = choose::earliest)
        -> zoned_time<common_type_t<Duration, seconds>, TimeZonePtr>;

zoned_time(string_view)
    -> zoned_time<seconds>;

template <class Duration>
zoned_time(string_view, sys_time<Duration>)
    -> zoned_time<common_type_t<Duration, seconds>>;

template <class Duration>
zoned_time(string_view, local_time<Duration>, choose = choose::earliest)
    -> zoned_time<common_type_t<Duration, seconds>>;

template <class Duration, class TimeZonePtr, class TimeZonePtr2>
zoned_time(TimeZonePtr, zoned_time<Duration, TimeZonePtr2>, choose = choose::earliest)
    -> zoned_time<Duration, TimeZonePtr>;
```

An invariant of `zoned_time<Duration>` is that it always refers to a time zone, and represents a point in time that exists and is not ambiguous.

If `Duration` is not an specialization of `duration`, the program is ill-formed.

```
zoned_time<Duration, TimeZonePtr>::zoned_time();
```

> *Remarks:* This constructor does not participate in overload resolution unless the expression `traits::default_zone()` is well-formed.
>
> *Effects:* Constructs a `zoned_time` by initializing `zone_` with `traits::default_zone()` and default constructing `tp_`.

```
zoned_time<Duration, TimeZonePtr>::zoned_time(const sys_time<Duration>& st);
```

> *Remarks:* This constructor does not participate in overload resolution unless the expression `traits::default_zone()` is well-formed.
>
> *Effects:* Constructs a `zoned_time` by initializing `zone_` with `traits::default_zone()` and `tp_` with `st`.

```
explicit zoned_time<Duration, TimeZonePtr>::zoned_time(TimeZonePtr z);
```

> *Requires:* `z` refers to a time zone.
>
> *Effects:* Constructs a `zoned_time` initializing `zone_` with `std::move(z)`.

```
explicit zoned_time<Duration, TimeZonePtr>::zoned_time(string_view name);
```

Remarks: This constructor does not participate in overload resolution unless the expression `traits::locate_zone(string_view{})` is well-formed and `zoned_time` is constructible from the return type of `traits::locate_zone(string_view{})`.

Effects: Constructs a `zoned_time` by initializing `zone_` with `traits::locate_zone(name)` and default constructing `tp_`.

```
template <class Duration2, TimeZonePtr2>
    zoned_time<Duration, TimeZonePtr>::zoned_time(const zoned_time<Duration2, TimeZonePtr>& y) noexcept;
```

Remarks: Does not participate in overload resolution unless `sys_time<Duration2>` is implicitly convertible to `sys_time<Duration>`.

Effects: Constructs a `zoned_time` by initializing `zone_` with `y.zone_` and `tp_` with `y.tp_`.

```
zoned_time<Duration, TimeZonePtr>::zoned_time(TimeZonePtr z, const sys_time<Duration>& st);
```

Requires: `z` refers to a time zone.

Effects: Constructs a `zoned_time` by initializing `zone_` with `std::move(z)` and `tp_` with `st`.

```
zoned_time<Duration, TimeZonePtr>::zoned_time(string_view name, const sys_time<Duration>& st);
```

Remarks: This constructor does not participate in overload resolution unless `zoned_time` is constructible from the return type of `traits::locate_zone(name)` and `st`.

Effects: Equivalent to construction with `{traits::locate_zone(name), st}`.

```
zoned_time<Duration, TimeZonePtr>::zoned_time(TimeZonePtr z, const local_time<Duration>& tp);
```

Requires: `z` refers to a time zone.

Remarks: This constructor does not participate in overload resolution unless `declval<TimeZonePtr&>()->to_sys(local_time<Duration>{})` is convertible to `sys_time<duration>`.

Effects: Constructs a `zoned_time` by initializing `zone_` with `std::move(z)` and `tp_` with `zone_->to_sys(tp)`.

```
zoned_time<Duration, TimeZonePtr>::zoned_time(string_view name, const local_time<Duration>& tp);
```

Remarks: This constructor does not participate in overload resolution unless `zoned_time` is constructible from the return type of `traits::locate_zone(name)` and `tp`.

Effects: Equivalent to construction with `{traits::locate_zone(name), tp}`.

```
zoned_time<Duration, TimeZonePtr>::zoned_time(TimeZonePtr z, const local_time<Duration>& tp, choose c);
```

Requires: `z` refers to a time zone.

Remarks: This constructor does not participate in overload resolution unless `decltype(declval<TimeZonePtr&>()->to_sys(local_time<Duration>{}, choose::earliest))` is convertible to `sys_time<duration>`.

Effects: Constructs a `zoned_time` by initializing `zone_` with `std::move(z)` and `tp_` with `zone_->to_sys(tp, c)`.

```
zoned_time<Duration, TimeZonePtr>::zoned_time(string_view name, const local_time<Duration>& tp, choose c);
```

Remarks: This constructor does not participate in overload resolution unless `zoned_time` is constructible from the return type of `traits::locate_zone(name)`, `local_time<Duration>` and `choose`.

Effects: Equivalent to construction with `{traits::locate_zone(name), tp, c}`.

```
template <class Duration2, TimeZonePtr2>
    zoned_time<Duration, TimeZonePtr>::zoned_time(TimeZonePtr z, const zoned_time<Duration2, TimeZonePtr2>& y);
```

Remarks: Does not participate in overload resolution unless `sys_time<Duration2>` is implicitly convertible to `sys_time<Duration>`.

Requires: `z` refers to a valid time zone.

Effects: Constructs a `zoned_time` by initializing `zone_` with `std::move(z)` and `tp_` with `y.tp_`.

```
template <class Duration2, TimeZonePtr2>
    zoned_time<Duration, TimeZonePtr>::zoned_time(TimeZonePtr z, const zoned_time<Duration2, TimeZonePtr2>& y,
                                                  choose);
```

Remarks: Does not participate in overload resolution unless `sys_time<Duration2>` is implicitly convertible to `sys_time<Duration>`.

Requires: `z` refers to a valid time zone.

Effects: Equivalent to construction with `{z, y}`.

[*Note:* The `choose` parameter is allowed here, but has no impact. — *end note*]

```
zoned_time<Duration, TimeZonePtr>::zoned_time(string_view name, const zoned_time<Duration>& y);
```

Remarks: This constructor does not participate in overload resolution unless `zoned_time` is constructible from the return type of `traits::locate_zone(name)` and `zoned_time`.

Effects: Equivalent to construction with `{traits::locate_zone(name), y}`.

```
zoned_time<Duration, TimeZonePtr>::zoned_time(string_view name, const zoned_time<Duration>& y, choose c);
```

Remarks: This constructor does not participate in overload resolution unless `zoned_time` is constructible from the return type of `traits::locate_zone(name)`, `zoned_time`, and `choose`.

Effects: Equivalent to construction with `{traits::locate_zone(name), y, c}`.

[*Note:* The `choose` parameter is allowed here, but has no impact. — *end note*]

```
zoned_time<Duration, TimeZonePtr>& zoned_time<Duration, TimeZonePtr>::operator=(const sys_time<Duration>& st);
```

Effects: After assignment `get_sys_time() == st`. This assignment has no effect on the return value of `get_time_zone()`.

Returns: `*this`.

```
zoned_time<Duration, TimeZonePtr>& zoned_time<Duration, TimeZonePtr>::operator=(const local_time<Duration>& lt);
```

Effects: After assignment `get_local_time() == lt`. This assignment has no effect on the return value of `get_time_zone()`.

Returns: `*this`.

```
zoned_time<Duration, TimeZonePtr>::operator sys_time<duration>() const;
```

Returns: `get_sys_time()`.

```
explicit zoned_time<Duration, TimeZonePtr>::operator local_time<duration>() const;
```

Returns: `get_local_time()`.

```
TimeZonePtr zoned_time<Duration, TimeZonePtr>::get_time_zone() const;
```

Returns: `zone_`.

```
local_time<typename zoned_time<Duration, TimeZonePtr>::duration> zoned_time<Duration, TimeZonePtr>::get_local_time() const;
```

Returns: `zone_->to_local(tp_)`.

```
sys_time<typename zoned_time<Duration, TimeZonePtr>::duration> zoned_time<Duration, TimeZonePtr>::get_sys_time() const;
```

Returns: `tp_`.

```
sys_info zoned_time<Duration, TimeZonePtr>::get_info() const;
```

Returns: `zone_->get_info(tp_)`.

```
template <class Duration1, class Duration2, class TimeZonePtr>
bool
operator==(const zoned_time<Duration1, TimeZonePtr>& x,
           const zoned_time<Duration2, TimeZonePtr>& y);
```

Returns: `x.zone_ == y.zone_ && x.tp_ == y.tp_`.

```
template <class Duration1, class Duration2, class TimeZonePtr>
bool
operator!=(const zoned_time<Duration1, TimeZonePtr>& x,
           const zoned_time<Duration2, TimeZonePtr>& y);
```

Returns: `!(x == y)`.

```
template <class charT, class traits, class Duration, class TimeZonePtr>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os,
           const zoned_time<Duration, TimeZonePtr>& t)
```

Effects: Streams `t` to `os` using the format "%F %T %Z" and the value returned from `t.get_local_time()`.

Returns: `os`.

```
template <class charT, class traits, class Duration, class TimeZonePtr>
basic_ostream<charT, traits>&
to_stream(basic_ostream<charT, traits>& os, const charT* fmt,
          const zoned_time<Duration, TimeZonePtr>& tp);
```

Effects: First obtains a `sys_info` via `tp.get_info()` which for exposition purposes will be referred to as `info`. Then calls `to_stream(os, fmt, tp.get_local_time(), &info.abbrev, &info.offset)`.

Returns: `os`.

Back to TOC

Add a new section 23.17.12.7 leap [time.timezone.leap]:

23.17.12.7 class `leap` [time.timezone.leap]

```
class leap
{
    sys_seconds date_;  // exposition only

public:
    leap(const leap&)            = default;
    leap& operator=(const leap&) = default;

    // Unspecified constructors

    constexpr sys_seconds date() const noexcept;
};

constexpr bool operator==(const leap& x, const leap& y) noexcept;
constexpr bool operator!=(const leap& x, const leap& y) noexcept;
constexpr bool operator< (const leap& x, const leap& y) noexcept;
constexpr bool operator> (const leap& x, const leap& y) noexcept;
constexpr bool operator<=(const leap& x, const leap& y) noexcept;
constexpr bool operator>=(const leap& x, const leap& y) noexcept;

template <class Duration> constexpr bool operator==(const leap&               x, const sys_time<Duration>& y) noexcept;
template <class Duration> constexpr bool operator==(const sys_time<Duration>& x, const leap&               y) noexcept;
template <class Duration> constexpr bool operator!=(const leap&               x, const sys_time<Duration>& y) noexcept;
template <class Duration> constexpr bool operator!=(const sys_time<Duration>& x, const leap&               y) noexcept;
template <class Duration> constexpr bool operator< (const leap&               x, const sys_time<Duration>& y) noexcept;
template <class Duration> constexpr bool operator< (const sys_time<Duration>& x, const leap&               y) noexcept;
template <class Duration> constexpr bool operator> (const leap&               x, const sys_time<Duration>& y) noexcept;
template <class Duration> constexpr bool operator> (const sys_time<Duration>& x, const leap&               y) noexcept;
template <class Duration> constexpr bool operator<=(const leap&               x, const sys_time<Duration>& y) noexcept;
template <class Duration> constexpr bool operator<=(const sys_time<Duration>& x, const leap&               y) noexcept;
template <class Duration> constexpr bool operator>=(const leap&               x, const sys_time<Duration>& y) noexcept;
template <class Duration> constexpr bool operator>=(const sys_time<Duration>& x, const leap&               y) noexcept;
```

`leap` is a copyable class that is constructed and stored in the time zone database when initialized. One can explicitly convert it to a `sys_seconds` with the member function `date()` and that will be the date of the leap second insertion. `leap` is equality and less-than comparable, both with itself, and with `sys_time<Duration>`.

[*Example:*

Here is the date of all of the leap second insertions at the time of this writing:

```
for (auto& l : get_tzdb().leaps)
    cout << l.date() << '\n';
```

which outputs:

```
1972-07-01 00:00:00
1973-01-01 00:00:00
1974-01-01 00:00:00
1975-01-01 00:00:00
1976-01-01 00:00:00
1977-01-01 00:00:00
1978-01-01 00:00:00
1979-01-01 00:00:00
1980-01-01 00:00:00
1981-07-01 00:00:00
1982-07-01 00:00:00
1983-07-01 00:00:00
1985-07-01 00:00:00
1988-01-01 00:00:00
1990-01-01 00:00:00
1991-01-01 00:00:00
1992-07-01 00:00:00
1993-07-01 00:00:00
1994-07-01 00:00:00
1996-01-01 00:00:00
1997-07-01 00:00:00
1999-01-01 00:00:00
2006-01-01 00:00:00
2009-01-01 00:00:00
2012-07-01 00:00:00
2015-07-01 00:00:00
```

```
2017-01-01 00:00:00
```

— *end example*]

```
constexpr sys_seconds leap::date() const noexcept
```

Returns: `date_`.

```
constexpr bool operator==(const leap& x, const leap& y) noexcept
```

Returns: `x.date() == y.date()`.

```
constexpr bool operator!=(const leap& x, const leap& y) noexcept
```

Returns: `!(x == y)`.

```
constexpr bool operator<(const leap& x, const leap& y) noexcept
```

Returns: `x.date() < y.date()`.

```
constexpr bool operator>(const leap& x, const leap& y) noexcept
```

Returns: `y < x`.

```
constexpr bool operator<=(const leap& x, const leap& y) noexcept
```

Returns: `!(y < x)`.

```
constexpr bool operator>=(const leap& x, const leap& y) noexcept
```

Returns: `!(x < y)`.

```
template <class Duration> constexpr bool operator==(const leap& x, const sys_time<Duration>& y) noexcept
```

Returns: `x.date() == y`.

```
template <class Duration> constexpr bool operator==(const sys_time<Duration>& x, const leap& y) noexcept
```

Returns: `y == x`.

```
template <class Duration> constexpr bool operator!=(const leap& x, const sys_time<Duration>& y) noexcept
```

Returns: `!(x == y)`.

```
template <class Duration> constexpr bool operator!=(const sys_time<Duration>& x, const leap& y) noexcept
```

Returns: `!(x == y)`.

```
template <class Duration> constexpr bool operator< (const leap& x, const sys_time<Duration>& y) noexcept
```

Returns: `x.date() < y`.

```
template <class Duration> constexpr bool operator< (const sys_time<Duration>& x, const leap& y) noexcept
```

Returns: `x < y.date()`.

```
template <class Duration> constexpr bool operator> (const leap& x, const sys_time<Duration>& y) noexcept
```

Returns: `y < x`.

```
template <class Duration> constexpr bool operator> (const sys_time<Duration>& x, const leap& y) noexcept
```

Returns: `y < x`.

```
template <class Duration> constexpr bool operator<=(const leap& x, const sys_time<Duration>& y) noexcept
```

Returns: `!(y < x)`.

```
template <class Duration> constexpr bool operator<=(const sys_time<Duration>& x, const leap& y) noexcept
```

Returns: `!(y < x)`.

```
template <class Duration> constexpr bool operator>=(const leap& x, const sys_time<Duration>& y) noexcept
```

Returns: `!(x < y)`.

```
template <class Duration> constexpr bool operator>=(const sys_time<Duration>& x, const leap& y) noexcept
```

Returns: `!(x < y)`.

[Back to TOC]

Add a new section 23.17.12.8 link [time.timezone.link]:

23.17.12.8 class `link` [time.timezone.link]

```
class link
{
private:
    string_view name_;    // exposition only
    string_view target_;  // exposition only

public:
    link(link&&)            = default;
    link& operator=(link&&) = default;

    // Unspecified constructors

    string_view name()   const noexcept;
    string_view target() const noexcept;
};

bool operator==(const link& x, const link& y) noexcept;
bool operator!=(const link& x, const link& y) noexcept;
bool operator< (const link& x, const link& y) noexcept;
bool operator> (const link& x, const link& y) noexcept;
bool operator<=(const link& x, const link& y) noexcept;
bool operator>=(const link& x, const link& y) noexcept;
```

A `link` is an alternative name for a `time_zone`. The alternative name is `name()`. The name of the `time_zone` for which this is an alternative name is `target()`. `link`s will be constructed when the time zone database is initialized.

`string_view link::name() const noexcept`

Returns: `name_`.

`string_view link::target() const noexcept`

Returns: `target_`.

`bool operator==(const link& x, const link& y) noexcept`

Returns: `x.name() == y.name()`.

`bool operator!=(const link& x, const link& y) noexcept`

Returns: `!(x == y)`.

`bool operator< (const link& x, const link& y) noexcept`

Returns: `x.name() < y.name()`.

`bool operator> (const link& x, const link& y) noexcept`

Returns: `y < x`.

`bool operator<=(const link& x, const link& y) noexcept`

Returns: `!(y < x)`.

`bool operator>=(const link& x, const link& y) noexcept`

Returns: `!(x < y)`.

Back to TOC

Modify the synopsis in section [fs.filesystem.syn] 30.10.6 Header `<filesystem>` synopsis:

```
using file_time_type = chrono::time_point<~~trivial-clock~~ chrono::file_clock>;
```

Back to TOC

Add to [thread.req.paramname] 33.2.1 Template parameter names:

1 Throughout this section, the names of template parameters are used to express type requirements. If a template parameter is named `Predicate`, `operator()` applied to the template argument shall return a value that is convertible to `bool`. If a template parameter is named `Clock`, the corresponding template argument shall be a type `C` for which `is_clock_v<C>` is `true`; otherwise the program is ill-formed.

Back to TOC

Acknowledgements

A database parser is nothing without its database. I would like to thank the founding contributor of the IANA Time Zone Database Arthur David Olson. I would also like to thank the entire group of people who continually maintain it, and especially the IESG-designated TZ

Coordinator, Paul Eggert. Without the work of these people, this software would have no data to parse.

I would also like to thank Jiangang Zhuang and Bjarne Stroustrup for invaluable feedback for the timezone portion of this library, which ended up also influencing the date.h library. Thanks also to Jonathan Wakely for agreeing to present this paper in Oulu for me. Thank you Daniel Krügler for the incredibly thorough review. Thank you Tomasz Kamiński for the very helpful changes to the proposed wording.

And I would also especially like to thank the growing list of contributors to this library.

References

1. N. Dershowitz and E. Reingold, *Calendrical Calculations 3rd ed.*, Cambridge University Press 2008.